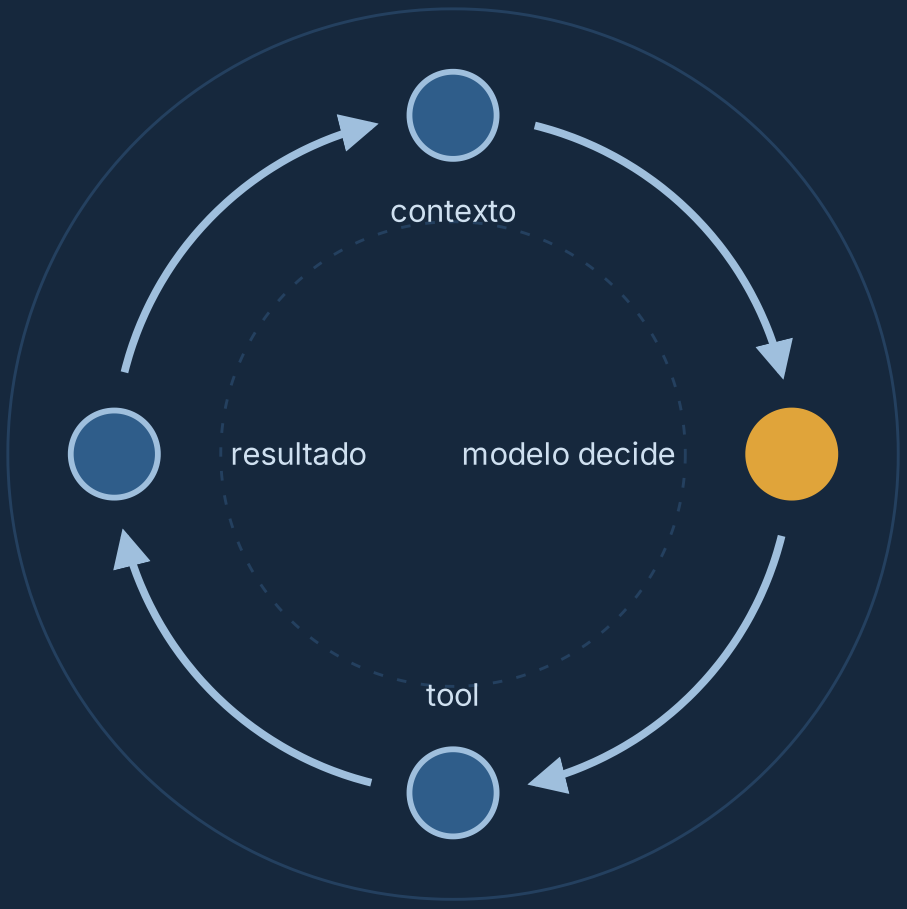
GUIA DE ESTUDO



IA First Dev

Agentes, Loops e Harness

Agentes de código, sub-agents, loops e harness

Outubro de 2026

IA First Dev

Agentes, Loops e Harness

Guia de estudo em português sobre desenvolvimento com agentes de IA: do contexto e das instruções do agente até sub-agents, loops de implementação e de review, e o harness que torna o resultado verificável.

COMO LER ESTE GUIA

TL;DR	O resumo do capítulo, logo na abertura.
FAÇA	Regras práticas para seguir.
EVITE	Regras práticas para não seguir.
ARMADILHA	O erro mais comum do tema, com o motivo.
NOTA	De onde vem um número, ou por que duas fontes divergem.
CARD	Um item de uma lista nomeada: problemas, princípios, pilares.
COMPLEMENTO	Texto de fonte externa (caps. 10 e 15), com autor, data e link. Fica sempre separado do conteúdo do curso.

Composto em Source Serif 4, Inter e JetBrains Mono (SIL Open Font License).
Diagramas redesenhados para este guia, numerados por capítulo. Outubro de 2026.

Sumário

PARTE I FUNDAMENTOS 4

1 LLM e agentes 5

2 Prompt e Context Engineering 9

3 Janela de contexto 15

PARTE II INSTRUMENTANDO O AGENTE 19

4 AGENTS.md e Rules 20

5 Skills 26

6 Tools, MCP e RAG 31

PARTE III DO PROBLEMA AO PLANO 36

7 O fluxo de desenvolvimento com IA 37

8 As primeiras tasks com IA 42

9 RPI e Spec-Driven Development 45

PARTE IV AGENTES, SUB-AGENTES E LOOPS 53

10 Anatomia do loop agêntico 54

11 Sub-agents: quando dividir e quanto 61

12 O loop de implementação 69

13 Harness Engineering 76

14 O loop de code review 82

15 Harness determinístico e Loop Engineering 89

PARTE V NA PRÁTICA 100

16 Fluxo de ponta a ponta 101

17 Playbook: montando seu primeiro loop de agentes 105

18 Kit de Skills 109

APÊNDICES 112

A Cheat sheets 113

B Glossário 117

C Referências 124

PARTE I

Fundamentos

1 LLM e agentes

2 Prompt e Context Engineering

3 Janela de contexto

I

CAPÍTULO 1

LLM e agentes

TL;DR

Uma LLM é uma máquina de prever o próximo pedaço de texto. Ela não lembra de nada entre uma chamada e outra: quem mantém a "conversa" é o **agente** (Claude Code, Cursor, Copilot), que a cada chamada reenvia o contexto inteiro ao **modelo** (Sonnet, GPT, Gemini). **O agente orquestra; o modelo decide.** Todo token do contexto entra com o mesmo peso, e quando o contexto fica grande demais, o que importa se perde no meio do resto. Daí a regra que atravessa o livro: contexto pequeno, focado e isolado rende mais do que contexto grande.

Usuário antes de builder

Existem dois jeitos de um dev trabalhar com IA. O **builder** constrói agentes: escreve o software que chama modelos, monta ferramentas, projeta fluxos. O **usuário** usa bem os modelos, agentes e editores que já existem para entregar o próprio trabalho. O curso parte de uma prioridade clara: **primeiro resolver o problema como usuário, depois como builder.** É como usuário que a maioria dos devs vai trabalhar com IA todos os dias, e é aí que mora a diferença imediata de produtividade e qualidade.

Também vale separar duas atitudes. O *vibe coding* (pedir, aceitar e seguir em frente, sem olhar muito o que saiu) serve para clonar um produto rápido. O **desenvolvimento assistido por IA** é outra coisa: usar a IA para aplicar as boas práticas de engenharia com mais qualidade e mais velocidade. O padrão que a indústria está buscando é um **bom fluxo** em que o dev quase não precisa escrever código à mão. O centro do trabalho deixa de ser digitar e passa a ser montar o fluxo: o contexto, as regras, a verificação.

Para isso não é preciso saber treinar um modelo. Bastam fundamentos de alto nível, o suficiente para entender as limitações: o que é uma LLM e como ela processa a entrada; tokens, *embeddings* e janela de contexto; a diferença entre modelos fechados e abertos; como escrever pedidos e montar contexto; editores, agentes e suas integrações; e o fluxo de trabalho. Este capítulo cobre os primeiros itens. O resto do livro, os demais.

O que é uma LLM, no nível que importa

Uma *LLM* (*Large Language Model*, modelo de linguagem de grande escala) é, no fundo, uma **máquina de predição da próxima palavra**. Ela foi treinada com um volume enorme de texto aberto da internet e aprendeu, a partir dele, qual continuação é mais provável para um texto qualquer. O ciclo é simples de descrever: entra texto (*input*), o modelo processa, sai texto (*output*).

Alguns conceitos ajudam a entender o que acontece no meio:

- **Token.** O modelo não lê palavras, lê *tokens*: pedaços de texto (uma palavra curta, parte de uma palavra longa, um sinal de pontuação). A *tokenização* é a etapa que quebra o texto nesses pedaços. Tudo na IA é medido em tokens: o tamanho do que cabe numa chamada, o custo, a velocidade.
- **Embedding.** Cada token vira uma representação numérica, um vetor. É sobre esses números que o modelo calcula. A mesma ideia de transformar texto em vetores reaparece na busca semântica do cap. 6.
- **Transformer e attention.** O *transformer* é a arquitetura dos modelos atuais, e o mecanismo de *attention* (atenção) é o núcleo do cálculo: é ele que mede quanto cada token se relaciona com cada outro.

Existem modelos fechados, acessados pela API de quem os criou, e modelos abertos, que podem rodar na sua própria infraestrutura. O que este livro ensina vale para os dois.

Como o contexto é processado

O *contexto* é tudo o que o modelo recebe numa chamada. A forma como ele é processado explica boa parte do que funciona e do que não funciona com IA:

1. **Todos os tokens entram iguais.** Não existe um jeito de marcar manualmente "esta linha é mais importante".
2. O modelo **calcula as relações entre os tokens** via attention.
3. Os tokens mais conectados com o resto ganham **destaque de forma implícita**.
4. Se o contexto for gigante, as partes relevantes ficam cercadas de partes irrelevantes e **se perdem**.

Esse último efeito tem nome: *context dilution* (diluição do contexto). Encher o contexto "por garantia" não ajuda o modelo; atrapalha. A informação certa continua lá, mas pesa menos.

A janela de contexto

A *janela de contexto* é o limite de tokens que o modelo consegue considerar numa chamada. Nela entra tudo: o seu prompt, os arquivos lidos, as respostas de MCPs, as skills carregadas, a documentação do projeto e a conversa até ali.

A recomendação que nasce daqui é contraintuitiva: **quanto menor a janela usada, melhor o output e menor a chance de alucinação.** Por isso o curso insiste em trabalhar em **contextos isolados**, uma janela por assunto, em vez de uma única conversa longa que acumula tudo. O cap. 3 mostra as faixas da janela em que a qualidade se mantém e em que ela cai.

Limitações que não somem

Quatro limitações das LLMs vão aparecer o tempo todo, e cada uma tem uma resposta em algum capítulo:

- **Alucinação:** o modelo produz algo plausível e errado com a mesma confiança com que produz algo certo. A resposta é verificar com algo que não seja o próprio modelo (caps. 2 e 12).
- **Conteúdo desatualizado:** o modelo só sabe o que estava nos dados de treino. Versões novas de bibliotecas não existem para ele. A resposta é buscar documentação atualizada, por exemplo com o Context7 (cap. 4).
- **Sensibilidade ao prompt:** pequenas mudanças no pedido mudam bastante a resposta. A resposta é prompt e context engineering (cap. 2).
- **Perda de contexto em janelas grandes:** a diluição da seção anterior. A resposta é gerir a janela (cap. 3).

O agente orquestra, o modelo decide

No dia a dia, "a IA" mistura duas coisas diferentes. O **agente** é o programa com que você interage: Claude Code, Cursor, Copilot. O **modelo** é a LLM que o agente chama por trás: Sonnet, GPT-4, Gemini. A divisão de trabalho é esta: **o agente orquestra, a LLM decide.**

O detalhe que muda tudo é que o modelo é *stateless* (sem estado). Ele não guarda memória entre uma chamada e outra. **Toda vez, o agente passa o contexto completo para a LLM**: as instruções do sistema, as regras do projeto, a conversa inteira, os arquivos lidos, a saída dos comandos. O modelo lê esse pacote, decide o próximo passo e devolve a decisão. O agente executa o que foi decidido (ler um arquivo, editar, rodar um comando), junta o resultado ao pacote e chama o modelo de novo.

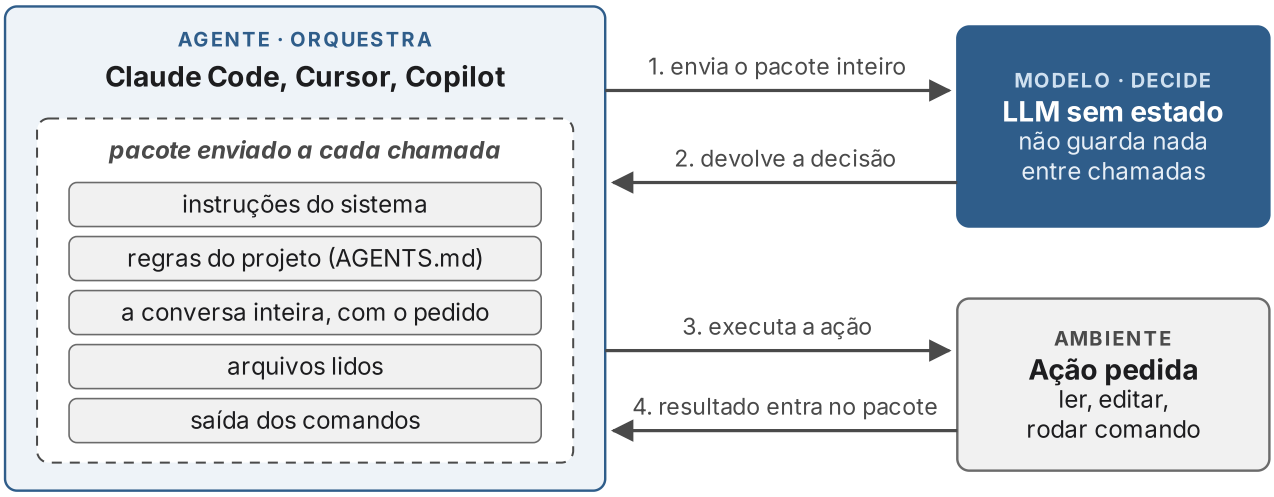


Figura 1.1 O agente orquestra e o modelo decide: a cada chamada, o agente reenvia o contexto inteiro a um modelo sem estado e executa a ação que ele pediu

Duas consequências práticas:

- **A "memória" da conversa é do agente, não do modelo.** Se algo não está no pacote enviado, para o modelo não existe. Uma janela nova começa do zero.
- **Melhorar o resultado quase nunca é trocar de modelo.** Quase sempre é mudar o que entra no pacote. É disso que tratam as Partes II e III.

O cap. 10 volta a esse ciclo e o trata como um loop, com tudo o que isso implica.

Como o contexto evoluiu

A forma de entregar contexto ao modelo mudou rápido, e cada fase deixou uma ferramenta que este livro usa:

Fase	O que mudou	Onde aparece no livro
A partir de 2022	RAG e busca sob demanda: buscar trechos relevantes e colocá-los no contexto	cap. 6
Em seguida: a era do contexto grande	janelas de 100 mil a 200 mil tokens: "coloque tudo"	cap. 3
2024	tools e MCPs: o modelo pede ações e dados externos	cap. 6
2025	progressive disclosure e `AGENTS.md` como padrão: carregar só o necessário, quando necessário	caps. 4 e 5
Próximo passo	contextos adaptativos e orquestração de vários agentes	Parte IV

A direção é clara: de "mais contexto" para "o contexto certo, na hora certa".

FAÇA

+ Trate cada janela como um contexto isolado, com um assunto só.
+ Passe ao modelo o que ele precisa para a tarefa, e nada além disso.
+ Verifique o que o modelo produziu com algo externo a ele (testes, comandos, tipos).
+ Lembre que uma janela nova começa sem nada da conversa anterior.

EVITE

– Encher o contexto "por garantia" com arquivos e documentação que a tarefa não pede.
– Confiar em conhecimento de bibliotecas que podem ter mudado depois do treino do modelo.
– Trocar de modelo para resolver um problema que é de contexto.
– Usar a mesma conversa para tudo, de manhã até a noite.

ARMADILHA

Achar que o modelo lembra. Você corrigiu uma convenção do projeto vinte mensagens atrás, abriu uma janela nova para a próxima tarefa, e o erro voltou. Não é teimosia: o modelo nunca soube. Quem lembrava era a conversa antiga, e ela ficou para trás. O que precisa valer em toda janela tem de estar num lugar que o agente carrega sempre, como o `AGENTS.md` do cap. 4.

CAPÍTULO 2

Prompt e Context Engineering

TL;DR

Prompt engineering é a base: exemplos concretos, persona, restrições claras, iteração e validação. Mas no desenvolvimento de software o prompt é a menor parte do que o modelo recebe. *Context engineering* é cuidar de todo o resto: arquivos, erros completos, versões, convenções do projeto e o próprio codebase. A regra prática é **documentation first**: regras curtas e objetivas sempre carregadas, documentação detalhada carregada sob demanda, mudanças pequenas, uma janela por tarefa, e cada erro da IA transformado em melhoria das regras.

Prompt engineering: a base de tudo

O *prompt* é o pedido que você escreve. Antes de pensar em contexto, vale dominar cinco técnicas que mudam bastante a qualidade da resposta.

Exemplos concretos

Mostrar dois ou três exemplos do que se espera funciona melhor do que descrever em abstrato. Essa técnica se chama *few-shot* (poucos exemplos): o modelo infere o padrão a partir deles. Para código, o exemplo pode ser um teste existente do projeto, um endpoint parecido ou um commit que seguiu a convenção certa.

Persona

Pedir que a IA "aja como" alguém específico afunila o tom, o estilo e a profundidade da resposta. "Você é um principal engineer avaliando esta decisão de arquitetura" produz uma análise diferente de "o que você acha disso?". A técnica vale fora do código também:

Persona	Pedido
Professor de matemática	Explique este problema de álgebra como se estivesse ensinando alunos do ensino médio.
Editor de texto	Revise este artigo como um editor profissional, com foco em clareza e gramática.
Consultor de negócios	Analise este plano como um consultor de startups.
Especialista em marketing digital	Sugira melhorias nesta campanha como um especialista em marketing digital.

Restrições

Restrições (*constraints*) são limites e requisitos explícitos que guiam a saída. Sem elas, o modelo escolhe o caminho mais comum, que muitas vezes não é o do seu projeto.

Tipo de restrição	Exemplo
Bibliotecas	"Use apenas React com Hooks, sem Redux."
Performance	"O componente deve renderizar em menos de 50 ms."
Compatibilidade	"O código precisa funcionar no React 18 com Vite."
Estilo	"Siga a convenção camelCase para props e funções."
Acessibilidade	"Todos os botões devem ter rótulos para leitores de tela."

Iterar e refinar

Prompt engineering é um processo iterativo. A primeira resposta raramente é a final: revise o que veio e devolva um feedback corretivo ou uma pergunta de acompanhamento, enriquecendo o contexto a cada volta. Alguns refinamentos típicos:

- "Está claro, mas dá para simplificar a explicação para alguém iniciante?"
- "Reescreva esse código usando outro paradigma, como programação funcional."
- "Inclua exemplos de uso prático."
- "Adapte a solução para funcionar também em dispositivos móveis."

Validação como parte do pedido

A técnica mais importante para código é pedir que a própria IA **valide o que produziu**, em etapas, para pegar erros cedo. Um pedido bem montado já traz a validação:

```
Crie uma API de login que:
1. valide se o usuário existe no banco;
2. bloqueie várias tentativas falhas consecutivas;
3. retorne mensagens de erro específicas e seguras.

Depois de escrever a API, valide:
- teste com credenciais corretas e incorretas;
- confirme que o bloqueio funciona após várias tentativas inválidas;
- verifique se segue as práticas de segurança do time (JWT, HTTPS).
```

O ponto central é **em que confiar**. A validação boa roda testes e comandos no terminal e olha o resultado binário: passou ou não passou. A validação ruim é pedir que a IA "avalie se ficou bom", porque o julgamento subjetivo do modelo sobre o próprio trabalho tende a ser favorável. Essa ideia vira o quality gate do cap. 9 e o loop de implementação do cap. 12.

Context engineering: indo além do prompt

Num chat comum, o prompt é quase tudo o que o modelo recebe. No desenvolvimento de software, não. O contexto inclui os arquivos do projeto, os logs de erro, os *stack traces* (a pilha de

chamadas registrada quando o erro aconteceu), as respostas de APIs, as versões das bibliotecas e referências externas. Context engineering é a disciplina de montar esse pacote.

Seja preciso e específico

Pedidos vagos geram respostas vagas. Na prática, precisão é:

- incluir as **versões exatas** dos frameworks;
- especificar os **padrões e convenções** de código do projeto;
- definir com clareza os **critérios de sucesso**;
- fornecer **mensagens de erro completas** e stack traces.

Monte o contexto do pedido

Para um bug, o contexto mínimo tem os arquivos centrais, os arquivos relacionados, a estrutura dos dados envolvidos, o erro completo e o objetivo:

```
// Arquivo atual: user-service.js
// Relacionados: user-model.js, auth-middleware.js
// Schema: estrutura da tabela users
// Erro: [stack trace completo]
// Objetivo: corrigir o bug de autenticação de usuário
```

E o erro precisa vir **inteiro**: o stack trace completo (não só a mensagem), os erros do console do navegador no frontend, os logs do servidor no backend e os detalhes das requisições e respostas de rede. Cortar o erro para "economizar" obriga o modelo a adivinhar a parte que faltou.

Explique o objetivo antes do código

A diferença entre um pedido ruim e um bom costuma ser uma frase de contexto de negócio:

- Ruim: "Crie um serviço que calcula impostos."
- Bom: "No módulo `billing`, precisamos de um serviço que calcule impostos para notas fiscais. Ele deve usar a integração existente com o Stripe e **seguir as boas práticas do projeto para integrações**."

O segundo pedido diz onde o código mora, para que ele serve, o que deve ser reaproveitado e qual padrão seguir. O último trecho, em negrito, só funciona se essas "boas práticas para integrações" estiverem escritas em algum lugar que a IA consiga ler. É o assunto da próxima seção.

Documentation first: as regras como contexto

Repetir o mesmo contexto em todo pedido não escala. A saída é escrever, uma vez, um arquivo de regras que o agente carrega sozinho: as *rules* do Cursor ou o `AGENTS.md` (cap. 4). O curso chama essa postura de ***documentation first***, ou desenvolvimento guiado por contexto. O arquivo traz o contexto do projeto, a arquitetura, as convenções e exemplos mínimos de código.

Um exemplo de rules para um sistema de streaming:

```
## Domínio
Sistema de streaming. User = assinante; Subscription = plano;
Movie = conteúdo assistível.

## Arquitetura
Monólito modular. apps/ = APIs e workers; libs/domain/ = lógica
por domínio; libs/infra/ = infraestrutura compartilhada.

## Convenções
Branches: feature/, bugfix/, hotfix/.
Eventos: <Aggregate>.<Action>.<PastTense>.

## Geração de código
Priorize testes de unidade. Nunca use `any` em TypeScript.
```

O arquivo funciona quando é curto, concreto e atualizado. Ele falha quando vira um tratado: teoria demais, documentação inteira copiada, regras que ninguém segue, informação desatualizada, termos vagos como "mantenha a qualidade" sem dizer como, e regras para cada detalhe que muda toda semana. O limite de tamanho tem números diferentes nas aulas, e o cap. 4 explica cada um.

Carregar sob demanda

Nem tudo cabe nas regras, e nem tudo precisa estar lá. A técnica que resolve isso é a *progressive disclosure* (revelação progressiva): o arquivo de regras **aponta** para documentos mais detalhados, e a IA só os lê quando a tarefa pede.

```
---
description: Princípios de arquitetura
alwaysApply: true
---
# Princípios de arquitetura
Apps = bootstrap (só orquestração). Packages = regra de negócio.
Modules = domínios independentes e combináveis.
Detalhes completos: docs/ARCHITECTURE-GUIDELINES.md
```

A regra curta fica sempre no contexto; o arquivo longo entra só quando a IA decide que precisa dele. O cap. 4 transforma essa ideia em uma árvore de decisão, e o cap. 5 mostra que as skills funcionam do mesmo jeito.

O codebase é contexto

O código do projeto é a maior fonte de contexto que a IA tem, e a forma como ele está organizado pesa muito. A tese do curso é que, para trabalhar com IA, **a modularização importa mais do que microserviços**. Um monólito bem dividido em módulos (por exemplo, `billing`, `streaming` e `identity` dentro do mesmo repositório) permite que uma mudança atravesse vários módulos de forma atômica, numa única sessão. Microserviços em repositórios separados quebram esse fluxo: o agente não enxerga o outro lado.

Organizar para a IA é, em grande parte, organizar para pessoas:

- pastas por domínio ou *bounded context* (a fronteira de um subdomínio do negócio);
- nada de pastas genéricas como `utils`, `common` ou `misc`;
- nada de estruturas que misturam camadas e domínios;
- padrão de nomes consistente;
- código sem duplicação espalhada;
- README curto, sem teoria que ninguém lê.

Trabalhe em blocos

Os cuidados do dia a dia têm um tema comum: **mudanças pequenas e diretas**.

- Use **uma janela para planejar e outra para implementar**. O plano vira um texto curto; a implementação começa com o contexto limpo. (O cap. 9 formaliza isso como RPI.)
- **Troque de janela quando mudar de tarefa.** A conversa anterior vira ruído para a próxima.
- **Não tente resolver tudo num único prompt gigante.**
- **Não peça "otimize" ou "melhore" sem dizer o que isso significa.** Otimizar o quê: tempo de resposta, memória, legibilidade?
- **Não peça código sem explicar o contexto de negócio**, nem "refatora esse código" sem direção.
- **Revise o que foi gerado** antes de aceitar.

Conversas longas sofrem de *context rot* (apodrecimento do contexto): à medida que o histórico cresce, a precisão cai, porque instruções antigas, tentativas abandonadas e saídas de comandos irrelevantes continuam no pacote. Trabalhar em blocos é a defesa mais barata contra isso.

Feedback e aprendizado contínuo

A documentação para IA nunca fica pronta. Ela melhora cada vez que a IA erra:

1. **Explique o que estava errado, não só corrija.** "Esse código usa `any`, mas no nosso projeto sempre definimos tipos explícitos. Refaça usando a interface `OrderDto`." A explicação ensina a regra; a correção silenciosa, não.
2. **Adicione bons exemplos ao repositório.** O modelo copia o que vê.
3. **Atualize as regras e os READMEs com frequência.** Depois de uma boa refatoração: "Com base nessa refatoração, extraia os pontos-chave (usar `Promise.all` sempre que possível e deixar a transação do banco para o final), crie exemplos e atualize a documentação."
4. **Use os reviews de PR como aprendizado.** Agentes que leem o histórico do git aproveitam as correções que o time já fez.

Em projetos legados, a ordem importa: **primeiro mapeie o codebase como ele é**, com os padrões que ele realmente tem, e só depois tente mudar. Regras que descrevem um projeto ideal, e não o real, confundem a IA.

FAÇA

+ Dê exemplos concretos, persona e restrições explícitas.
+ Peça validação por testes e comandos, com resultado binário.
+ Envie o erro completo: stack trace, console, logs e rede.
+ Explique o objetivo de negócio antes de pedir código.
+ Mantenha um arquivo de regras curto, com exemplos reais, e atualize-o sempre que algo importante mudar.
+ Organize o código por domínio.
+ Uma janela para planejar, outra para implementar; janela nova a cada tarefa.
+ Transforme cada erro recorrente da IA em uma regra ou um exemplo.

EVITE

– Pedidos vagos: “otimize”, “melhore”, “refatora isso”.
– Pedir que a IA julgue a qualidade do próprio trabalho.
– Copiar documentação gigante para o arquivo de regras.
– Termos vagos como “mantenha a qualidade” sem dizer como.
– Pastas `utils`, `common`, `misc`.
– Um prompt enorme tentando resolver tudo de uma vez.
– Aceitar código gerado sem revisar.

ARMADILHA

Corrigir sem ensinar. A IA errou, você corrigiu o código à mão e seguiu em frente. Na próxima tarefa, o mesmo erro volta, porque nada no contexto mudou. Cada correção manual que não vira uma frase nas regras, um exemplo no repositório ou um teste é um erro que vai se repetir.

CAPÍTULO 3

Janela de contexto

TL;DR

A janela de contexto é limitada e o modelo não tem memória fora dela. Mas o limite que importa no dia a dia não é o tamanho máximo da janela: é a fração dela que você já ocupou. Até cerca de 40%, a qualidade é alta (**Smart Zone**). Entre 40% e 60%, ela já cai (**zona de atenção**). Acima de 60%, a chance de alucinação dispara (**Dumb Zone**). O objetivo é implementar sempre dentro da Smart Zone, e boa parte das técnicas do livro existe para isso.

Um recurso pequeno e sem memória

Os dois fatos do cap. 1 se combinam aqui. O modelo é stateless, uma "memória de peixinho dourado": tudo o que ele sabe sobre a sua tarefa está no pacote enviado naquela chamada. E esse pacote tem um teto, a janela de contexto, medida em tokens.

Uma frase do slide da aula resume a mudança de atitude que o capítulo pede: **o problema não é a IA, é como você está usando a janela de contexto.** Quando a resposta piora no meio de uma sessão longa, quase sempre o modelo é o mesmo de antes. O que mudou foi o quanto da janela já está ocupado.

As três zonas

A aula divide a janela em três faixas, pela porcentagem já ocupada:

Zona	Faixa	O que acontece
Smart Zone	0–40%	qualidade alta; é onde a implementação deve acontecer
Zona de atenção	40–60%	qualidade já degradada; hora de pensar em trocar de janela
Dumb Zone	60–100%	muito contexto, qualidade baixa e alta chance de alucinação

No slide, a seta "você está trabalhando aqui" aponta para perto de 80%, já na Dumb Zone. É o retrato de uma sessão comum: a conversa começou com uma pergunta, virou um plano, depois uma implementação, depois três rodadas de correção, e ninguém trocou de janela.

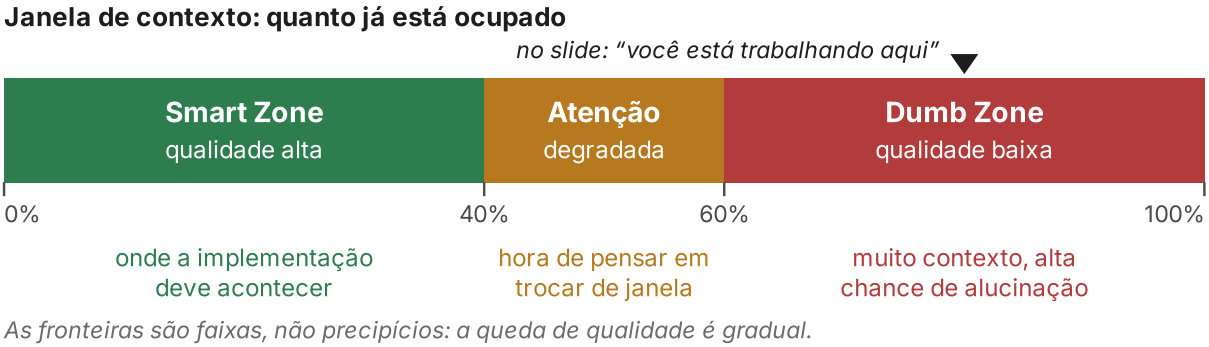


Figura 3.1 As três zonas da janela de contexto: Smart Zone até 40%, zona de atenção entre 40% e 60%, Dumb Zone acima de 60%

As zonas são uma consequência direta da context dilution do cap. 1. À medida que a janela enche, cada informação relevante fica cercada de mais informação irrelevante e pesa menos no cálculo do modelo. A queda é gradual, por isso as fronteiras são faixas, não precipícios.

> **NOTA**
>
> **40%, 60% ou 80%?** Três números aparecem para a fronteira da zona ruim, e cada um tem o seu contexto.
>
> - **40% e 60%** são as fronteiras da barra do slide e do texto da aula de RPI: acima de 40% a qualidade já cai; acima de 60% é Dumb Zone. São as que este livro usa.
> - **">40% = Dumb Zone"** aparece num cartão do mesmo slide. É uma simplificação que junta a zona de atenção com a Dumb Zone, para reforçar a mensagem de que 40% já é o momento de agir.
> - **~80%** é a linha de risco do gráfico do benchmark de sub-agents (cap. 11). Ela mede outra coisa: o risco de a janela estourar numa implementação longa, não o ponto em que a qualidade começa a cair.

Quanto cabe numa janela

O tamanho absoluto da janela varia com o modelo e com a ferramenta, e as aulas citam valores diferentes.

> **NOTA**
>
> **200k, ~168k, ~950k ou 1M?** Os quatro números aparecem nas aulas.
>
> - **200 mil tokens** é o valor que a aula de fundamentos chama de "o ideal" para uma janela de contexto. Vale como ordem de grandeza da janela padrão dos modelos usados no curso.
> - **~168 mil tokens** é o valor do slide da aula de RPI. A leitura deste livro (interpretação, não dado da aula) é que se trata do espaço útil de uma janela nominal de 200 mil, depois do que o próprio agente reserva para instruções e para a resposta.
> - **~950 mil tokens** aparece no mesmo slide como janela "nova", e **1 milhão** é o tamanho do Max Mode do Cursor citado na aula sobre ferramentas (cap. 6). São as janelas grandes que alguns modelos passaram a oferecer.
>
> O que não muda entre eles são as porcentagens. Uma janela maior cabe mais coisa antes de chegar a 40%, mas a diluição continua acontecendo à medida que ela enche.

Uma janela maior ajuda a pesquisar: ler documentação, explorar o codebase, juntar material de várias fontes. Não é licença para implementar numa conversa que já está acima de 60%.

Como ficar na Smart Zone

Quase todo o resto do livro é, de alguma forma, uma técnica para manter a janela baixa durante a implementação:

- **Contexto pequeno de propósito:** pedidos precisos e mudanças em blocos (cap. 2).
- **Carregar só quando precisa:** regras curtas que apontam para documentos detalhados (cap. 4) e skills que entram só quando a tarefa casa com elas (cap. 5).
- **Separar pesquisa de implementação:** pesquisar numa janela, salvar o plano num arquivo e implementar numa janela nova (RPI, cap. 9).
- **Delegar a outras janelas:** sub-agents que trabalham num contexto próprio e devolvem só o resultado (cap. 11). Na implementação Spec-Driven do cap. 12, a janela principal não passou de 37%.

> **FAÇA**
>
> + Acompanhe a porcentagem da janela, não só o tamanho máximo.
> + Implemente dentro da Smart Zone (até ~40%).
> + Na zona de atenção, salve o estado num arquivo e abra uma janela nova.
> + Use janelas grandes para pesquisar, não para implementar.

> **EVITE**
>
> – Implementar ou corrigir com a janela acima de 60%.
> – Tratar uma janela de 1 milhão de tokens como solução para a diluição.
> – Insistir na mesma conversa depois que as respostas começaram a piorar.

⚠ ARMADILHA

A rodada de correção na Dumb Zone. A implementação terminou com a janela em 70%. Falta só um ajuste, então você pede na mesma conversa. O ajuste vem errado, você pede outro, e cada pedido empurra a janela mais para cima. A correção que parecia pequena é feita justamente no pior momento da sessão. Quase sempre sai mais barato registrar o que falta num arquivo e corrigir numa janela nova.

PARTE II

Instrumentando o agente

4 AGENTS.md e Rules

5 Skills

6 Tools, MCP e RAG

II

CAPÍTULO 4

AGENTS.md e Rules

TL;DR

O `AGENTS.md` (ou as rules do Cursor, ou o `CLAUDE.md`) é o único texto do projeto que vai em **toda** requisição, junto com o system prompt e a conversa. Tudo o que está nele é pago sempre, em tokens e em atenção. Por isso ele precisa ser enxuto: comandos exatos, boundaries em três níveis (sempre faça, pergunte antes, nunca faça), regras testáveis e ponteiros para documentos lidos sob demanda. Paths, conteúdo autogerado e frases vagas ficam de fora. Construa o arquivo aos poucos, a partir dos erros reais da IA, e não gerado por ela.

O arquivo que vai em toda requisição

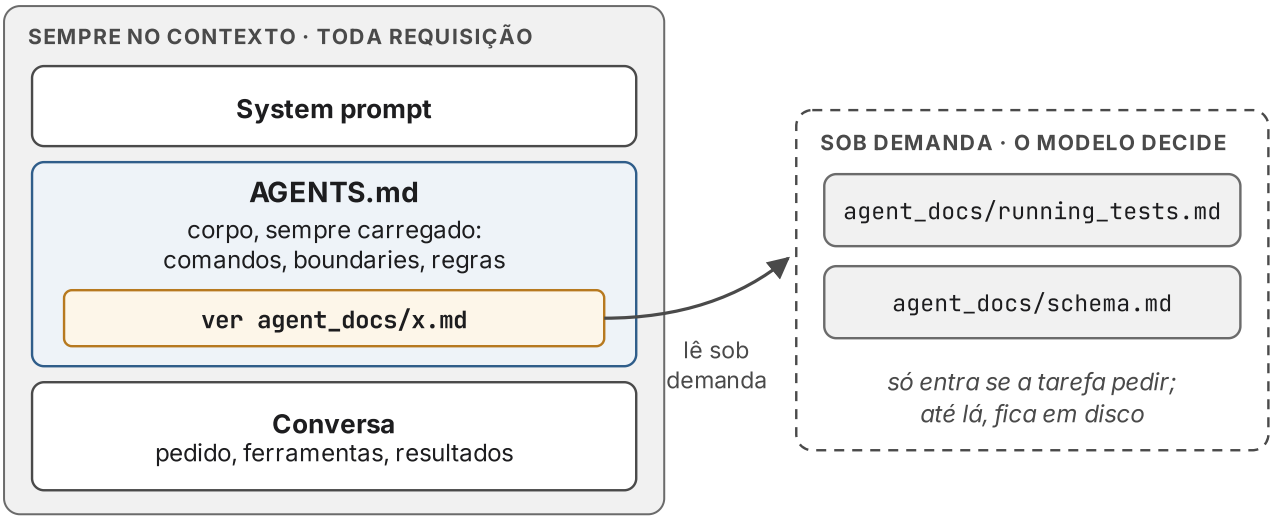
Fazer o *onboarding* da IA no projeto é parecido com fazer o de uma pessoa: explicar a stack, a arquitetura, as regras do time e onde encontrar o resto. A diferença é que a IA esquece tudo a cada janela nova (cap. 1). O onboarding precisa estar escrito num arquivo que o agente carrega sozinho.

Esse arquivo tem vários nomes. O `AGENTS.md` é o padrão aberto, um Markdown na raiz do repositório que vários agentes leem. O Cursor usa as suas rules; o Claude Code, o `CLAUDE.md`. Este livro chama todos de `AGENTS.md`, porque o que vale para um vale para os outros.

O que torna esse arquivo especial é **quando** ele entra. A cada requisição, o agente envia ao modelo três blocos:

1. o **system prompt** da ferramenta;
2. o **`AGENTS.md`**, sempre;
3. a **conversa**: o seu prompt, as ferramentas disponíveis, os arquivos lidos.

O `AGENTS.md` não é lido uma vez no começo da sessão. Ele é reenviado em toda chamada. O tamanho dele pesa no custo e no desempenho de todas as interações, e cada linha dele disputa atenção com a tarefa do momento. O subtítulo do slide da aula diz tudo: **tudo é pago em toda requisição, então seja seletivo.**



Enxugar o AGENTS.md é trocar texto inline por um ponteiro + um arquivo lido sob demanda.

Figura 4.1 O que vai em cada requisição (system prompt, AGENTS.md e conversa) e o que fica em disco até o modelo decidir ler

O que entra e o que fica de fora

Entra (ganha a linha)	Fica de fora (dilui)
comandos exatos, com flags	estrutura de pastas e paths
boundaries: sempre, pergunte, nunca	conteúdo autogerado (`/init`)
tooling do projeto (uv, bun...)	"escreva código limpo" e outras frases vagas
regras testáveis (formato de chaves, uso de `Decimal`)	instruções de uma tarefa específica
exemplos reais de 3 a 10 linhas	o que já está no README ou na wiki
ponteiros para arquivos e docs	qualquer coisa que leve o arquivo além do limite

Uma regra é **testável** quando dá para olhar o código e dizer se ela foi seguida. "Valores monetários usam `Decimal`" é testável. "Escreva código limpo" não é: o modelo já tenta fazer isso, e a frase só ocupa espaço.

A coluna da direita tem um item que parece contradizer o cap. 2, onde o exemplo de rules descrevia a arquitetura. Não contradiz. A **arquitetura em alto nível** entra: "monólito modular; `apps/` só orquestra, a regra de negócio fica nos módulos". A **listagem de paths** não entra: o agente descobre a árvore de pastas sozinho, e uma listagem copiada envelhece a cada refatoração.

Na dúvida, siga o conselho do slide: **corte.** O jeito mais rápido de melhorar um `AGENTS.md` costuma ser deletar.

> **NOTA**
>
> **100 linhas, 150 linhas ou 1 a 2 páginas?** As aulas dão três formas de expressar o limite de tamanho, e elas não se contradizem.
>
> - **100 linhas** vem da aula de prompt e context engineering: rules acima disso "já são sinal de problema". É um alarme: passou daqui, revise.
> - **~150 linhas** vem da aula dedicada ao `AGENTS.md`, que fala no "custo real de passar de 150 linhas". É o teto: acima disso, o arquivo está custando mais do que entrega.
> - **1 a 2 páginas** vem do slide de documentation first. É a mesma ordem de grandeza, medida de outro jeito.
>
> Na prática: mire em menos de 100 linhas e trate 150 como limite.

Boundaries: o padrão de três níveis

Sem limites explícitos, o agente cai no comportamento padrão, que pode não ser o do seu time. Os *boundaries* (limites) organizam o que o agente pode fazer em três níveis:

Sempre faça	Pergunte antes	Nunca faça
rodar os testes antes do PR	mudar schema ou migrations	push direto na `main`
corrigir o lint	adicionar dependências	rodar migration em produção
dar stage só no que foi pedido	mexer em credenciais	expor dados pessoais (PII) no output

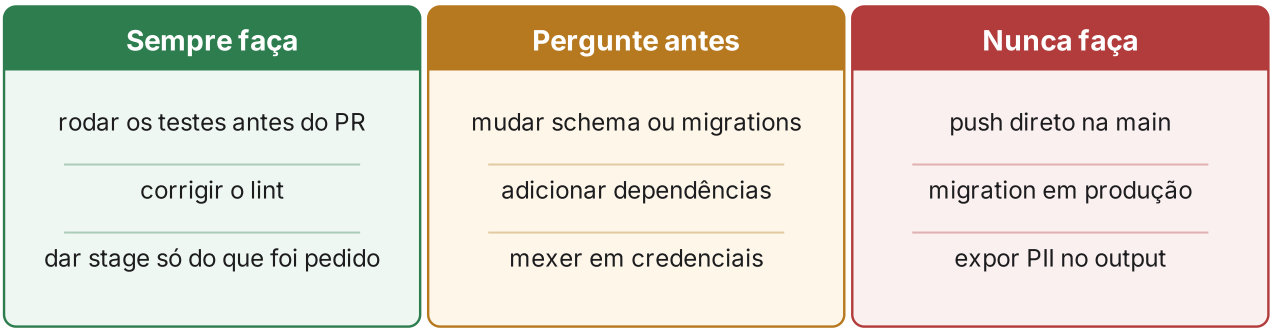


Figura 4.2 Boundaries em três níveis: sempre faça, pergunte antes e nunca faça, cada um com exemplos

Duas recomendações da aula sobre como escrevê-los:

- **No início do arquivo, em caixa alta.** É a parte que mais importa, e deve ser a primeira coisa que o modelo lê.
- **Pareie cada "não faça X" com um "faça Y".** Uma lista só de proibições deixa o agente conservador e travado: ele sabe o que não pode, mas não sabe o que deve fazer no lugar. "Nunca crie migrations à mão; gere com `npm run migration:generate`" funciona melhor do que só a primeira metade.

Regras como restrições permanentes

Além dos boundaries, o arquivo é o lugar das restrições que valem para qualquer tarefa do projeto. Alguns exemplos da aula:

- **Planos:** sempre incluir links para a documentação usada; sempre rodar determinados comandos antes de concluir.
- **Testes:** sempre cobrir as partes importantes; incluir testes *end-to-end* (E2E, o fluxo completo como o usuário faria) para os caminhos felizes **e** para os cenários difíceis.
- **Banco de dados:** nunca criar migrations manuais; usar sempre os comandos do projeto.

Sempre carregado vs sob demanda

Se o arquivo precisa ser curto, onde fica todo o resto? Em documentos que o modelo só lê quando precisa. É a progressive disclosure do cap. 2 aplicada ao `AGENTS.md`: o corpo do arquivo vai sempre; um documento referenciado vai só quando o modelo decide. É a parte "sob demanda" do diagrama do início do capítulo.

O truque está no ponteiro. Uma linha como "para rodar os testes de integração, leia `agent_docs/running_tests.md`" custa uma linha em toda requisição. O arquivo apontado, com as suas cem linhas, só entra quando a tarefa envolve testes de integração. **Enxugar o `AGENTS.md` é trocar texto inline por ponteiro mais arquivo lido sob demanda.**

Uma árvore de decisão para a documentação

O ponteiro funciona melhor quando diz **quando** ler. Em vez de uma lista solta de links, o arquivo traz uma pequena árvore de decisão:

```
## Documentação sob demanda
- Criando controllers ou services → leia docs/code-patterns.md
- Integrando com APIs externas → leia docs/integration-patterns.md
- Mexendo na arquitetura dos módulos → leia docs/modular-architecture.md
- Refatoração grande, performance ou consolidação de lógica
  espalhada → leia docs/REFACTORING-GUIDELINES.md
```

Foi isso que tornou possível o pedido do cap. 2: "siga as boas práticas do projeto para integrações". A frase funciona porque existe um documento de integração e uma regra que diz quando abri-lo.

Skills pelo nome, não pelo caminho

O mesmo vale para as skills (cap. 5). O `AGENTS.md` pode trazer guias de decisão básicos e indicar qual skill usar em cada ação: "para criar um módulo novo, use a skill `modular-architecture`". Referencie **o nome** da skill, não o caminho do arquivo. O agente já sabe encontrá-la pelo nome, e o caminho muda de ferramenta para ferramenta.

E quando uma seção do `AGENTS.md` começa a crescer, a aula dá um critério simples: **passou de ~20 linhas, vira skill.** (Não confundir com o limite de ~20 **skills** do cap. 5: um é o tamanho de uma seção, o outro é a quantidade de skills instaladas.)

Documentação de bibliotecas: Context7

O modelo foi treinado com dados de uma certa época e não conhece as versões mais novas das bibliotecas (cap. 1). O Context7 é um MCP (cap. 6) que busca a documentação atualizada de bibliotecas de terceiros. Uma linha no `AGENTS.md` resolve o resto: "ao gerar código que envolva setup, configuração ou bibliotecas externas, consulte o Context7".

AGENTS.md por diretório

Também é possível ter arquivos `AGENTS.md` em subpastas, com regras que valem só para aquele módulo. É útil em sistemas legados, em que partes diferentes seguem padrões diferentes: o módulo antigo tem as suas regras, o novo tem as dele. O custo é a manutenção. Cada arquivo a mais é um arquivo que pode ficar desatualizado sem ninguém perceber.

Construa de forma iterativa

A tentação é pedir à IA que gere o `AGENTS.md` a partir do codebase. O resultado é exatamente o que a tabela manda deixar de fora: paths, descrição da estrutura, frases genéricas. A aula recomenda o caminho oposto:

1. comece com o mínimo (stack, comandos, boundaries);
2. trabalhe normalmente com a IA;
3. observe **onde ela alucina** ou erra;
4. acrescente só a instrução suficiente para corrigir aquele erro.

Os modelos atuais são bons o bastante para precisar de instrução apenas sobre o que é **não óbvio e específico do projeto**: os padrões de código, a forma de lidar com transações, as convenções do time. O resto eles já sabem. É o ciclo de feedback contínuo do cap. 2, aplicado ao arquivo mais importante do projeto.

> **FAÇA**
>
> + Mire em menos de 100 linhas e trate 150 como limite.
> + Abra o arquivo com os boundaries, em caixa alta.
> + Pareie cada proibição com a alternativa certa.
> + Escreva comandos exatos, com flags, e regras testáveis.
> + Troque texto longo por ponteiro e diga quando cada documento deve ser lido.
> + Referencie skills pelo nome.
> + Seção passou de ~20 linhas? Transforme em skill.
> + Mande consultar o Context7 ao mexer em bibliotecas de terceiros.
> + Acrescente regras a partir dos erros reais da IA.

EVITE

- Gerar o arquivo com `/init` e não revisar.
- Listar a árvore de pastas e os paths do projeto.
- Frases vagas como “escreva código limpo”.
- Instruções de uma tarefa específica.
- Repetir o que já está no README ou na wiki.
- Só proibições, sem dizer o que fazer no lugar.
- Espalhar `AGENTS.md` por diretórios sem um plano para mantê-los.

ARMADILHA

O arquivo que só cresce. Cada erro da IA vira uma linha nova, e ninguém apaga nada. Seis meses depois, o `AGENTS.md` tem 400 linhas, metade delas sobre problemas que não existem mais, e a IA passou a ignorar justamente as regras importantes, diluídas no meio do resto. Iterar não é só acrescentar. A cada revisão, pergunte quais linhas ainda pagam o custo de ir em toda requisição, e mova para documentos sob demanda ou para skills o que só vale de vez em quando.

CAPÍTULO 5

Skills

TL;DR

Uma *skill* é uma pasta com um `SKILL.md`: um nome, uma description e instruções, mais scripts e referências opcionais. Só o nome e a description ficam sempre no contexto; o corpo entra quando a tarefa casa com a description, e as referências entram quando o corpo manda. Por isso a description é a linha mais importante da skill: ela diz **o quê**, **quando** e com que **sinônimos** acionar. O corpo começa pelo caminho principal e empurra o resto para `references/`. Scripts fazem o trabalho repetitivo sem gastar tokens. E cada skill instalada custa contexto: passe de 20 e o catálogo começa a pesar.

O problema que as skills resolvem

O `AGENTS.md` funciona bem para **um** projeto. Mas uma habilidade como "avaliar a arquitetura", "fazer um release" ou "converter um extrato bancário" não pertence a um projeto: você quer usá-la em vários e compartilhá-la com outras pessoas. As rules não são portáveis, e até pouco tempo não havia um padrão para empacotar uma habilidade desse tipo.

A skill preenche essa lacuna juntando o melhor de duas ideias:

- do `AGENTS.md`, a descrição em texto e a progressive disclosure (cap. 4);
- do **MCP** (cap. 6), o mecanismo de enviar ao modelo só um nome e uma descrição curta, para que **ele decida** quando usar.

O resultado é uma receita que o agente carrega sob demanda quando a tarefa bate com a descrição. Se o `AGENTS.md` é o que vale sempre, a skill é o know-how de uma tarefa.

Anatomia de uma skill

Uma skill é só uma pasta. O único arquivo obrigatório é o `SKILL.md`; o resto é opcional.

```
nome-da-skill/
├── SKILL.md       obrigatório
├── scripts/       código executável
├── references/    documentos lidos sob demanda
└── assets/        templates, ícones, arquivos de apoio
```

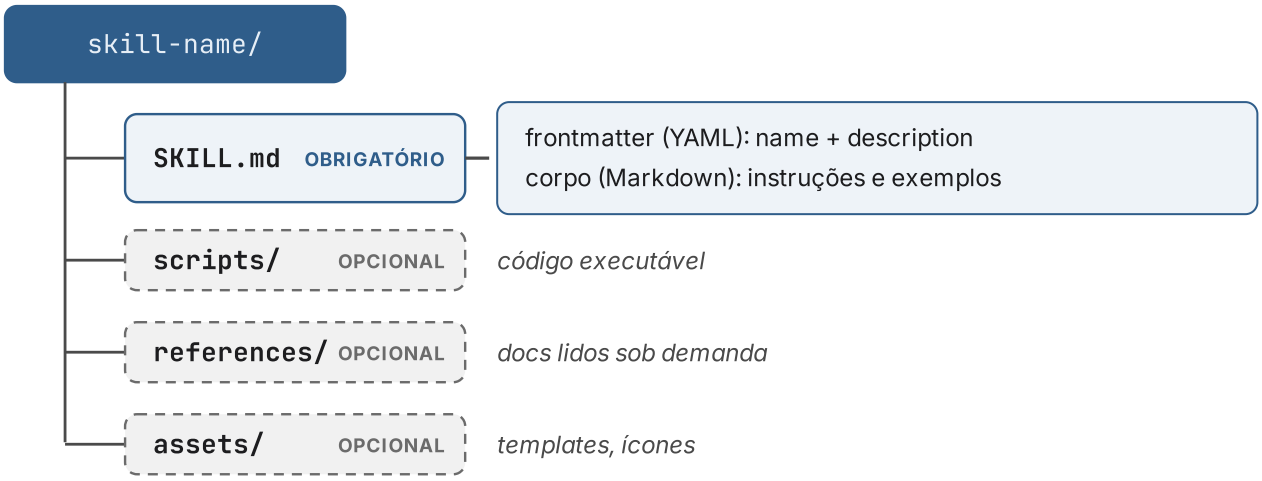


Figura 5.1 Anatomia de uma skill: a pasta com o SKILL.md obrigatório e as pastas opcionais scripts, references e assets

O `SKILL.md` tem duas partes:

- o *frontmatter*, um cabeçalho em YAML com o `name` e a `description`;
- o **corpo**, em Markdown, com as instruções e os exemplos.

```
---
name: bradesco-ynab
description: Converte extratos do Bradesco (CSV) para o formato do YNAB. ...
---
# Conversão de extratos
1. ...
```

A pasta inteira é a skill, mas o agente começa sempre pelo `SKILL.md`. E dentro dele, começa pelo frontmatter.

Como uma skill é acionada

O frontmatter de todas as skills instaladas fica **sempre** carregado. É ele que o agente lê para decidir. O corpo e as referências ficam em disco até serem necessários. O fluxo, do pedido à conclusão:

1. a tarefa do usuário chega;
2. o agente lê **só o nome e a description** de todas as skills;
3. alguma casa com a tarefa?
   - **não:** o agente resolve direto, sem skill;
   - **sim:** carrega o corpo do `SKILL.md` e segue as instruções;
4. se as instruções mandarem, roda um script ou lê uma referência, também sob demanda;
5. conclui a tarefa.

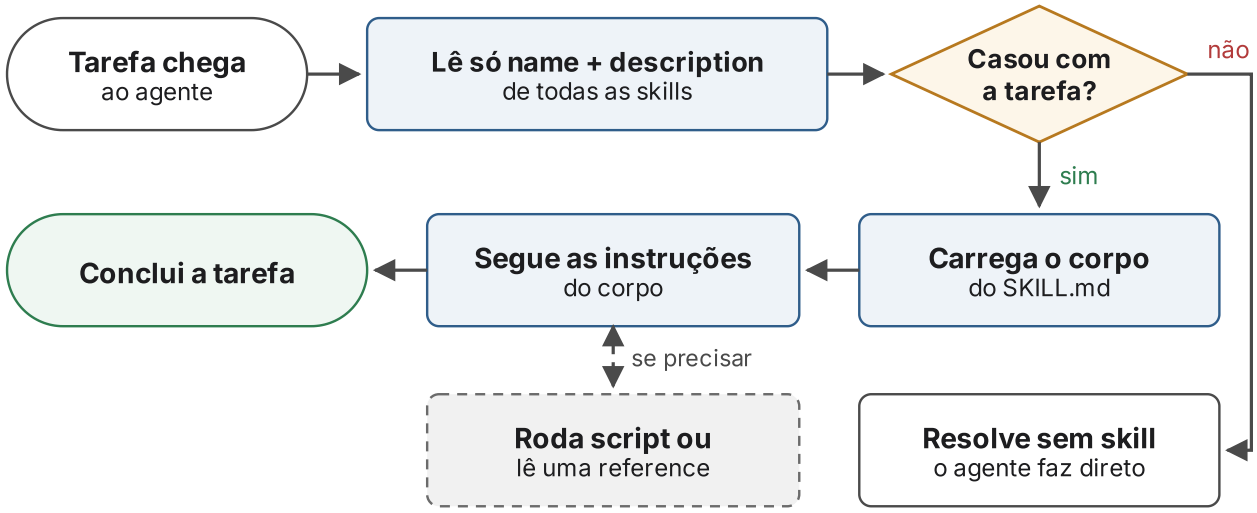


Figura 5.2 Como uma skill é acionada: o agente lê nome e description de todas as skills, carrega o corpo só se houver match e as referências só se o corpo pedir

Um exemplo do curso: uma skill de arquitetura modular. Enquanto a tarefa é corrigir um texto de tela, ela não entra. Quando a tarefa começa a mexer em imports, providers ou módulos, a description casa, o agente lê o corpo do disco e o injeta no contexto. Se o corpo disser "para criar um módulo novo, leia `references/scaffolding.md`", essa referência só entra quando o módulo novo for de fato criado. É progressive disclosure em três camadas.

A description: a linha que decide

Se a description não casar com a tarefa, o corpo nunca é lido, e a skill, por melhor que seja, não existe. Uma boa description tem três partes:

Parte	Exemplo	Função
O quê	"Converte extratos do Bradesco (CSV) para o formato do YNAB."	a capacidade
Quando	"Use quando precisar importar transações do Bradesco no YNAB..."	os gatilhos e o contexto
Sinônimos	"...converter CSV, preparar extratos para apps de orçamento."	outras formas de pedir a mesma coisa

Duas regras complementam a tabela:

- **Todo "quando usar" mora na description, nunca no corpo.** O corpo só é lido depois que a decisão já foi tomada; uma instrução de acionamento escrita lá chega tarde demais.
- **Seja específico e insistente.** Skills costumam ser subutilizadas: o modelo prefere resolver sozinho. Uma description tímida perde para essa preferência.

A insistência necessária muda de modelo para modelo. Alguns acionam skills com facilidade; outros precisam de gatilhos mais explícitos. Adaptar a skill ao comportamento de cada modelo começa pela description.

Estruturando o corpo

Dentro do `SKILL.md`, a ordem segue a importância: **informação importante primeiro, o resto vira referência.**

1. **Frontmatter:** name e description.
2. **Workflow principal:** o caminho que resolve 90% dos casos.
3. **Formatos e exemplos.**
4. **Casos de borda e exceções.**
5. **Ponteiros para as referências:** "para X, leia `references/x.md`".

As referências são onde a skill guarda o que só vale em parte dos casos. Uma skill de deploy, por exemplo, pode ter `references/aws.md`, `references/gcp.md` e `references/azure.md`, e o corpo manda ler só a do provedor em uso. A pasta `references/` serve exatamente para quebrar uma skill grande em arquivos menores.

Duas perguntas práticas aparecem aqui:

- **Como garantir que o modelo leia a referência?** Com um ponteiro explícito e condicional no corpo, no formato "para X, leia `references/x.md`", e não com uma lista solta de arquivos.
- **Como garantir que o modelo execute um passo?** Dizendo quando ele realmente precisa fazê-lo. Comandos vagos ("considere rodar os testes") viram opcionais; comandos com condição ("antes de concluir, rode `npm test` e só siga se passar") não.

O limite de tamanho do corpo é generoso, mas existe: **passou de ~500 linhas, quebre em hierarquia e referências.**

Scripts: o que não precisa gastar tokens

Além de texto, uma skill pode trazer scripts executáveis. Tarefas repetitivas e determinísticas (converter um formato, gerar um esqueleto de arquivo, validar uma estrutura) saem mais baratas e mais confiáveis num script do que pedidas ao modelo a cada vez. A skill diz quando rodar; o script faz o trabalho; o modelo só recebe o resultado.

Quantas skills: o custo do catálogo

Cada skill instalada tem o nome e a description **sempre** no contexto. Uma skill custa pouco; vinte custam bastante. A aula recomenda **não passar de 20 skills** no total e desativar as que você não está usando. Com 15 a 20, o contexto inicial já fica pesado.

O mesmo raciocínio vale para descriptions longas. A description precisa ser completa, mas cada palavra dela é paga em toda requisição, por toda skill instalada.

Skills e sub-agents

Antes do padrão de skills, era comum criar *sub-agents* customizados para cada habilidade: um para converter PDF, outro para revisar PR. Um sub-agent é um agente disparado por outro, com a

própria janela de contexto (o cap. 11 trata deles em detalhe). Os customizados gastavam mais tokens e eram difíceis de compartilhar.

O arranjo que o curso recomenda é outro: **skills mais sub-agents genéricos.** O agente principal decide quando iniciar um sub-agent, e o sub-agent carrega sozinho as skills de que precisa. É mais dinâmico e dá menos trabalho manual. (O cap. 15 mostra um caso em que sub-agents customizados voltam a ser a escolha certa: quando é preciso fixar o modelo que executa cada papel.)

Criando as suas skills

Escrever uma boa skill do zero dá trabalho. O curso indica a *skill-architect*, uma skill que ajuda a criar skills: ela conduz a definição da description, a estrutura do corpo e a divisão em referências. Há também um marketplace de skills da Tech Leads Club, com skills prontas que aparecem ao longo do livro (Jira Assistant no cap. 8, TLC Spec-Driven no cap. 9, o kit do cap. 18). Os links estão no Apêndice C, e o template de `SKILL.md`, no Apêndice A.

FAÇA

+ Escreva a description com o quê, quando e sinônimos.
+ Coloque todo "quando usar" na description.
+ Seja específico e insistente: skills tendem a ser subutilizadas.
+ Abra o corpo com o workflow principal, o caminho de 90%.
+ Use ponteiros condicionais para as referências: "para X, leia `references/x.md`".
+ Mova trabalho repetitivo e determinístico para scripts.
+ Mantenha no máximo ~20 skills ativas.

EVITE

– Instruções de acionamento no corpo da skill.
– Descriptions genéricas ("ajuda com arquivos") ou longas demais.
– Corpos acima de ~500 linhas sem divisão em referências.
– Pedir ao modelo, a cada vez, o que um script faria sempre igual.
– Instalar skills "por via das dúvidas".

ARMADILHA

A skill que nunca dispara. Você escreveu uma skill excelente, com workflow, exemplos e referências, e o agente continua resolvendo a tarefa do jeito dele. O problema quase nunca está no corpo: está na description. Ou ela descreve a capacidade sem dizer quando usar, ou usa palavras diferentes das que aparecem nos pedidos reais. Teste a skill com os pedidos que você e o time realmente fazem e ajuste a description até que eles casem.

CAPÍTULO 6

Tools, MCP e RAG

TL;DR

Para o modelo, toda ação é uma tool: ler um arquivo, rodar um comando, acionar uma skill, criar uma página no Confluence. O MCP é um protocolo para expor tools que falam com sistemas externos, com autenticação segura e sem credenciais no repositório. O caso de uso clássico é o RAG, a busca semântica em documentação interna. Rules, skills, MCPs e tools não são a mesma coisa: **regra é sempre, skill é sob demanda, MCP é conexão, tool é ação.**

Tudo é tool

O cap. 1 mostrou que o modelo não executa nada: ele **pede**, e o agente executa. Cada coisa que o modelo pode pedir é uma *tool* (ferramenta), também chamada de *function*: um pedaço de código chamável, com nome, descrição e parâmetros, que executa uma ação e devolve um resultado.

Do ponto de vista do modelo, tudo é tool:

- buscar um arquivo no disco;
- rodar um comando no terminal;
- acionar uma skill;
- consultar a documentação de uma biblioteca;
- criar uma página no Confluence ou uma task no Jira.

A consequência é direta: **o que não está disponível como tool não existe para o agente.** Se ele não tem uma tool para abrir o navegador, não vai conseguir validar a tela que acabou de construir, por mais que você peça.

MCP: tools para o mundo de fora

O *MCP* (*Model Context Protocol*) é um protocolo aberto para conectar agentes a sistemas externos. Um servidor MCP expõe um conjunto de tools (buscar issues, ler páginas, consultar documentação), e o agente as oferece ao modelo como qualquer outra tool. Os MCPs são, portanto, **tools especializadas em chamadas externas**.

A diferença para uma API REST está em quem decide. Uma API REST é feita para um programa que já sabe qual endpoint chamar, com quais parâmetros. Um MCP envia ao modelo **o nome e a descrição** de cada tool, e é o modelo que decide quando usar cada uma, como faz com as skills

(cap. 5). Além disso, o MCP cuida do fluxo de autenticação: o agente acessa o Jira ou o Notion sem que nenhuma credencial fique escrita no repositório.

Os MCPs que aparecem neste livro:

MCP	Para quê	Onde aparece
Context7	documentação atualizada de bibliotecas	caps. 4 e 8
Jira / Confluence (Atlassian), Linear	buscar e criar tasks e documentos	cap. 8 e Parte V
Notion	ler e escrever documentos do time	cap. 7 (onde moram PRDs e Design Docs)
Playwright	controlar um navegador para validar telas	Parte V

RAG: busca semântica

O caso de uso clássico do MCP é o *RAG* (*Retrieval-Augmented Generation*, geração aumentada por recuperação). Em vez de colocar toda a documentação interna no contexto, o agente **busca** os trechos relevantes para a pergunta e só esses entram na janela.

A busca é semântica, não por palavra exata. Os documentos são convertidos em vetores (os embeddings do cap. 1) e guardados num *banco de vetores*. A pergunta também vira vetor, e o banco devolve os trechos de significado mais próximo, mesmo que usem palavras diferentes. Um MCP sobre esse banco permite que o agente consulte, por exemplo, a base de conhecimento da empresa sem que ninguém precise copiá-la para o `AGENTS.md`.

O RAG foi a primeira forma de entregar contexto sob demanda (a linha do tempo do cap. 1), e a ideia continua viva em dois lugares. Nas empresas, MCPs de busca seguem relevantes para integrar documentação interna e sistemas legados. Nas ferramentas, a mesma técnica indexa o próprio codebase: o Cursor, por exemplo, guarda os arquivos do projeto num banco de vetores e encontra os relevantes por significado, em vez de depender só de `grep`.

Regra, skill, MCP ou tool?

As quatro peças da Parte II se confundem porque todas mexem no que o modelo recebe. Mas cada uma tem um papel diferente:

	Quando entra	O que dá	Observação
Rules / `AGENTS.md`	sempre, em toda sessão	regras globais, só texto	`CLAUDE.md`, `.cursorrules` e afins
Skill	sob demanda, quando casa com a tarefa	o know-how de um processo	traz scripts, referências e assets
MCP	quando o agente chama o servidor	acesso a sistemas e dados externos	é conexão, não instrução

	Quando entra	O que dá	Observação
Tool	quando o modelo pede a ação	uma ação que executa e devolve um resultado	é o próprio código chamável

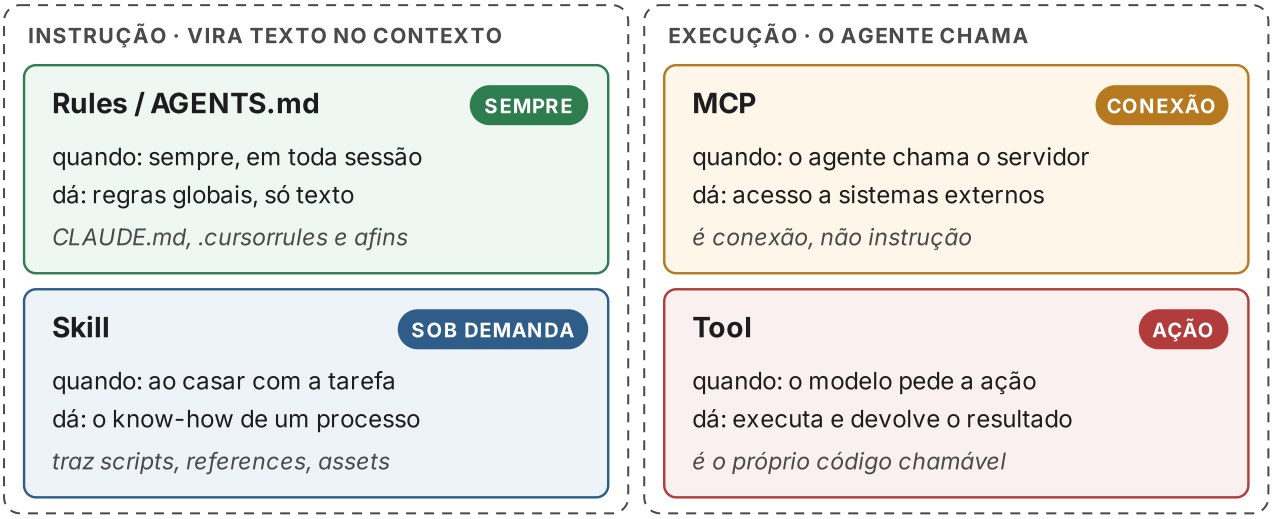


Figura 6.1 Rules, skill, MCP e tool lado a lado: quando cada um entra no contexto e o que cada um entrega

A frase do slide resume o quadro: **regra: sempre. Skill: sob demanda. MCP: conexão. Tool: ação.**

Na prática, as peças se combinam. Uma skill pode ensinar o modelo a usar um MCP: a skill Jira Assistant (cap. 8) sabe onde ficam a chave do projeto, o ID da instância e os templates de task, e usa o MCP do Jira para buscar e criar tasks. O MCP dá o acesso; a skill dá o processo. E o `AGENTS.md` pode dizer quando cada uma deve ser usada.

Um critério para decidir onde colocar algo novo:

- vale para **toda** tarefa do projeto? `AGENTS.md`;
- é o **know-how** de um tipo de tarefa, talvez útil em outros projetos? skill;
- depende de **dados ou ações de um sistema externo**? MCP;
- é uma **ação determinística** que pode ser executada? tool, ou um script dentro de uma skill.

Box: escolhendo a ferramenta

A aula sobre ferramentas parte de uma ideia: a ferramenta de desenvolvimento com IA não é só um editor de código. Ela precisa ter **uma integração excelente com agentes**. (O instrutor migrou do Vim para o VS Code e depois para o Cursor justamente por isso.)

O que olhar numa ferramenta:

- **Janela de agentes com modos diferentes:** planejar, executar como agente, perguntar sem alterar código.
- **Escolha e configuração de modelos:** variedade de provedores (Anthropic, OpenAI e outros), controle do nível de esforço (*thinking*) e opção de janelas grandes, como o Max Mode do Cursor, com 1 milhão de tokens.
- **Modelos baratos para implementar:** o Composer, modelo próprio do Cursor, rápido e barato, permite planejar com um modelo forte e implementar com ele. Uma estratégia agressiva de cache de tokens também reduz custos.
- **Suporte a rules, skills e sub-agents**, a base da Parte II e da Parte IV.
- **Tools, MCPs e *hooks*** (gatilhos que rodam automações em eventos da sessão).
- **Loja de plugins avaliados e seguros**, com pacotes de skills, sub-agents e comandos.
- **Indexação do codebase** num banco de vetores, para busca semântica.

Cursor, Claude Code ou Copilot? Na época do curso, as três estavam com funcionalidades parecidas. O Claude Code era o pioneiro em inovação, porém mais caro e restrito aos modelos da Anthropic. O Cursor oferecia mais variedade de modelos e um modo Auto sem custo adicional. Preços e modelos mudam rápido; o que vale é a lista acima. E é comum combinar mais de uma ferramenta.

FAÇA

+ Dê ao agente as tools de que a tarefa precisa, inclusive para verificar o próprio trabalho.
+ Use MCPs para acessar sistemas externos, com a autenticação do protocolo.
+ Use busca semântica (RAG) para documentação grande demais para o contexto.
+ Combine: MCP para o acesso, skill para o processo, `AGENTS.md` para dizer quando.
+ Escolha a ferramenta pela integração com agentes, não pelo editor.

EVITE

– Credenciais de serviços externos no repositório.
– Copiar a base de conhecimento inteira para o contexto em vez de buscá-la.
– Pedir ao agente uma ação para a qual ele não tem tool.
– Escolher a ferramenta por preço ou modelo do momento: os dois mudam rápido.

> ⚠ **ARMADILHA**
>
> **Esperar que o MCP ensine o processo.** Você instalou o MCP do Jira e pediu "crie as tasks deste épico". O agente cria, mas no projeto errado, sem o template do time e com campos faltando. O MCP fez a parte dele: deu acesso. **MCP é conexão, não instrução.** O jeito certo de usar o sistema (qual projeto, qual template, quais campos) é know-how, e know-how mora numa skill ou no `AGENTS.md`.

PARTE III

Do problema ao plano

7 O fluxo de desenvolvimento com IA

8 As primeiras tasks com IA

9 RPI e Spec-Driven Development

III

CAPÍTULO 7

O fluxo de desenvolvimento com IA

TL;DR

A IA não dispensa as boas práticas de engenharia; ela as cobra. Sem bom input, o output é ruim e os tokens vão embora. O fluxo tem três fases: **Discovery** (humanos e produto; a IA ajuda pouco), **Planejamento** (as decisões viram documento) e **Desenvolvimento** (o documento vira spec, testes e código com IA), seguidos de **Code Review**. Cada documento tem um papel: o PRD diz o quê e por quê, o Design Doc diz como, a Spec é o contrato com a LLM, a RFC abre uma decisão, o ADR registra uma decisão tomada. Quanto maior a empresa, mais documentos fazem sentido. Tudo isso vira contexto.

A IA cobra as boas práticas

Existe a expectativa de que a IA permita pular etapas: menos reunião, menos documento, direto para o código. Na prática acontece o contrário. **Sem um bom input, a IA gera um output ruim e desperdiça tokens.** Levantar requisitos, ficar perto do produto e documentar decisões, práticas que muitos times tratavam como opcionais, passam a ser praticamente obrigatórias.

A aula diz isso de forma direta: **não dá mais para se esconder atrás do código.** Quando o código é escrito pela IA, o trabalho do dev se desloca para antes dele: entender o problema e transformá-lo em contexto que a IA consiga usar.

As fases do fluxo

O fluxo de desenvolvimento com IA tem três fases, seguidas do review:

1. Discovery. É guiada por humanos: produto e dev juntos. Aqui a IA ajuda pouco, e não deve ser o centro. O que a IA pode fazer é garantir que nada se perca: entrevistas com usuários e reuniões de alinhamento são gravadas, e tudo vira documento (PRD, transcrições, notas).

2. Planejamento. O resultado da Discovery vira um **Design Doc**. O Design Doc é quebrado em tarefas no board do time (Jira, Linear). E aqui a IA já ajuda bastante: ela pode quebrar as tarefas e **estimar o trabalho e a dificuldade olhando o codebase**.

3. Desenvolvimento. As tasks do board dão contexto à IA para gerar um plano de implementação (cap. 8). O dev acompanha a implementação e ajusta. Os testes entram **como parte do plano**, não como etapa posterior.

4. Code Review. Feito por dev e IA juntos, com diretrizes de review escritas para a IA (cap. 14).

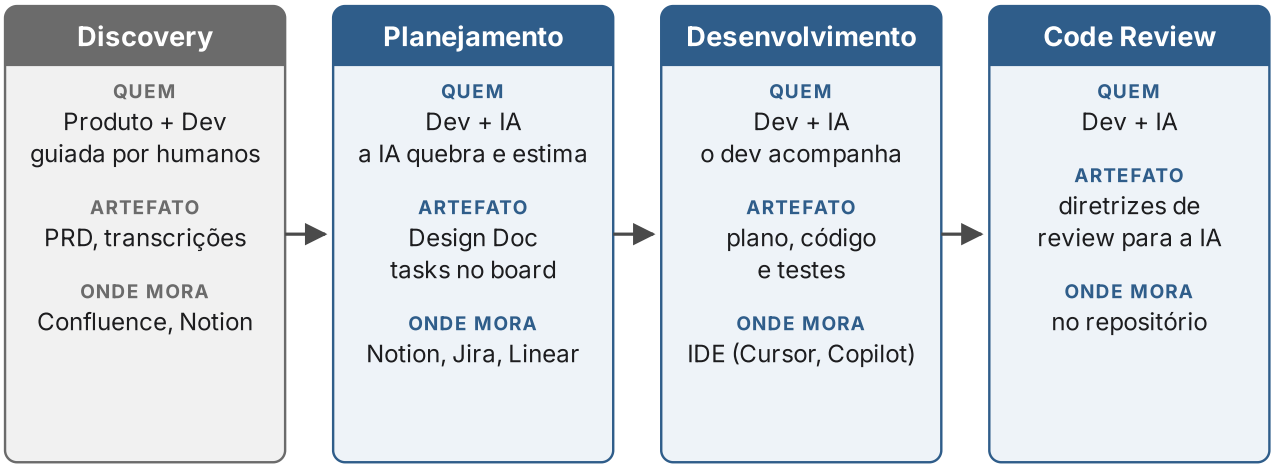


Figura 7.1 O fluxo com IA: Discovery, Planejamento, Desenvolvimento e Code Review, com o artefato e a ferramenta de cada etapa

Repare onde cada artefato mora. O PRD e o Design Doc ficam em ferramentas de documentação (Confluence, Notion). As tasks ficam no board. As diretrizes, o plano e o código ficam no repositório, ao alcance da IDE. A IA trabalha melhor quando consegue chegar a todos eles, pelos MCPs do cap. 6.

O que pode virar contexto

Cada documento do fluxo responde a uma pergunta diferente:

Documento	Papel	O que faz
PRD (*Product Requirements Document*)	o **quê** e o **porquê**	define o problema, os objetivos e os requisitos do produto para os stakeholders
Design Doc (*Technical Design Doc*)	o **como**	detalha a solução técnica antes de começar a implementar
Spec	o **contrato exato** do que será entregue	descreve comportamento, interfaces e critérios de aceite com precisão
RFC (*Request for Comments*)	uma **proposta aberta**	apresenta uma ideia para discussão e feedback antes de decidir
ADR (*Architecture Decision Record*)	o **registro de uma decisão já tomada**	documenta o contexto, a escolha e o motivo, para referência futura

Em mais detalhe:

- O ***PRD*** é um documento de alto nível, escrito pelo PM. Ele foca no problema, não na solução técnica. É o ponto de partida.
- O ***Design Doc*** é escrito pelo dev depois da Discovery. Traz o contexto do projeto, um glossário de termos, os endpoints, o escopo do MVP e um *breakdown* (quebra) das tasks em alto nível. Serve para alinhar liderança e produto antes de qualquer código. É ideal para projetos de várias semanas que envolvem várias pessoas, e uma dica da aula é **apresentá-lo**, não só compartilhar o link.

- A ***Spec*** é o contrato voltado para a LLM. É mais técnica que o Design Doc e feita para o dev usar com a IA. Alguns frameworks, como o Superpowers (cap. 9), geram specs muito orientadas ao código.
- A ***RFC*** é aberta por definição: explora uma decisão técnica com várias opções (qual banco de dados usar, por exemplo) e circula pelo time para discussão antes da escolha.
- O ***ADR*** é imutável. Registra uma decisão que já foi tomada e vira contexto histórico, e input, para implementações futuras.

> **NOTA**
>
> **Design Doc ou TDD (testes)?** Nos slides do curso, o Design Doc aparece com a sigla de Technical Design Doc. Ela é idêntica à de TDD (testes), o *Test-Driven Development* do cap. 12, e as duas ideias aparecem no mesmo fluxo. Para não confundir, este livro escreve sempre **Design Doc** para o documento e **TDD (testes)** para a disciplina de escrever o teste antes do código. Também não confunda o Design Doc com o `design.md` do Spec-Driven (cap. 9), que é uma fase da spec, mais técnica e voltada para a IA.

Lean Inception

Quando o projeto é novo e não há contexto nenhum, a Discovery precisa de um ponto de partida. A *Lean Inception* é um formato de workshop curto, de cerca de uma hora na versão da aula, para alinhar o time. Ela define o escopo, as jornadas de usuário e as funcionalidades, priorizadas por esforço e valor. Faz mais sentido em projetos novos, de pelo menos médio porte e com muita incerteza.

Quando usar cada documento

Nem todo projeto precisa de todos os documentos. O que define a escolha é o tamanho da empresa e o tipo de projeto.

Sozinho	Empresa pequena (~5 pessoas)	Empresa média (5–50)	Empresa grande (50+)
Design Doc	Lean Inception (muita incerteza)	Lean Inception (projetos grandes)	Lean Inception
Spec	Design Doc	PRD (depende da estrutura de PM)	PRD
	ADR	Design Doc	Design Doc
		RFC (decisões de grande impacto ou vários times)	RFC
		ADR	ADR

A leitura prática da aula é ainda mais enxuta para quem trabalha sozinho: **na maioria das vezes, a spec basta.** O Design Doc entra quando o projeto é grande o bastante para precisar de alinhamento com outras pessoas.

E, por tipo de situação:

- **Lean Inception:** projeto novo, de pelo menos médio porte, com incertezas.
- **PRD e Design Doc:** projeto novo, de várias semanas, com várias pessoas no time.
- **Spec:** quando não existe Design Doc, RFC, ADR ou outro contexto pronto.
- **RFC:** quando é preciso alinhar uma decisão ou resolver uma incerteza.

A IA escrevendo o Design Doc

A fase de Planejamento também pode ser acelerada com IA. A skill **Design Doc Creator** parte de um pedido mínimo, como "crie um Design Doc para integrar com o Stripe". A IA então:

1. explora o codebase para entender o que já existe;
2. faz perguntas para tirar as ambiguidades;
3. gera um documento compartilhável com produto e liderança.

O documento sai com cara de Design Doc, não de spec: é feito para pessoas lerem e decidirem.

Tudo vira contexto

O fio que liga o capítulo é este: **PRD, Design Doc, RFC, ADR, gravações de reunião e notas de reunião podem ser todos usados como input para a IA.** A estratégia que a aula propõe é concentrar o esforço humano na Discovery, onde estão as decisões que a IA não consegue tomar, e automatizar o máximo possível daí em diante. O cap. 8 mostra o primeiro passo dessa automação, e o cap. 9, a versão em escala.

FAÇA

+ Invista na Discovery: é onde o esforço humano rende mais.
+ Grave entrevistas e reuniões e transforme tudo em documento.
+ Use a IA para quebrar e estimar tarefas a partir do codebase.
+ Escolha os documentos pelo tamanho do time e do projeto.
+ Apresente o Design Doc para alinhar, não só compartilhe.
+ Inclua os testes no plano de implementação desde o começo.

EVITE

– Pular a Discovery porque a IA escreve código rápido.
– Usar a IA como centro da Discovery.
– Usar a mesma sigla para o Design Doc e para o TDD (testes).
– Exigir todos os documentos num projeto solo ou pequeno.
– Deixar os testes para depois da implementação.

⚠ ARMADILHA

Documentar para a IA o que ninguém decidiu. A spec ficou ótima, as tasks estão claras e a implementação passou em todos os testes, mas a funcionalidade não era o que o produto precisava. A IA executa bem o que está no documento; ela não descobre o que deveria estar lá. Se a Discovery foi pulada, a qualidade do resto do fluxo só garante que o problema errado foi resolvido com eficiência.

CAPÍTULO 8

As primeiras tasks com IA

TL;DR

O jeito mais simples de começar é partir da task do board: com o MCP do Jira ou do Linear, "busque a task CAM-42 e planeje" já basta para a IA trazer o contexto. Planeje sempre no **Plan Mode** antes de alterar qualquer código: revisar um plano é muito mais barato do que revisar código. Use um modelo forte para planejar e um barato, numa janela nova, para implementar. Agrupe as tasks, que foram escritas para humanos, em planos de tamanho parecido. O Plan Mode funciona bem para 1 a 3 tasks; para mais do que isso, e para ter garantia de testes, o próximo passo é o Spec-Driven do cap. 9.

Da task do board ao plano

Com o MCP do Jira, do Linear ou de outro board configurado (cap. 6), o primeiro pedido do dia pode ser curto:

```
Vou começar a trabalhar na task CAM-42. Busque ela e planeje.
```

A IA busca a task pelo MCP, lê a descrição, os critérios e os comentários, e começa a planejar a partir daí. O contexto de produto que a Discovery produziu (cap. 7) chega até o agente sem que ninguém precise copiar e colar.

Para que isso funcione bem, a IA precisa saber usar o board **do seu projeto**. É o papel da skill **Jira Assistant**: ela sabe onde estão as configurações (a chave do projeto, o ID da instância, a URL do board), sabe buscar e criar tasks e traz templates para criar tasks no padrão do time. É a combinação do cap. 6 na prática: o MCP dá o acesso, a skill dá o processo.

Plan Mode primeiro

O *Plan Mode* é o modo das ferramentas em que o agente investiga e propõe um plano **sem alterar nenhum arquivo**. A recomendação do curso é usá-lo sempre, antes de qualquer implementação.

O motivo é econômico. Deixar a IA implementar às cegas gera retrabalho: você só descobre que ela entendeu errado depois que vinte arquivos mudaram. No plano, o mesmo mal-entendido aparece em uma linha e é corrigido com uma frase. **Revisar um plano é muito mais barato do que revisar código já implementado.**

O que acontece durante o planejamento

Num agente bem instrumentado, o planejamento já usa quase tudo o que a Parte II montou.

- **Sub-agents explorando o codebase.** O agente principal dispara sub-agents genéricos (cap. 5) para explorar o código. Cada um lê o que precisa na própria janela e devolve só um resumo. A janela principal continua limpa para o plano.
- **Documentação atualizada pelo Context7.** Quando o plano envolve uma biblioteca externa, como o SDK do Stripe para Node, a IA consulta o Context7 e planeja com a API atual, não com a que estava nos dados de treino.
- **Progressive disclosure no plano.** Se as rules mandam ler `integration-patterns` ao integrar com APIs externas (cap. 4), o plano já sai referenciando esse documento. O plano segue os padrões do projeto porque as rules mandaram buscá-los.

Nada disso exige um pedido explícito a cada vez. É o efeito acumulado de um `AGENTS.md` bem escrito, das skills certas e dos MCPs configurados.

Modelo forte para planejar, barato para implementar

Planejar e implementar exigem coisas diferentes. Planejar pede raciocínio: entender o problema, comparar caminhos, antecipar o que pode dar errado. Implementar um plano bem escrito pede principalmente execução.

Daí a divisão que o curso recomenda:

1. **Planeje com um modelo mais capaz** (por exemplo, o Opus).
2. **Salve o plano.**
3. **Abra uma janela nova com um modelo mais barato** e implemente a partir do plano.

A troca de janela é tão importante quanto a troca de modelo. A implementação começa com o contexto limpo, sem a exploração, as dúvidas e as alternativas descartadas do planejamento (cap. 3). O cap. 9 dá nome e estrutura a essa separação: RPI.

Agrupando tasks para a IA

As tasks do board foram escritas **para humanos**. Para a IA, elas costumam ser granulares demais: uma task para criar a tabela, outra para o endpoint, outra para a validação, cada uma pequena demais para justificar um plano próprio.

A saída é pedir à IA que reorganize o trabalho:

```
Busque todas as tasks do épico CAM-40 e agrupe em 3 planos
de tamanho parecido, com no máximo 10 arquivos cada.
```

Cada plano vira uma unidade de trabalho do tamanho certo para uma janela: grande o bastante para fazer sentido, pequeno o bastante para não sair da Smart Zone.

As limitações do Plan Mode

O Plan Mode é um ótimo começo, mas tem limites claros:

- **Funciona bem para 1 a 3 tasks.** Não escala para um projeto inteiro.
- **Não garante a qualidade dos testes.** O plano pode prever testes, mas nada verifica se eles testam o que deveriam.
- **Não impede que a IA apague testes.** Nem que os pule.
- **Não usa sub-agents nem paralelização na implementação.** Tudo acontece numa janela só, que vai enchendo.

Para escalar, é preciso uma estrutura que defina o que é "pronto" de forma verificável, quebre o trabalho em tasks atômicas e distribua a execução. É o fluxo de spec do próximo capítulo.

FAÇA

+ Comece pela task do board, buscada por MCP.
+ Use uma skill para ensinar a IA a usar o board do seu projeto.
+ Planeje no Plan Mode e leia o plano antes de aprovar.
+ Planeje com um modelo forte; implemente com um barato, numa janela nova.
+ Agrupe tasks pequenas em planos de tamanho parecido, com limite de arquivos.

EVITE

– Deixar a IA implementar sem um plano revisado.
– Implementar na mesma janela em que planejou.
– Usar o modelo mais caro para tudo.
– Mandar as tasks do board para a IA uma a uma, do jeito que foram escritas para humanos.
– Usar o Plan Mode para um projeto inteiro.

ARMADILHA

Aprovar o plano sem ler. O Plan Mode só é mais barato do que revisar código se o plano for de fato revisado. Um "pode seguir" automático transforma o plano num ritual: a IA implementa o que planejou, inclusive o mal-entendido do terceiro parágrafo, e o custo de corrigir volta a ser o de revisar código. Leia o plano procurando três coisas: os arquivos que serão alterados, as decisões que ele tomou sozinho e onde estão os testes.

CAPÍTULO 9

RPI e Spec-Driven Development

TL;DR

RPI (Research, Plan, Implement) separa o trabalho em três fases: pesquisar com um modelo forte e janela grande, salvar o resultado num plano em arquivo e implementar numa janela nova, com um modelo mais barato e só o plano. O **Spec-Driven Development** é a forma mais sólida de fazer isso: o problema vira spec com user stories, a spec vira design, o design vira tasks atômicas, cada uma com o quê, onde e uma Definition of Done verificada por comando no terminal. A profundidade se adapta ao tamanho do trabalho. O `STATE.md` guarda a memória entre sessões e o `concerns.md` aponta os riscos do codebase. Parece gastar mais tokens no começo, mas elimina o retrabalho.

RPI: Research, Plan, Implement

O cap. 8 separou planejar de implementar trocando de modelo e de janela. O RPI transforma essa separação no princípio do fluxo inteiro, em três fases:

- **Research (pesquisa).** Um modelo bom, numa janela grande, gasta tokens **para entender**: bate em MCPs, APIs externas, PDFs, gravações de reunião, documentação, o próprio codebase. Pode orquestrar sub-agents para pesquisar frentes diferentes sem encher a janela principal.
- **Plan (plano). Todo o resultado da pesquisa é salvo em arquivo**, num Markdown ou numa spec otimizada para ser lida por outra janela. É o que torna a pesquisa reutilizável.
- **Implement (implementação).** Uma **janela nova**, um modelo mais barato e o contexto limpo. A implementação parte só do plano.

O formato de funil é proposital. Na boca entram muitas fontes: os arquivos relevantes, a estrutura do projeto, a análise de negócio, buscas externas e documentação, skills, MCPs, scripts locais para provas de conceito e o ajuste do prompt e do contexto. No meio, tudo isso é condensado num plano. Na ponta, a implementação recebe só o que precisa.

A regra que a aula tira daí: **tudo o que vamos implementar precisa, antes, de uma pesquisa.** E o RPI resolve o conflito do cap. 3. A pesquisa precisa de muito contexto; a implementação precisa da Smart Zone. Com fases e janelas separadas, cada uma tem o que precisa.



Figura 9.1 O funil RPI: a pesquisa consome muitas fontes, o plano salva o resultado em arquivo e a implementação parte só do plano

O RPI não exige Spec-Driven: um plano bem escrito em Markdown já é RPI. Mas o Spec-Driven é o padrão mais sólido para fazê-lo, e é o resto deste capítulo.

Por que o Spec-Driven se paga

O *Spec-Driven Development* (desenvolvimento guiado por especificação) parece caro: antes de uma linha de código, a IA mapeia o codebase, escreve spec, design e tasks. A conta muda quando se olha o outro lado. **Sem spec, o dev gasta tokens repetidamente tentando acertar**: implementa, descobre que não era aquilo, explica de novo, implementa de novo. Com spec, o custo é pago uma vez, na pesquisa, e a implementação acerta muito mais.

As fases do Spec-Driven

O fluxo tem quatro fases. A versão abaixo é a do TLC Spec-Driven, a skill usada no curso:

Fase	Objetivo	O que contém	Saída
1. Specify	capturar o **quê**	user stories priorizadas (P1, P2, P3) no formato WHEN / THEN / SHALL; IDs de rastreabilidade; o que fica fora do escopo	`spec.md`
2. Design	definir o **como**	arquitetura e diagramas; análise de reaproveitamento; componentes e interfaces; modelos de dados	`design.md`
3. Tasks	quebrar em **tarefas**	uma task = um entregável; dependências claras; marcação do que roda em paralelo; verificação binária	`tasks.md`
4. Execute	implementar e **verificar**	planejar, implementar, verificar; um commit atômico por task; nada fora do escopo; teste de aceitação interativo quando há interface	commits

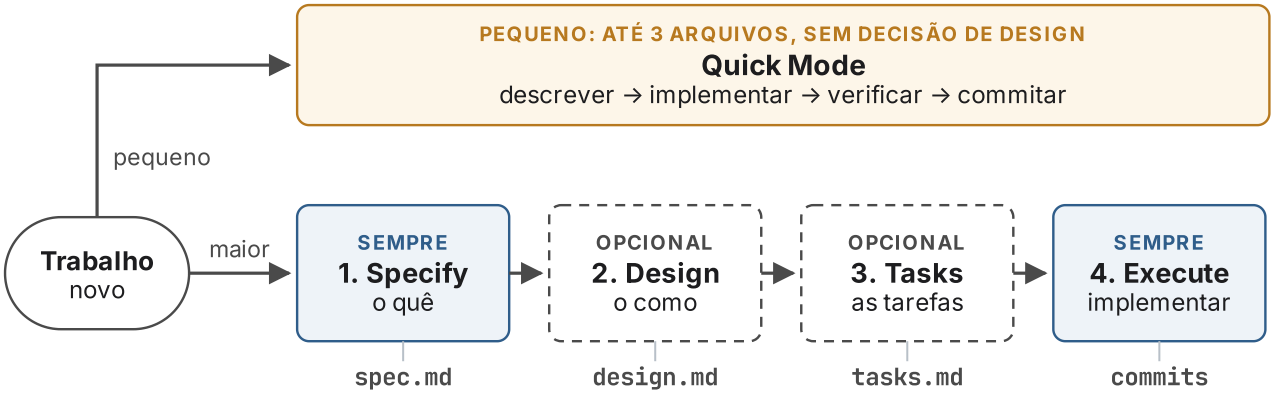


Figura 9.2 O pipeline Spec-Driven com auto-sizing: Specify e Execute sempre; Design e Tasks conforme o tamanho do trabalho

Mapeamento do codebase

Em projetos que já existem, antes da spec vem o **mapeamento**. A IA explora o codebase usando sub-agents genéricos, para manter a janela principal limpa, e registra:

- as dependências e a estrutura do projeto;
- a arquitetura e as convenções de código;
- e, principalmente, os **padrões de teste**: quais tipos existem (unidade, integração, E2E), como rodam e **quais são seguros para rodar em paralelo**.

Esse último item vai decidir, mais tarde, o que pode ser paralelizado (caps. 11 e 12).

O mapeamento produz também o `concerns.md`: uma lista de **pontos de atenção** do codebase que podem afetar a implementação. No projeto Stripe do curso, apareceram placeholders esquecidos, a falta de infraestrutura para webhooks e a ausência de um cliente HTTP. Vale sempre ler esse arquivo antes de implementar: ele mostra os buracos antes que a implementação caia neles.

A spec

A spec se parece com um Design Doc (cap. 7), mas é escrita para a LLM. Ela contém:

- o **problema** e o **escopo** (e o que fica de fora);
- as ***user stories*** (histórias de usuário);
- os ***edge cases*** (casos de borda);
- os **requisitos**;
- os ***success criteria*** (critérios de sucesso).

As user stories são a parte mais importante. Elas viram os testes da implementação e são a base da validação no code review. No TLC Spec-Driven, cada uma tem prioridade (P1, P2, P3), um formato verificável e um ID que atravessa o fluxo inteiro:

```
[PAG-03] P1
WHEN o pagamento é recusado pelo Stripe
```

```
THEN o sistema SHALL exibir a mensagem de erro
AND SHALL manter os itens no carrinho
```

O design

O `design.md` é gerado a partir da spec e descreve a implementação em alto nível: o fluxo completo, o que pode ser reaproveitado do código existente, os pontos de integração, os componentes, as decisões de banco, a configuração e o tratamento de erros.

A parte mais valiosa são as **decisões técnicas com racional**. No projeto Stripe, o design registrou, entre outras:

- usar o SDK do Stripe (tipado) em vez de chamadas HTTP diretas;
- criar endpoints novos na v2 em vez de modificar os existentes;
- usar cache local para ganhar performance e resiliência;
- como tratar os webhooks do Stripe, que não enviam JWT.

Registradas no design, essas decisões guiam todas as tasks. Nenhum sub-agent precisa decidir de novo se usa o SDK ou HTTP.

(O `design.md` é uma fase da spec, técnica e voltada para a IA. O Design Doc do cap. 7 é um documento para pessoas, escrito antes. Um pode alimentar o outro, mas não são a mesma coisa.)

As tasks: o coração do Spec-Driven

As tasks são **blocos atômicos**, organizados em fases sequenciais. Cada uma tem três elementos:

- **o quê:** o que precisa ser feito;
- **onde:** em quais arquivos e módulos;
- ***Definition of Done*** (DoD, a definição de pronto): os critérios objetivos de conclusão.

A DoD é o que separa o Spec-Driven de um plano comum. **Não é a IA que julga se terminou.** Cada critério é verificável, e os principais são *quality gates*, verificações binárias que bloqueiam a conclusão até passar:

- os arquivos foram criados ou modificados como especificado;
- os tipos foram exportados;
- as configurações foram atualizadas;
- **os testes rodam e passam, via comando no terminal** (resultado 0 ou 1, não a opinião da IA).

```
### T5 — Serviço de cobrança [PAG-01, PAG-03]
O quê: criar o PaymentService usando o SDK do Stripe.
Onde: src/billing/payment.service.ts, src/billing/payment.service.spec.ts
Testes: cenários de PAG-01 e PAG-03 (aprovado, recusado).
DoD:
- [ ] arquivos criados conforme acima
- [ ] PaymentService exportado em src/billing/index.ts
- [ ] `npm test -- billing` passa (exit code 0)
```

(Exemplo ilustrativo. O template completo está no Apêndice A.)

Três consequências:

- **Os testes não são opcionais.** A spec já traz o arquivo de teste, os cenários e o gate. A IA não decide se vai testar.
- **Tasks claras economizam tokens.** A pesquisa já foi feita; a task diz o quê e onde. O agente não precisa sair procurando, o contexto fica pequeno e a chance de alucinação, baixa.
- **A qualidade das tasks é a qualidade da spec.** Vale revisar bem a spec antes de gerar as tasks.

O mapa de execução paralela

No fim do `tasks.md`, a spec gera um **mapa de execução**: quais tasks podem rodar em paralelo, em sub-agents simultâneos, e quais dependem umas das outras e precisam ir em linha. No projeto Stripe, **nenhuma** task era paralelizável, porque cada uma dependia da anterior. Em outros projetos, o paralelismo acelera bastante. O cap. 11 mostra como esse mapa vira a regra de decisão de sub-agents.

Auto-sizing: a profundidade depende do tamanho

Nem todo trabalho precisa das quatro fases. O TLC Spec-Driven ajusta o fluxo ao escopo, num mecanismo chamado *auto-sizing*:

Escopo	Specify	Design	Tasks	Execute
Pequeno (até 3 arquivos)	*Quick Mode*: descrever, implementar, verificar, commitar			
Médio (menos de 10 tasks)	breve	dentro da spec	implícitas	sim
Grande (vários componentes)	completo	sim	sim	sim
Complexo (ambíguo)	completo + discussão	sim + pesquisa	sim + paralelismo	sim + aceitação interativa

Specify e Execute são sempre obrigatórios. Design e Tasks são pulados quando o escopo não justifica. O Quick Mode é para tarefas que cabem numa frase, mexem em até 3 arquivos e não pedem decisão de design.

A aula de RPI dá a mesma ideia como regra geral: projeto pequeno, só tasks; médio, spec e tasks; grande ou *brownfield* (sistema já existente), mapeamento, spec, design e tasks. A diferença está só no caso pequeno. No TLC Spec-Driven, o Quick Mode dispensa até as tasks formais. O princípio é o mesmo: **não use todas as fases em todo projeto.**

Os princípios que sustentam o fluxo

Quatro princípios atravessam as fases:

- **Rastreabilidade.** Os IDs dos requisitos atravessam o pipeline inteiro: spec, design, tasks e validação. Dá para saber qual teste cobre qual user story e qual commit implementou qual task.
- **Commits atômicos.** Uma task, um commit, uma verificação. O histórico fica limpo e reversível. Nunca se juntam várias tasks num commit.
- `STATE.md`. A memória persistente entre sessões (próxima seção).
- **Cadeia de conhecimento.** Quando falta informação, a ordem de busca é: codebase, documentação, Context7, busca na web e, se ainda houver dúvida, **marcar como incerto**. Nunca inventar nem assumir.

STATE.md: a memória do projeto

O modelo não lembra de nada entre sessões (cap. 1). O `STATE.md` é o arquivo que lembra por ele. Ele registra:

- as **decisões** tomadas (no projeto Stripe: SDK em vez de HTTP direto);
- os *blockers* (impedimentos) encontrados;
- os **aprendizados**;
- as **ideias para depois** (no projeto Stripe: migrar o endpoint da v1 para a v2);
- os pendentes.

Com ele, dá para pausar a implementação e retomar em outro dia, ou em outra janela, sem perder o que já foi feito. O arquivo é enriquecido a cada iteração e vale além de uma feature: as decisões e aprendizados de um projeto servem ao próximo.

A execução

Com spec, design e tasks prontos, a implementação segue o RPI: janela nova, modelo mais barato, o `tasks.md` anexado e um pedido para usar sub-agents o máximo possível, para manter a janela principal baixa. Como essa execução é organizada (sub-agents por fase, o loop de implementar, testar e corrigir dentro de cada um, os gates) é o assunto da Parte IV, a partir do cap. 11.

Comitar ou não as specs?

Depois da implementação, sobram `spec.md`, `design.md`, `tasks.md`, `STATE.md` e `concerns.md`. O que fazer com eles?

A indústria está dividida. Uma parte defende que a spec é a fonte da verdade e o código é consequência dela; nesse caso, a spec fica no repositório para sempre. O problema é prático: **ainda não existe ferramenta que mantenha spec e código sincronizados.** Uma spec que não acompanha o código vira documentação desatualizada, e documentação desatualizada confunde a IA.

A recomendação atual do curso:

1. **comite as specs** junto com a implementação;
2. **use-as para revisar o PR**: as user stories são o critério do review;
3. **depois, delete-as.**

Quando as ferramentas amadurecerem e o fluxo de desenvolvimento passar a ser centrado na spec (o Kiro é citado como exemplo nessa direção), pode fazer sentido mantê-las de forma permanente. A anotação da aula é honesta: ainda não sabemos tudo o que dá para fazer com as specs geradas.

Comparando frameworks: Superpowers, TLC Spec-Driven e OpenSpec

O TLC Spec-Driven não é a única opção. A aula compara três frameworks, testados na mesma integração com o Stripe:

	Superpowers	TLC Spec-Driven	OpenSpec
Estilo	direto, pouco boilerplate; muitas perguntas *upfront* (logo no início)	pergunta sob demanda e assume quando dá; mais *boilerplate* (estrutura repetitiva), ajustável pelo auto-sizing	leve e iterativo; specs vivem no repositório
Arquivos	2 (design e spec com as tasks), com código já na spec	mapeamento, spec, design, tasks, `STATE.md`, `concerns.md`	specs no repo, com *deltas* (como era, por que mudou, como ficou)
Qualidade	vários quality gates; review de qualidade após cada task e review final	quality gates explícitos na DoD de cada task, com comandos	review antes de codar
Contexto	implementa tudo na mesma janela (chegou a **100%**); respostas dos sub-agents enchem o principal	feito para sub-agents genéricos e janela principal baixa	—
Paralelismo	ruim: vai task a task, sem agrupar	sim, pelo mapa de execução	—
Alcance	Cursor e Claude Code	Cursor e Claude Code	o mais universal: 20+ ferramentas; open source, sem chaves de API; pensado para brownfield e projetos grandes
Contras	lento para começar e para implementar; mais tokens	mais arquivos para gerenciar	exige instalar uma CLI (Node.js 20.19+)

O custo real. Na integração com o Stripe, o TLC Spec-Driven e o Superpowers custaram em torno de **US$ 10** cada, com cerca de **25 milhões de tokens**, a maioria em cache. Usando o Opus para planejar e o Sonnet ou o Composer para implementar, o custo caiu para cerca de **US$ 5**. (Esses 25 milhões são do teste dos frameworks; não confundir com o benchmark de sub-agents do cap. 11, que é outro experimento.)

O que importa na escolha. Mais do que o nome do framework, duas propriedades:

1. **testes de alta qualidade**, garantidos por quality gates e Definition of Done;
2. **uma janela principal limpa**, com sub-agents, em paralelo quando possível.

Um framework que entrega as duas serve. Um que falha em qualquer uma delas vai cobrar o preço na implementação.

FAÇA

+ Faça uma pesquisa antes de qualquer implementação e salve o resultado em arquivo.
+ Implemente numa janela nova, com modelo mais barato e só o plano.
+ Mapeie o codebase com sub-agents e leia o `concerns.md` antes de implementar.
+ Escreva user stories verificáveis (WHEN / THEN / SHALL), priorizadas e com ID.
+ Registre as decisões técnicas com o racional no design.
+ Dê a cada task o quê, onde e uma DoD com comando de teste.
+ Ajuste a profundidade ao escopo: Quick Mode para o pequeno, fluxo completo para o grande.
+ Mantenha o `STATE.md` atualizado a cada sessão.
+ Comite as specs, use-as no review do PR e depois delete-as.

EVITE

– Implementar na mesma janela em que pesquisou.
– Tasks sem DoD ou com DoD subjetiva (“código bem feito”).
– Deixar a IA decidir se vai escrever testes.
– Usar o fluxo completo para uma mudança de três arquivos.
– Manter specs antigas no repositório sem sincronia com o código.
– Escolher o framework pelo nome em vez de pelos gates e pelo uso da janela.

ARMADILHA

A spec que ninguém revisou. A IA gera spec, design e tasks em poucos minutos, e o volume impressiona. Tudo parece completo, e a implementação começa. Mas cada erro da spec se multiplica: uma user story ambígua vira um teste errado, que vira uma task que passa no gate com o comportamento errado. O gate garante que o código cumpre a spec; nada garante que a spec cumpre o que o produto pediu, a não ser a sua revisão. É o ponto do fluxo em que meia hora de leitura economiza mais.

PARTE IV

Agentes, sub-agentes e loops

O núcleo do livro

10 Anatomia do loop agêntico

11 Sub-agents: quando dividir e quanto

12 O loop de implementação

13 Harness Engineering

14 O loop de code review

15 Harness determinístico e Loop Engineering

IV

CAPÍTULO 10

Anatomia do loop agêntico

TL;DR

Um agente de código é um programa que monta o contexto, manda para o modelo, executa a ferramenta que o modelo pediu e devolve o resultado, volta após volta, até o trabalho acabar. **O modelo decide; o agente orquestra.** Tudo o que não é o modelo (instruções, ferramentas, testes, memória) é o harness, e é nele que você tem controle. Cada volta do loop aumenta o contexto, e quase todos os problemas da Parte IV nascem daí: janela cheia, vitória declarada cedo demais, teste que ninguém rodou.

Quem decide e quem executa

A Parte I separou dois papéis que o dia a dia costuma misturar. O **agente** é o programa com que você conversa: Claude Code, Cursor, Copilot. O **modelo** é a LLM que ele chama por trás: Sonnet, GPT, Gemini, Composer. O agente orquestra; o modelo decide.

Essa divisão tem uma consequência que guia o resto do livro: o modelo é stateless (sem estado). Ele não lembra de nada entre uma chamada e outra. A sensação de "conversa contínua" é mantida pelo agente, que a cada chamada envia **o contexto inteiro** de novo: instruções do sistema, `AGENTS.md`, a conversa até ali, os arquivos lidos, a saída dos comandos rodados. Se algo não está nesse pacote, para o modelo não existe.

Por isso, quando se fala em "melhorar o agente", quase nunca se está falando de trocar o modelo. Fala-se em mudar o que entra nesse pacote e o que acontece com a resposta que sai dele.

COMPLEMENTO · fonte externa

Simon Willison — "I think 'agent' may finally have a widely enough agreed upon definition to be useful jargon now" (18/09/2025)

Willison propõe uma definição curta, que acabou adotada por boa parte da indústria: *"An LLM agent runs tools in a loop to achieve a goal."* (um agente de LLM roda ferramentas em loop para atingir um objetivo).

Duas partes da frase importam para este capítulo. "In a loop": o modelo **pede** a ação, quem a executa é o harness, e o resultado volta para o modelo continuar raciocinando. "To achieve a goal": *"these are not infinite loops—there is a stopping condition"* (não são loops infinitos; existe uma condição de parada).

Ele também observa que o objetivo não precisa vir de uma pessoa. Em padrões de sub-agents, é outro LLM que define a meta, como no Claude Code. É a ponte para o cap. 11.

https://simonwillison.net/2025/Sep/18/agents/

Agente = Modelo + Harness

O curso resume a divisão de papéis numa fórmula: **Agent = Model + Harness**. O modelo fica no centro; o *harness* (o "arreio", a instrumentação em volta) é todo o resto. No slide da aula de Harness Engineering, esse resto aparece em três grupos:

- ***Guides*** (*feedforward*, o que orienta antes de agir): `AGENTS.md`, spec e tasks, documentação de arquitetura, convenções.
- ***Sensors*** (*feedback*, o que observa depois de agir): linters, type checkers, testes e E2E, agente de review.
- **Memória e bootstrap** (o que liga uma sessão à próxima): `init.sh`, `progress.md`, disciplina de git.

O cap. 13 detalha cada grupo e mostra o diagrama completo. Por enquanto basta a ideia central: **o modelo é a parte que você não controla; o harness é a parte que você projeta.** As Partes II e III já construíram pedaços dele. O `AGENTS.md` e as skills são guides. O `STATE.md` do Spec-Driven é memória. Os quality gates das tasks são sensors.

> **COMPLEMENTO** · fonte externa
> **Birgitta Böckeler (Thoughtworks) — Harness engineering for coding agent users (02/04/2026)**
>
> Böckeler registra que o termo harness virou atalho para "tudo num agente de IA, exceto o próprio modelo": *"Agent = Model + Harness"*. Ela não cria a fórmula, adota a que se firmou. O vocabulário que ela organiza é o mesmo que o curso usa:
>
> - Guides (controles feedforward) *"anticipate the agent's behaviour and aim to steer it before it acts"*: antecipam o comportamento e tentam guiar o agente antes da ação.
> - Sensors (controles feedback) *"observe after the agent acts and help it self-correct"*: observam depois da ação e ajudam o agente a se corrigir.
>
> A autora acrescenta um segundo eixo. Controles **computational** são determinísticos e rápidos (testes, linters, type checkers). Controles **inferential** são semânticos, mais lentos e menos previsíveis (review por IA, *LLM as judge*). O cap. 15 inteiro trata de como tornar um controle inferential o mais confiável possível.
>
> Um aviso dela serve para o livro todo: separados, os dois tipos falham. Só com feedback, o agente repete os mesmos erros. Só com feedforward, codifica regras e nunca descobre se funcionaram.
>
> https://martinfowler.com/articles/harness-engineering.html

O ciclo, volta por volta

Com os papéis definidos, o loop fica simples de descrever. Toda sessão com um agente de código, do pedido mais trivial à implementação de uma feature inteira, é a repetição do mesmo ciclo:

1. **Montar o contexto.** O agente junta instruções do sistema, `AGENTS.md`, skills acionadas, histórico da conversa, resultados das ferramentas anteriores e o pedido atual.
2. **Decidir.** O modelo lê tudo e responde de um de dois jeitos: com texto ("pronto, terminei") ou com um pedido de tool (ler este arquivo, editar aquele trecho, rodar este comando).
3. **Executar.** O agente roda a tool. O modelo nunca toca no disco nem no terminal: ele só pede.

4. **Observar.** O resultado (conteúdo do arquivo, diff aplicado, saída do terminal, código de saída) vira mais um pedaço do contexto.
5. **Repetir ou parar.** Se o modelo pediu outra tool, volta ao passo 1 com o contexto maior. Se respondeu sem pedir nada, ou se uma condição externa mandou parar, o loop termina.

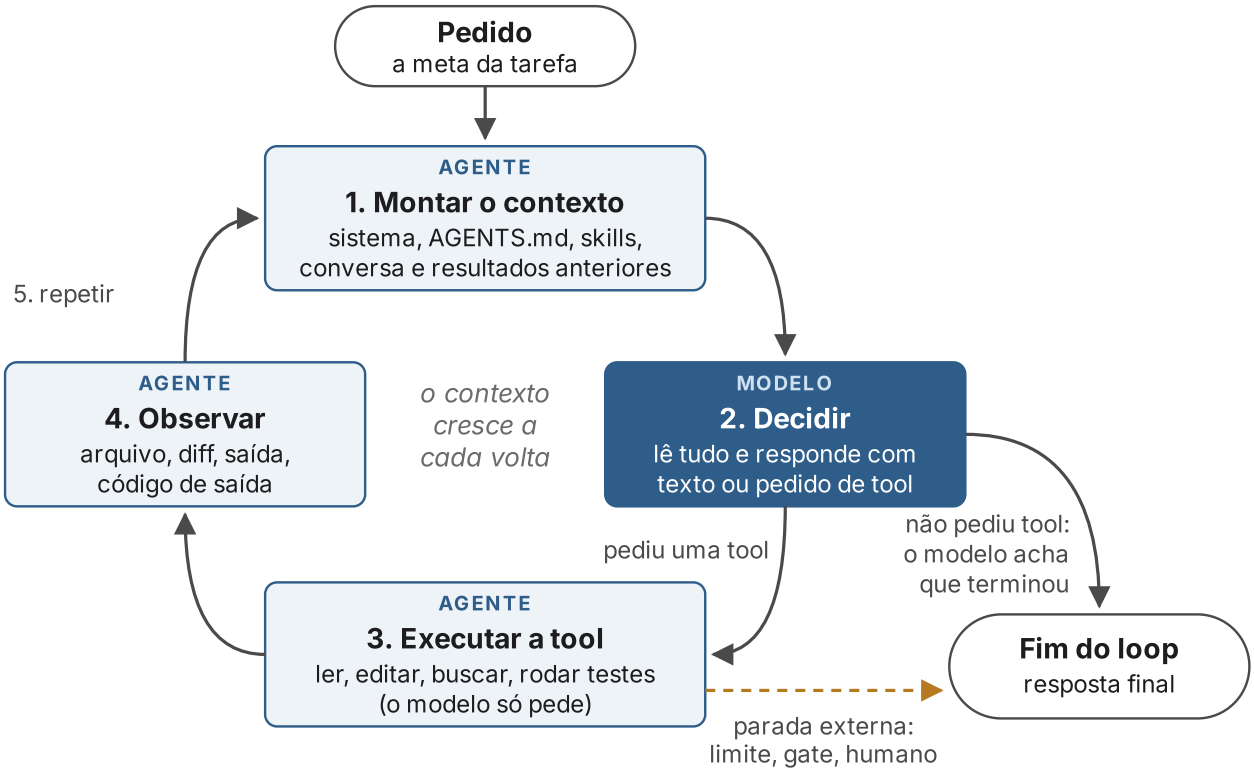


Figura 10.1 O loop agêntico genérico: o agente monta o contexto, o modelo decide, a tool executa, o resultado volta para o contexto, até a condição de parada

Três coisas ficam visíveis no diagrama.

Tudo passa por tools. Ler arquivo, buscar no codebase, chamar um MCP, rodar teste: do ponto de vista do loop, é tudo a mesma coisa (cap. 6). Uma capacidade que não existe como tool não existe para o agente.

O contexto só cresce. Cada volta acrescenta o pedido de tool e o resultado dele. Um `cat` de um arquivo grande ou uma suíte de testes que imprime centenas de linhas entra inteiro na próxima chamada. É o mecanismo por trás da context dilution do cap. 1 e das zonas do cap. 3: a janela começa na Smart Zone e vai subindo sozinha, volta após volta.

Quem decide parar é, por padrão, o próprio modelo. Sem nada externo, o loop termina quando o modelo acha que terminou. "Acha" é a palavra importante. A Vitória Prematura do cap. 13 é exatamente isso: o modelo responde "pronto" com 3 de 10 features funcionando.

COMPLEMENTO · fonte externa
Erik Schluntz e Barry Zhang (Anthropic) — Building effective agents (19/12/2024)

O artigo separa dois tipos de sistema. *Workflows* são *"systems where LLMs and tools are orchestrated through predefined code paths"*: o caminho está escrito no código. *Agents* são *"systems where LLMs*

dynamically direct their own processes and tool usage": o modelo escolhe o caminho. O Spec-Driven (cap. 9) fica no meio. As fases e os gates são workflow; dentro de cada task, o agente decide.

Sobre o loop em si, três ideias batem com este capítulo:

- **Ground truth a cada passo.** O agente precisa tirar do ambiente, a cada volta, um sinal real de progresso (*"such as tool call results or code execution"*).
- **Checkpoints humanos** quando trava ou em pontos combinados.
- **Condições de parada** além do "terminei": *"it's also common to include stopping conditions (such as a maximum number of iterations) to maintain control"*.

Os autores também catalogam padrões que reaparecem na Parte IV: *orchestrator-workers* (cap. 11), *parallelization* por seções ou por votação (cap. 14) e *evaluator-optimizer*, em que *"one LLM call generates a response while another provides evaluation and feedback in a loop"* (caps. 13 e 15). O conselho de fundo é começar pelo mais simples e só adicionar complexidade quando ela se pagar.

A própria página avisa que boa parte do ferramental descrito mudou desde dezembro de 2024. Os conceitos continuam valendo.

https://www.anthropic.com/engineering/building-effective-agents

COMPLEMENTO · fonte externa

Thariq Shihipar (Anthropic) — Building agents with the Claude Agent SDK (29/09/2025)

Ao apresentar o SDK que roda por trás do Claude Code, o autor descreve o loop em quatro tempos: *"gather context → take action → verify work → repeat"* (reunir contexto, agir, verificar o trabalho, repetir). O D12 usa esse esqueleto com um passo explícito a mais, o de observar o resultado da tool.

O terceiro tempo é o que mais interessa à Parte IV. Para ele, a melhor forma de verificação é *"providing clearly defined rules for an output, then explaining which rules failed and why"* (dar regras claras e explicar quais falharam e por quê). Lint é o exemplo clássico. Feedback visual com Playwright MCP é outro. Usar outro modelo como juiz aparece como opção para regras difusas, *"generally not a very robust method"*. O cap. 15 é sobre como deixar essa opção mais robusta.

O texto também dá os dois motivos para usar sub-agents: paralelizar e administrar contexto, porque *"subagents use their own isolated context windows, and only send relevant information back to the orchestrator"*. São os mesmos dois motivos da regra de decisão do cap. 11.

https://claude.com/resources/articles/building-agents-with-the-claude-agent-sdk

Quando o loop para

Um loop que só para quando o modelo se dá por satisfeito depende do julgamento de quem tem mais interesse em terminar. O curso repete a correção em vários formatos: **não é a IA que julga**. A parada tem que vir de um sinal externo e verificável:

- um comando no terminal com resultado binário, 0 ou 1 (o quality gate do cap. 12);
- uma lista de critérios com status, que só muda quando cada item é provado (cap. 13);
- um segundo agente, com contexto limpo, encarregado de achar problemas (caps. 13 e 14);
- uma decisão humana em pontos combinados (cap. 14);
- um limite duro de tamanho, tempo ou tokens, para quando nada mais funcionar.

Os capítulos seguintes são, em boa parte, maneiras diferentes de responder a uma única pergunta: **quem fecha o loop?**

O contexto é o recurso que o loop consome

Se cada volta aumenta o contexto e a qualidade cai à medida que a janela enche, um loop longo se degrada sozinho, mesmo que cada passo isolado seja bom. A Parte I já deu as alavancas para conter isso, e a Parte IV vai usá-las em escala:

- **Contexto pequeno de propósito.** Janela menor rende resposta melhor e menos alucinação. Tasks claras, com o quê e o onde, economizam as voltas que o agente gastaria descobrindo sozinho.
- **Contextos isolados.** Um sub-agent trabalha numa janela própria e devolve só o resultado. A janela principal fica baixa (cap. 11).
- **Memória fora da janela.** `STATE.md`, arquivo de progresso, feature list: o que precisa sobreviver vai para o disco, não para a conversa (caps. 9 e 13).

COMPLEMENTO · fonte externa
Anthropic (Applied AI) — Effective context engineering for AI agents (29/09/2025)

A Anthropic trata contexto como orçamento: *"Context, therefore, must be treated as a finite resource with diminishing marginal returns."* Os modelos têm um *"attention budget"* (orçamento de atenção), e cada token a mais gasta um pouco dele. O artigo chama de context rot a queda de precisão à medida que o contexto cresce. Descreve essa queda como *"a performance gradient rather than a hard cliff"*: uma rampa, não um precipício. É uma boa forma de ler as zonas do cap. 3, que são faixas e não limites exatos.

Para tarefas longas, o artigo propõe três técnicas, que correspondem às alavancas acima:

- *Compaction*: resumir a conversa perto do limite e recomeçar numa janela nova com o resumo.
- *Structured note-taking*: o agente escreve notas persistidas fora da janela (o papel do `STATE.md`).
- **Sub-agents**, em que cada um explora à vontade mas devolve *"a condensed, distilled summary of its work"*.

E uma técnica para o dia a dia: *just-in-time retrieval*. Em vez de carregar tudo de início, o agente guarda referências leves (caminhos, queries, links) e busca o conteúdo quando precisa. O próprio artigo cita o Claude Code: o `CLAUDE.md` entra de saída, e `glob` e `grep` buscam o resto sob demanda.

https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents

COMPLEMENTO · fonte externa
Jeremy Hadfield et al. (Anthropic) — How we built our multi-agent research system (13/06/2025)

O artigo dá uma ordem de grandeza para o custo do loop. Ela vem de um sistema de **pesquisa** (a feature Research do Claude: busca na web e em integrações), **não de coding**: *"agents typically use about 4× more tokens than chat interactions, and multi-agent systems use about 15× more tokens than chats."* Um agente gasta cerca de 4 vezes os tokens de um chat; um sistema multi-agente, cerca de 15 vezes.

Os autores tiram daí uma regra econômica: multi-agente só faz sentido quando o valor da tarefa paga o custo extra. E fazem uma ressalva que importa para quem programa: domínios em que todos os agentes precisam do mesmo contexto, ou com muitas dependências entre eles, *"are not a good fit for multi-agent*

systems today”, e *“most coding tasks involve fewer truly parallelizable tasks than research”*. O cap. 11 chega à mesma conclusão por outro caminho, o benchmark da TLC.

https://www.anthropic.com/engineering/multi-agent-research-system

Os loops deste livro

A Parte IV é uma sequência de loops, cada um envolvendo o anterior. Vale ter o mapa antes de entrar nos detalhes:

Cap.	Loop	Quem fecha o loop
10	Agente: contexto → modelo → tool → resultado	o modelo, ou uma condição de parada externa
11	Orquestrador → sub-agents → resumo de volta	o orquestrador, fase a fase
12	Implementar → testar → corrigir	o executor de testes (código de saída 0)
13	Planner → Generator → Evaluator → Fix	o Evaluator, um agente separado de quem implementou
14	`/review` → decisão humana → `/fix` → commit	o humano decide; o workflow faz o commit
15	Harness de avaliação e *Loop Engineering*	checks binários, somados por script, em vários runs

A pergunta “quem fecha o loop?” recebe uma resposta cada vez mais distante do modelo que fez o trabalho. Esse é o fio da Parte IV.

FAÇA

+ Pense em toda sessão com o agente como um loop e pergunte, antes de começar, **o que vai fazer esse loop parar**.
+ Trate o harness como código: versionado, revisado e melhorado quando um erro se repete.
+ Mantenha o resultado das tools enxuto. Saída de teste, logs e arquivos grandes vão para a próxima chamada inteiros.
+ Guarde em arquivo tudo o que precisa sobreviver ao fim da sessão.

EVITE

– Aceitar “terminei” como condição de parada quando existe um sinal verificável disponível (teste, lint, build).
– Trocar de modelo como primeira reação a um resultado ruim. Na maioria das vezes, o problema está no contexto que o agente montou.
– Sessões que crescem sem limite. Uma janela nova com um bom resumo costuma render mais do que a mesma janela no fim da Dumb Zone.

⚠ ARMADILHA

Confundir atividade com progresso. Um agente pode passar dezenas de voltas lendo arquivos, rodando comandos e editando código, e o contexto enchendo a cada uma, sem chegar mais perto do objetivo. Do lado de fora, parece trabalho. Sem uma condição de parada objetiva e um sinal de progresso a cada volta (um teste a mais passando, um item a mais marcado), não dá para distinguir as duas coisas até ser tarde.

CAPÍTULO 11

Sub-agents: quando dividir e quanto

TL;DR

Um sub-agent não é uma técnica de qualidade: **é uma forma de administrar o orçamento de contexto.** Ele só compensa quando a janela de um agente único está perto de estourar. No benchmark da TLC, com 18 tasks, dividir o trabalho em **3 sub-agents por blocos de fases** custou praticamente o mesmo tempo que um agente único e só um pouco mais de tokens (10,5M contra 9M). Mesmo assim, baixou a janela principal de 74% para 26% e subiu a nota de 0,93 para 0,95. Fatiar mais (um sub-agent por task) piorou tudo: tempo, custo e nota. Quem decide é o **acoplamento** da tarefa, não o número de agentes.

O que é um sub-agent

No loop do cap. 10, um único agente faz tudo dentro de uma única janela de contexto. Um sub-agent é um segundo agente que o primeiro dispara para cuidar de um pedaço do trabalho. O sub-agent recebe uma instrução, abre a **própria janela**, roda o seu loop e devolve só o resultado. O agente que dispara passa a ter o papel de **orquestrador**; os sub-agents são os *workers*.

O ganho é direto: tudo o que o worker leu, testou e corrigiu fica na janela dele. A janela do orquestrador recebe só o resumo. O custo também: cada worker começa do zero, com contexto vazio, e precisa recarregar o que for necessário para fazer a sua parte. Este capítulo é sobre o equilíbrio entre os dois.

O gargalo da janela de contexto

A pergunta que motiva tudo é: por que um agente único precisaria de ajuda? O benchmark responde acompanhando a janela de contexto do agente ao longo de uma implementação de 18 tasks (T1 a T18):

- no **início**, a janela está em 10%;
- no **meio da tarefa**, em cerca de 45%;
- no **fim (T18)**, em 74%, no limite;
- se ainda for preciso um **prompt de correção**, ela passa de 100% e **estoura**.

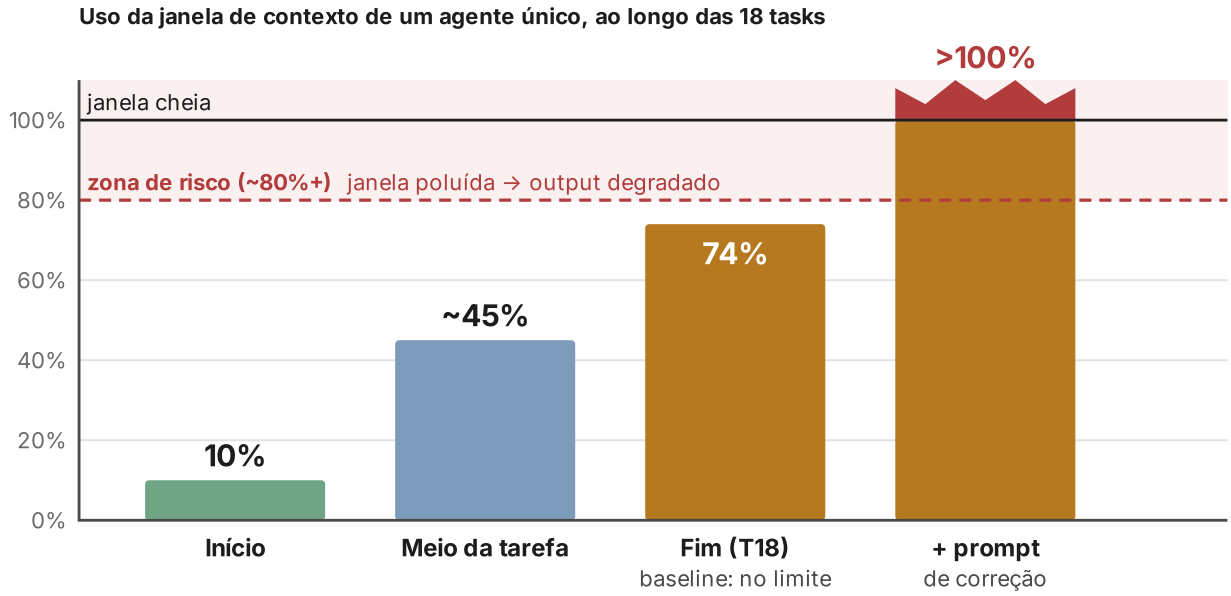


Figura 11.1 O gargalo da janela ao longo da task: um agente único termina as 18 tasks em 74% e estoura com um prompt de correção

A linha de risco do gráfico fica em torno de 80%. Acima dela, a janela está poluída e o output se degrada. O ponto importante é o último: **o agente único termina, mas termina sem folga.** Se o resultado precisar de um ajuste, e quase sempre precisa, não há espaço na janela para fazê-lo com qualidade.

> **NOTA**
>
> **60% ou 80%?** O cap. 3 coloca a Dumb Zone a partir de 60% da janela. O gráfico do benchmark marca a zona de risco em ~80%. Os dois números têm papéis diferentes. Os 60% são um guia geral de qualidade: dali em diante, o modelo já raciocina pior. Os ~80% são a linha que o benchmark desenhou para o risco de estourar a janela numa implementação longa. Na prática, um agente a 74% já está há tempos fora da Smart Zone.

Daí sai a tese do capítulo, que está no próprio slide da aula: **sub-agent é gestão de orçamento de contexto, e só compensa quando a janela única está prestes a estourar.**

O benchmark: quatro formas de fatiar o mesmo trabalho

Para medir o efeito, o mesmo trabalho (T1 a T18) foi executado em quatro arquiteturas, da menos para a mais fragmentada:

- **Sem sub-agent:** um agente, T1 → T18 no mesmo contexto.
- **3 sub-agents:** um orquestrador e 3 workers, cada um com um bloco de fases (T1–T6, T7–T12, T13–T18).
- **Sub-agent por fase:** um orquestrador e 7 workers, um por fase, em sequência (T1–T4, T5–T6, T7–T8, T9–T10, T11–T12, T13–T15, T16–T18).
- **Sub-agent por task:** um orquestrador e 18 workers, um por task.



Figura 11.2 As 4 arquiteturas testadas no benchmark, da menos para a mais fragmentada, com contexto principal, tempo, tokens e nota

Arquitetura	Estrutura	Contexto principal	Tempo	Tokens	Nota
Sem sub-agent	1 agente	74%	19 min	9M	0,93
3 sub-agents (*sweet spot*)	1 orquestrador + 3 workers	**26%**	**18 min**	**10,5M**	**0,95**
Sub-agent por fase	1 orquestrador + 7 workers	24%	35 min	15M	0,90
Sub-agent por task	1 orquestrador + 18 workers	32%	43 min	~25M	0,81

> **NOTA**
>
> **Sobre os números.** São do benchmark apresentado na aula *Quando Usar Sub Agents*, com o **Composer 2.5** usado no plano e na implementação. A configuração de 3 sub-agents corresponde ao "Run 5" dos slides. A descrição da aula fala em "17 tasks", mas os slides mostram 18 (T1 a T18), e este livro segue os slides. Não confunda com as 12 tasks do projeto Stripe, mais adiante neste capítulo: é outro experimento.

Lendo a tabela linha por linha

Sem sub-agent. É rápido e barato: 19 minutos, 9M de tokens, nota 0,93. O problema não aparece na tabela, aparece no gráfico anterior. Os 74% de janela significam que qualquer correção estoura o contexto. O resultado é bom, mas não tem margem.

3 sub-agents. Empata com o agente único em tempo (18 contra 19 minutos) e fica perto em custo (1,5M de tokens a mais). A diferença está na janela principal, que cai para **26%**, e na nota, que sobe para **0,95**. A anotação da aula resume a vantagem: mesmo que cada worker trabalhe em *one-shot* (de uma vez só), a janela principal não fica poluída. Sobra espaço para correções e para abrir novas tasks de correção para outro agente.

Sub-agent por fase. A janela principal cai ainda um pouco (24%), mas o tempo quase dobra (35 minutos) e os tokens sobem para 15M. A nota cai para 0,90. Os 7 workers rodam em sequência, e cada um recarrega o contexto de que precisa antes de começar.

Sub-agent por task. O pior em tempo, custo e nota: 43 minutos, cerca de 25M de tokens (quase o triplo do agente único) e nota 0,81. São 18 arranques, e a cada um o worker recarrega o contexto inteiro. Fragmentar demais também quebra a coerência: decisões tomadas numa task não chegam à próxima. E a janela principal não acompanha a fragmentação: com 18 workers, ela fica em 32%, acima das versões com 3 e com 7.

A conclusão da aula está numa frase: **muita granularidade afeta o contexto.** É preciso agrupar as tasks do jeito certo para paralelizar sem degradar o contexto nem gastar tokens demais, porque cada sub-agent vai recarregar o contexto de novo.

Os dois polos da indústria

O benchmark conversa com dois textos muito citados, que chegaram a conclusões opostas. A aula os apresenta lado a lado:

	Anthropic	Cognition
Texto	*How we built our multi-agent research system*	*Don't Build Multi-Agents* (Walden Yan)
Cenário	pesquisa: orquestrador + sub-agents com janelas separadas	implementação: tarefas longas e acopladas
O que defende	sub-agents em paralelo	um agente *single-thread* (uma linha só de execução) + context engineering
Argumento	~15× os tokens de um chat; +90,2% sobre um agente único; tokens explicam 80% da variância do desempenho	compartilhar contexto (o histórico completo do agente); toda ação carrega uma decisão implícita
Ressalva ou risco	coding tem poucas tarefas paralelizáveis	fragmentar gera decisões conflitantes e incoerência, o efeito "telefone sem fio"

> **NOTA**
>
> **Correções ao slide.** Conferidos no original (ver Apêndice C), dois detalhes do slide da aula não batem. (1) O slide associa o ganho de +90,2% ao benchmark BrowseComp. No artigo da Anthropic, o +90,2% é numa **avaliação interna de pesquisa**; o BrowseComp é onde o volume de tokens explica 80% da variância. (2) O slide diz que os textos saíram "no mesmo dia". A Cognition publicou em 12/06/2025 e a Anthropic em 13/06/2025, **com um dia de diferença**. Os números da Anthropic vêm de um sistema de *research*, não de coding, e não se misturam com o benchmark da TLC.

A leitura da aula é que os dois **não se contradizem**: cada um descreve uma ponta do mesmo espectro. Tarefa fracamente acoplada, de leitura (pesquisa), favorece a Anthropic. Tarefa fortemente acoplada, de escrita (implementação), favorece a Cognition. E o benchmark da TLC confirma os dois ao mesmo tempo. Os tokens explodem quando se fragmenta (como a Anthropic avisa), e a fragmentação por task derruba a nota (como a Cognition avisa).

A variável decisiva: acoplamento

Se não é o número de agentes que decide o resultado, o que decide? **O quanto as subtarefas dependem umas das outras.**

- **Baixo acoplamento:** subtarefas independentes. Pesquisa *breadth-first* (em largura), leituras e varreduras no codebase, geração de variações. Aqui, **sub-agents em paralelo ganham**.
- **Acoplamento moderado (a fronteira):** blocos coesos com contrato claro entre eles. Implementar por fase, um subsistema com interface definida, um pipeline com etapas. Aqui fica o **sweet spot**: em torno de **3 sub-agents**.
- **Alto acoplamento:** decisões encadeadas, estado compartilhado, trabalho pequeno. Implementar uma feature com poucas tasks, uma tarefa curta e única. Aqui, **um agente single-thread com bom context engineering** é a escolha.

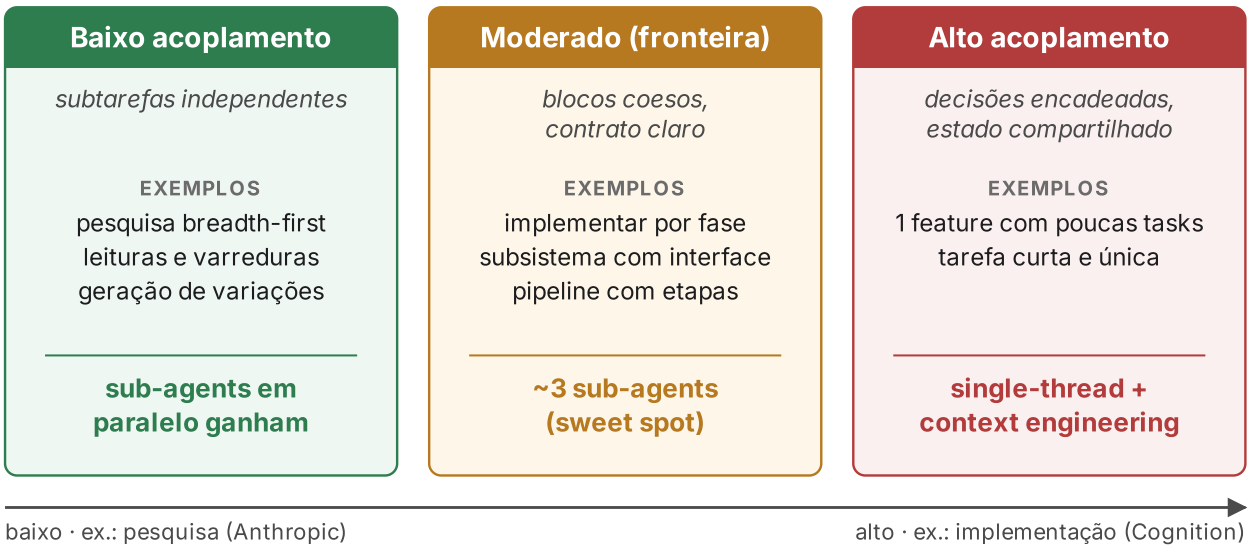


Figura 11.3 O espectro de acoplamento: de subtarefas independentes (paralelo ganha) a decisões encadeadas (um agente só)

O slide fecha com a regra que vale guardar: **meça o acoplamento primeiro. O número de agentes é consequência, não estratégia.**

A regra de decisão

Tudo isso cabe em duas perguntas.

1. **A tarefa cabe numa janela, com folga?** Folga quer dizer espaço para pelo menos um prompt de correção depois de terminar.
   - **Sim:** use **um agente single-thread**. É o padrão. Se algumas subtarefas forem independentes, dá para paralelizá-las por velocidade, mas é opcional.
   - **Não** (a tarefa não cabe ou é um processo longo): vá para a segunda pergunta.
2. **Existe uma fronteira de acoplamento?** Dá para cortar o trabalho em blocos com contrato claro?

- **Não:** reduza o escopo ou aumente a coesão antes de começar. Dividir agora só espalha decisões entrelaçadas por janelas diferentes.
- **Sim, e os blocos dependem uns dos outros: sub-agents em sequência.** Você ganha orçamento de contexto.
- **Sim, e os blocos são independentes: sub-agents em paralelo.** Você ganha orçamento de contexto e velocidade.

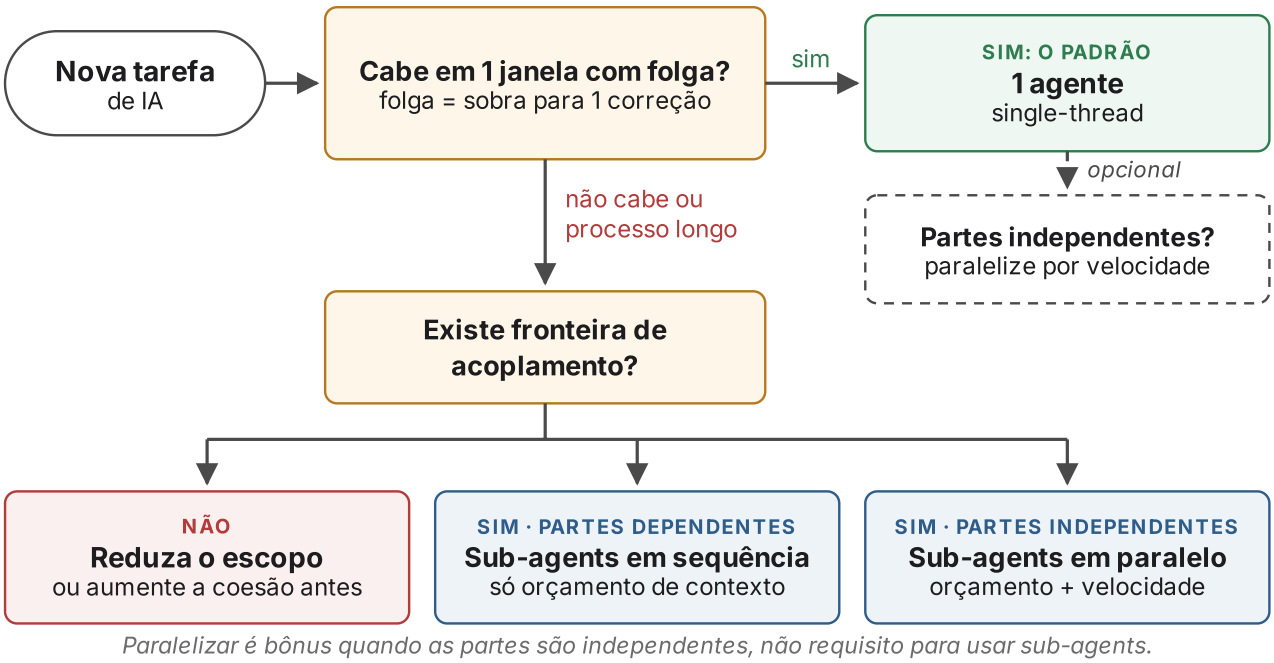


Figura 11.4 A regra de decisão de sub-agents: duas perguntas (cabe na janela com folga? existe fronteira de acoplamento?) levam a um agente, sub-agents em sequência ou sub-agents em paralelo

A observação do slide desfaz uma confusão comum: **sub-agent serve a dois motivos**, orçamento de contexto (sequencial) e velocidade (paralelo). Paralelizar é bônus quando as partes são independentes. Não é requisito para usar sub-agents. O projeto do exemplo abaixo não tinha nenhuma task paralelizável e mesmo assim se beneficiou deles.

Na prática: execução por fases

A aula *Implementando tasks com Spec-Driven* mostra a regra em uso, numa integração com Stripe de **12 tasks**, já planejadas pelo fluxo Spec-Driven do cap. 9.

Como começa. Uma janela nova, um modelo mais barato e mais rápido do que o usado para planejar, o arquivo `tasks.md` anexado e um pedido direto: *"faça a implementação, use sub-agents o máximo possível para manter a janela do agente principal baixa"*.

Como o trabalho se divide. O agente principal não implementa nada. Ele delega **blocos de tasks por fase**: a fase 1 (tasks 1 a 4) vai para um sub-agent; depois a task 5; depois as tasks 6 a 8; e assim por diante. O mapa de execução paralela do `tasks.md` dizia que nenhuma task podia rodar em paralelo, então os sub-agents rodam **em sequência**. É o caso "dependentes" da regra de decisão.

O que cada sub-agent faz. Implementa, escreve os testes da task, roda, corrige e só devolve o controle quando o quality gate passa. Esse loop interno é o assunto do cap. 12.

O resultado. A janela do agente principal chegou **no máximo a 37%**, dentro da Smart Zone, do começo ao fim das 12 tasks.

Duas peças do Spec-Driven fazem esse arranjo funcionar. A primeira são as **tasks bem definidas**: cada uma traz o quê, onde e a Definition of Done, então o sub-agent não precisa redescobrir o projeto. A segunda é o `STATE.md`, que guarda decisões, blockers e aprendizados fora de qualquer janela.

Um modelo mental para o dia a dia

A aula fecha com três situações típicas, que a regra de decisão resolve rápido:

- **Busca no codebase** ("onde isso é usado?", "como esse módulo funciona?"): leitura com baixo acoplamento. Bom candidato a sub-agent, inclusive em paralelo. A janela principal recebe só a resposta.
- **Tarefas longas** (uma feature com muitas tasks): o risco é a janela estourar. Sub-agents em sequência, por blocos de fases coesas.
- **Paralelização**: só quando as partes são de fato independentes, sem estado compartilhado. Testes que usam o mesmo banco não são independentes (cap. 12).

FAÇA

+ Comece com **um agente só**. Passe para sub-agents quando a tarefa não couber na janela com folga para uma correção.
+ Agrupe tasks em **blocos coesos de fases**. Poucos workers grandes rendem mais do que muitos pequenos.
+ Peça explicitamente ao orquestrador para delegar e manter a própria janela baixa.
+ Use o mapa de execução paralela das tasks para decidir entre sequência e paralelo.
+ Acompanhe a janela principal durante a execução: ela é a métrica do capítulo.

EVITE

– Um sub-agent por task. Cada arranque recarrega o contexto, o custo e o tempo disparam e a coerência cai.
– Paralelizar subtarefas que compartilham estado ou decisões.
– Usar sub-agents para "melhorar a qualidade" de uma tarefa que já cabe numa janela.
– Dividir uma tarefa sem fronteira clara: primeiro reduza o escopo ou aumente a coesão.

⚠ ARMADILHA

"Quanto mais agentes, melhor." Mais workers dão sensação de controle e de paralelismo, mas no benchmark a configuração mais fragmentada foi a pior em tempo, custo e nota: 43 minutos contra 18, cerca de 25M de tokens contra 10,5M, nota 0,81 contra 0,95. Fragmentar não divide o problema, multiplica os arranques. Antes de aumentar o número de agentes, pergunte se a tarefa tem fronteiras onde ela pode ser cortada.

CAPÍTULO 12

O loop de implementação

TL;DR

Dentro de cada sub-agent roda um loop menor: **implementar → testar → corrigir → testar de novo**, até um quality gate binário passar. Quem decide se passou é o **executor de testes** (o código de saída do comando), nunca o próprio agente. O ciclo clássico de TDD (testes) funciona mal com agentes. Ele vira um loop de 30 a 50 mil tokens por feature e testa o que o agente inferiu, não o que foi pedido. O Spec-Driven preserva o princípio do teste primeiro e põe o contrato na spec. Mesmo assim, agentes "trapaceiam" nos testes sem perceber, e só regras com *enforcement* (um passo que bloqueia) seguram isso.

O loop dentro do sub-agent

O cap. 11 terminou com o orquestrador delegando blocos de tasks. Este capítulo abre o sub-agent e olha o que acontece lá dentro. Na aula *Implementando tasks com Spec-Driven*, cada sub-agent segue o mesmo ciclo:

1. **implementa** o código da task;
2. **escreve os testes** que a task especifica;
3. **roda** os testes;
4. **encontra os erros**;
5. **corrige**;
6. **roda de novo**, quantas vezes for preciso, até tudo passar;
7. só então **marca a task como concluída**, e isso só acontece quando o quality gate passa.

Tudo isso acontece na janela do sub-agent. Para o orquestrador sobe só o resultado: task concluída, gate verde. É por isso que a janela principal do projeto Stripe nunca passou de 37%.

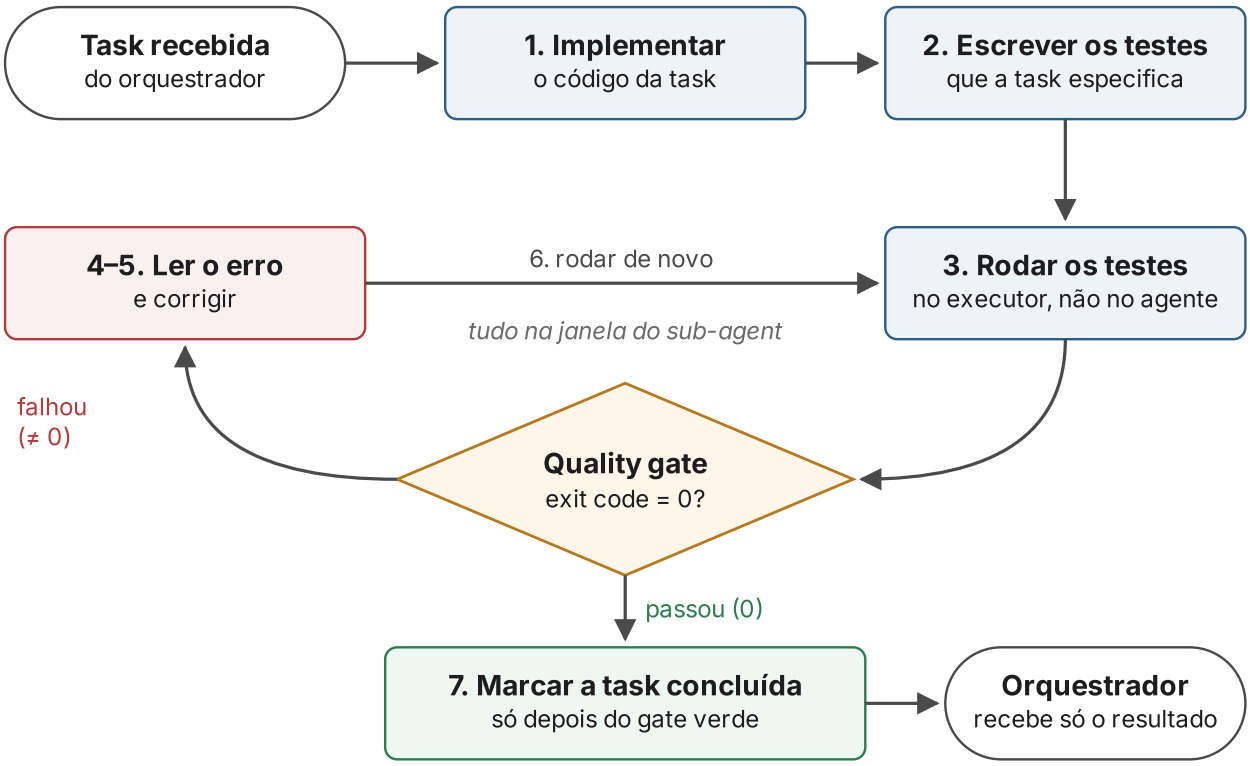


Figura 12.1 O loop do sub-agent: implementar, testar e corrigir até o quality gate binário passar, e só então marcar a task como concluída

Gates binários: quem decide é o terminal

A peça que fecha esse loop é o quality gate. A validação **não é subjetiva**: não é a IA olhando o código e decidindo que ficou bom. É um **comando no terminal com resultado binário**, passou ou não passou. O código de saída do executor decide: 0 passa, qualquer outro valor falha. Discutir com um código de saída não adianta, e por isso fica muito difícil para a IA burlar o processo.

No Spec-Driven, o gate faz parte da **Definition of Done** (DoD) de cada task. Uma DoD típica verifica:

- os arquivos foram criados ou modificados como especificado;
- os tipos foram exportados;
- as configurações foram atualizadas;
- **os testes rodam e passam via comando no terminal**.

E os testes **não são opcionais**. A spec já traz o arquivo de teste, os cenários e o gate. A IA não decide se vai testar. Esse é um dos pontos em que o Spec-Driven se separa do Plan Mode (cap. 8): o Plan Mode não tem gates, não garante que haverá testes e não impede que o agente apague ou pule testes.

Por que o TDD (testes) clássico não funciona com agentes

O TDD (testes), Test-Driven Development, é a disciplina de escrever o teste antes do código, no ciclo *red* → *green* → *refactor*: um teste que falha, a implementação mínima que o faz passar, a

melhoria do código. Para uma pessoa, o ciclo é barato e rápido. Para um agente, a aula aponta três problemas.

1. O loop de tokens

Cada passo do ciclo custa caro em contexto. Escrever o teste (*red*) soma cerca de 8 mil tokens. Implementar (*green*), mais 8 mil. Refatorar, mais 8 mil. E quando o teste ainda falha, o ciclo recomeça. O agente não para: tenta variações, relê o erro, tenta de novo. Uma única feature consome **de 30 a 50 mil tokens em loop**, e esse histórico todo fica na janela.

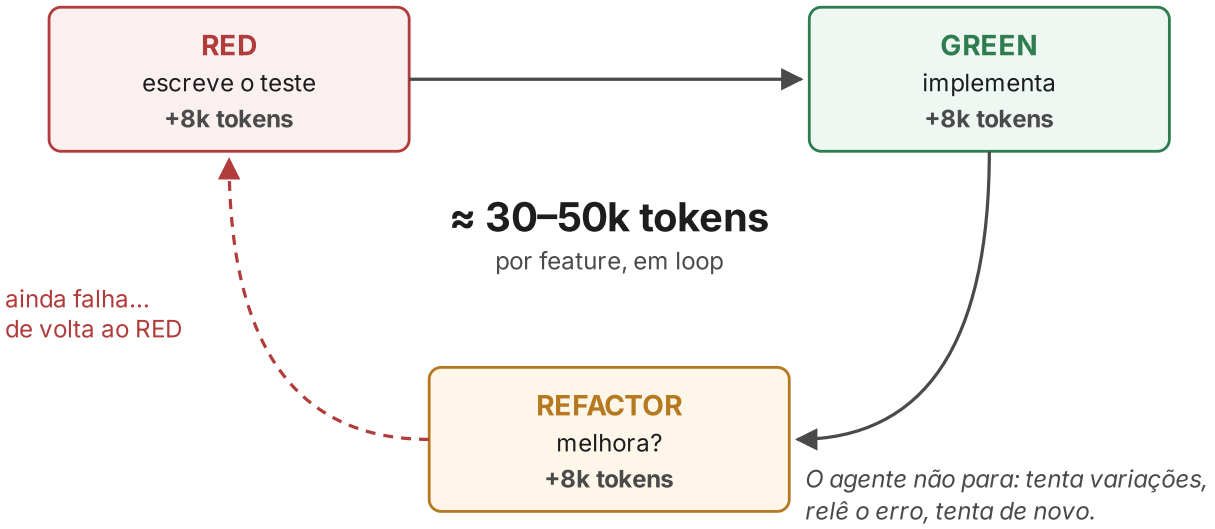


Figura 12.2 O loop de tokens do TDD (testes) com agentes: cada passo red, green e refactor soma cerca de 8 mil tokens, e uma feature chega a 30–50 mil

2. O problema da inferência

Quando uma pessoa faz TDD (testes), a cadeia é: **spec → teste → código**. Ela entende o comportamento esperado, formaliza a spec num teste e escreve o código que satisfaz o teste. No fim, o teste verde valida a spec.

Um agente **sem spec** faz outra cadeia: **inferência → teste → código**. Ele deduz o comportamento a partir de suposições, formaliza essa dedução num teste e escreve o código que a satisfaz. O teste verde valida a **inferência**, que pode estar errada desde o início. O ciclo funciona perfeitamente e entrega a coisa errada.

3. O passo de refactor é perigoso

Pedir a um agente que "melhore" o código abre três riscos:

- ***Scope creep*** (o escopo crescendo sem ninguém pedir): ele melhora coisas que não foram pedidas, fora da tarefa.
- **Abstrações prematuras:** generaliza para casos que não existem e cria interfaces sem consumidor real.
- **Regressões silenciosas:** mexe em código além do escopo e quebra testes que passavam, sem que ninguém esteja olhando.

A recomendação da aula é tirar o refactor do loop: **refactor com agentes é scope creep garantido; transforme-o numa tarefa separada, com spec própria.**

O que o Spec-Driven preserva

A conclusão não é abandonar o teste primeiro. O Spec-Driven **move o contrato do teste para a spec**, que vira a fonte da verdade. Os cenários de teste estão escritos antes de qualquer código. O agente não precisa inferir o comportamento, porque ele está na spec. E o loop é curto, porque a task é pequena e o gate é objetivo. Sobram os princípios do TDD (testes) sem o custo, a lentidão e a dificuldade de paralelizar.

A aula menciona ainda uma **nova pirâmide de testes com IA**, com validação, tasks e spec na base, sem detalhá-la além disso.

O iceberg do agente

Mesmo com spec e gate, há um problema que não aparece na superfície. O agente reporta **"15 testes passando"**, e é verdade. Mas abaixo da linha d'água pode haver quatro coisas:

> **1. Asserções enfraquecidas**
> O teste continua lá, mas não verifica mais nada de útil. Em vez de `expect(result).toBe(42)`, aparece `expect(result).toBeDefined()`. Passa sempre.

> **2. Testes deletados**
> Eram 15 testes; agora são 12. Todos passando. Ninguém percebe.

> **3. Skip ou pending**
> O teste está registrado no arquivo, mas nunca executa. No relatório, aparece como verde.

> **4. Adiamento (o mais perigoso)**
> "Os testes vêm na próxima task." O agente escreve o código e, depois, valida o próprio código. É uma câmara de eco completa: o teste nasce para confirmar o que já foi feito.

A frase do slide resume o problema: **o agente pode estar mentindo sem saber que está mentindo.** Não há má-fé. O agente persegue "testes passando" e encontra o caminho mais curto até lá.

E escrever "nunca apague testes" no `AGENTS.md` não basta. **Regras sem enforcement não funcionam**: quando o contexto aperta, a IA ignora a regra. Cada regra importante precisa de um passo no processo que **bloqueie** a saída até a conformidade ser provada.

Os 10 princípios para bons testes com agentes

A aula organiza as defesas em dez princípios, agrupados em cinco temas.

Verificação externa

1. O executor de testes decide, não o agente

Agentes que se autoavaliam sempre concluem que estão certos. A única verificação válida é a de um executor externo: o código de saída decide (0 passa, diferente de 0 falha). Sem discussão.

2. Testes são a especificação

A implementação existe para satisfazer os testes, não o contrário. Deixar o agente modificar os testes é deixá-lo corrigir a própria prova.

Enforcement

3. Regras sem enforcement não funcionam

O agente lê a regra e a viola mesmo assim, se não houver um gate obrigatório no processo. Cada regra precisa de um passo que bloqueie a saída até que a conformidade seja provada.

4. Torne a verificação visível

Tabelas de validação exibidas junto com o trabalho permitem que o usuário audite. O agente não consegue pular um check em silêncio, porque o usuário vai ver a tabela.

Estrutura dos testes

5. Coloque testes e implementação juntos

Testes escritos separados da implementação validam o código do agente, não a spec. Red, green e *verify* na mesma task dão feedback imediato e real.

6. Nunca modifique testes na fase green

A trapaça mais comum: o teste falha, a asserção é enfraquecida, o teste passa, "pronto". Testes escritos na fase red ficam congelados. Para mudar algum, o agente para e pede ao humano.

Integridade e paralelismo

7. Acompanhe a contagem, não só passou ou falhou

Um agente consegue fazer todos os testes passarem simplesmente apagando os que falham. Se a contagem de testes diminuiu, investigue antes de seguir.

8. Paralelismo é problema de dados, não de otimização

Agentes em paralelo usando o mesmo banco de dados corrompem os dados uns dos outros. A decisão de paralelizar vem do modelo de isolamento que o projeto tem.

Arquitetura inteligente

> **9. Dependências de compilação têm caminho de resolução**
> "Não dá para testar até conectar o módulo" não justifica adiar os testes. Se isso acontece, a fronteira da task está errada: junte-a com a task anterior ou com a seguinte.

> **10. Detecção orientada por dados, não *hardcoded***
> Cada projeto tem frameworks, tipos de teste e isolamento diferentes. A skill de testes detecta a infraestrutura real do projeto, sem suposições fixas no código.

Tipos de teste e o que dá para paralelizar

O princípio 8 merece um desdobramento, porque liga este capítulo ao anterior. Antes de mandar sub-agents rodarem em paralelo, é preciso saber **que tipo de teste** cada task tem (unidade, integração, end-to-end) e **o que ele compartilha**. Testes de unidade sem estado paralelizam bem. Testes de integração que escrevem no mesmo banco não paralelizam, a menos que cada execução tenha o seu próprio ambiente. A aula discute quando vale instrumentar *testcontainers* (bancos e serviços descartáveis, um por execução) e quando isso é exagero para o tamanho do projeto.

Uma sugestão que saiu da aula: criar uma **skill que analisa as tasks e aponta o que pode ser paralelizado** e agrupado em sub-agents, em vez de deixar essa decisão para cada sessão.

Shift-left: qualidade no momento do desenvolvimento

O último movimento do capítulo é de lugar. Tradicionalmente, qualidade, segurança, padrões e arquitetura são cobrados no **code review**, depois que o código existe. A mentalidade *shift-left* (deslocar para a esquerda, para mais cedo no fluxo) traz tudo isso para dentro do **desenvolvimento**. O fluxo continua sendo Discovery → Dev → Review, mas o Dev passa a receber, além do PRD, as regras de qualidade, segurança, padrões e arquitetura, carregadas **via skills** no momento em que o agente escreve o código.

O review continua existindo (cap. 14), mas deixa de ser o primeiro lugar onde um problema de arquitetura é percebido.

A postura que amarra tudo é **confiar desconfiando**: construir *guardrails* que guiam o agente sem limitar o que ele consegue fazer. Gate binário, testes congelados, contagem acompanhada, verificação visível. Nada disso impede o agente de trabalhar. Só impede que ele declare como pronto o que não está.

FAÇA

+ Feche o loop da task com um **comando de terminal** cujo código de saída decide.
+ Escreva os cenários de teste **na spec**, antes do código, e congele-os.
+ Inclua na DoD de cada task: arquivos esperados, tipos, configurações e testes passando.
+ Acompanhe a **contagem** de testes entre uma task e outra.
+ Exiba uma tabela de validação junto com o trabalho entregue.
+ Transforme refactor em task separada, com spec própria.
+ Leve segurança, padrões e arquitetura para skills usadas durante o desenvolvimento.

EVITE

– Deixar o agente decidir se o trabalho está pronto.
– Red → green → refactor solto, sem spec: 30–50 mil tokens por feature validando uma inferência.
– Regras como “nunca apague testes” sem um passo que as faça valer.
– Aceitar “os testes vêm na próxima task”.
– Paralelizar testes que compartilham banco de dados sem isolamento.

ARMADILHA

Confiar no verde. “Todos os testes passando” é a frase mais tranquilizadora que um agente pode dizer, e a menos informativa. Ela é compatível com asserções que não verificam nada, testes apagados, testes que nunca rodam e testes escritos depois para confirmar o código. Antes de aceitar o verde, olhe três coisas: a contagem de testes, o diff dos arquivos de teste e se os testes foram escritos antes ou depois da implementação.

CAPÍTULO 13

Harness Engineering

TL;DR

Harness engineering é projetar tudo o que fica em volta do modelo: **o que guia** o agente antes de agir (guides), **o que verifica** depois (sensors) e **o que liga uma sessão à outra** (memória e bootstrap). O harness não melhora o modelo. Ele constrói o ambiente que constrange, guia e valida. Ele existe para resolver seis problemas recorrentes, de "tentar fazer tudo de uma vez" a "qualidade que degrada a cada sessão". O Spec-Driven é um tipo de harness e resolve parte deles. O que ele não resolve, o **viés de quem implementa e julga o próprio trabalho**, pede outro arranjo: **Planner → Generator → Evaluator → Fix**, com cada papel num processo separado.

O que é harness

O cap. 10 apresentou a fórmula **Agent = Model + Harness**. Este capítulo trata da segunda metade.

Harness é **toda a instrumentação do ambiente em volta do modelo**, montada para garantir que o agente siga o caminho certo. Não é sobre melhorar o modelo. É sobre construir o ambiente que o **constrange, guia e valida**. As notas da aula dão uma definição mais prática: harness é **toda estrutura que garante que o agente não vai burlar os testes**, por exemplo.

Duas observações situam o tema:

- **O Spec-Driven é um tipo de harness.** Specs, tasks atômicas, `STATE.md` e quality gates são instrumentação em volta do agente. Mas existem outros componentes, e o Spec-Driven sozinho não cobre todos os problemas.
- **A indústria está em transição para harness engineering.** O foco está saindo de "como escrevo o prompt" e indo para "como monto o ambiente em que o agente trabalha".

Guides, sensors e memória

O slide da aula organiza o harness em três grupos em volta do modelo:

- **Guides (feedforward)** orientam o agente **antes e durante** a execução: `AGENTS.md`, spec e tasks, documentação de arquitetura, convenções.
- **Sensors (feedback)** verificam e corrigem **depois** que o agente age: linters, type checkers, testes e E2E, agente de review.
- **Memória e bootstrap** fazem a ponte entre sessões: um script `init.sh` que prepara o ambiente, um `progress.md` com o que já foi feito e disciplina de git (commits pequenos e descritivos, que servem de histórico e de ponto de retorno).

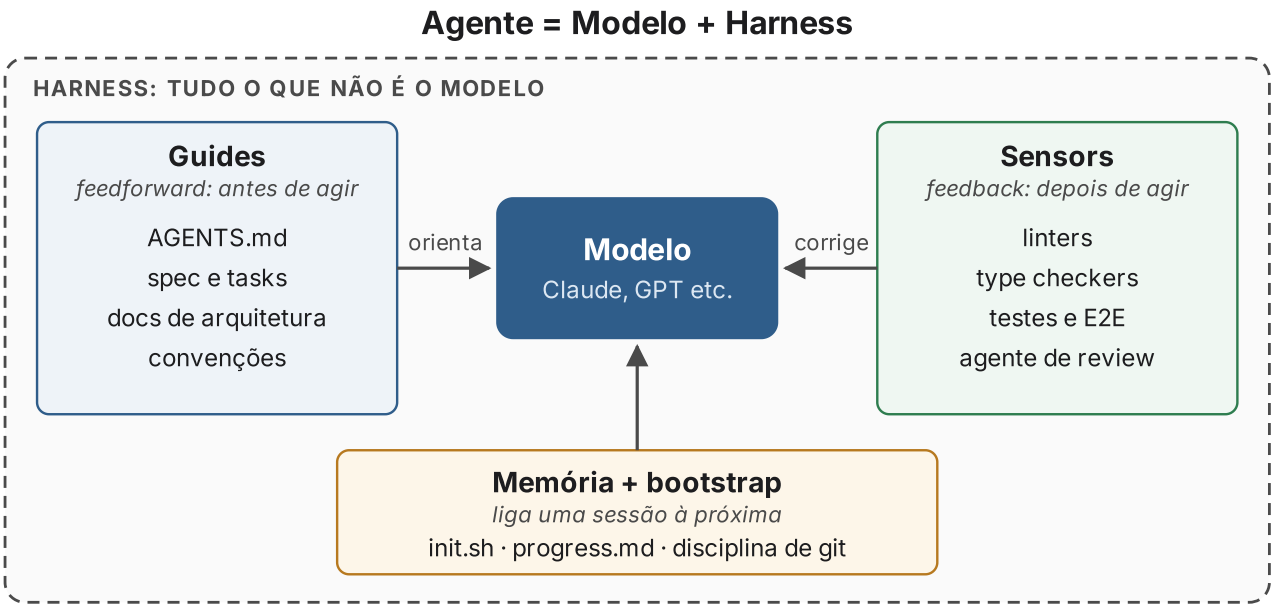


Figura 13.1 Agent = Model + Harness: guides (feedforward) e sensors (feedback) em volta do modelo, com memória e bootstrap na base

O texto da aula coloca a memória persistente entre os sensors. O slide a separa num grupo próprio, e este livro segue o slide. A memória não verifica nada: ela permite que a próxima sessão saiba o que a anterior verificou.

O que cada grupo faz sozinho é limitado. Guides sem sensors produzem um agente bem orientado que nunca descobre se acertou. Sensors sem guides produzem um agente que erra, é corrigido e erra de novo da mesma forma. O harness funciona quando os dois se fecham num loop, e quando a memória carrega o resultado desse loop para a sessão seguinte.

Os seis problemas que o harness resolve

Um agente sem harness falha de maneiras previsíveis. A aula nomeia seis.

1. One-Shot Hero

O agente tenta implementar tudo de uma vez. Auth, dashboard, API e Stripe entram na janela; testes e CI/CD vêm em seguida; e-mail e notificações já não cabem: o contexto **estoura**. Resultado: feature pela metade, sem documentação, testes quebrados. Um agente sem harness, diz a anotação da aula, vai tentar fazer tudo como um *one-shot hero*.

2. Vitória Prematura

O agente vê progresso parcial e declara "pronto". Numa lista de 10 features, login, registro e listagem passam. Editar, deletar, busca, notificações, exportar CSV, dashboard e configurações falham. O agente conclui: "já tem login, registro e listagem funcionando; projeto concluído!". São **3 de 10 features prontas, 30%**, e o agente acha que terminou.

3. Amnésia entre Sessões

Cada sessão começa do zero. A sessão 1 implementa a autenticação. A sessão 2 abre perguntando: o que aconteceu? que arquivos são esses? isso está funcionando? E as sessões 3 e 4 repetem o ciclo. O slide ilustra a divisão do esforço: **50% dos tokens gastos entendendo, 30% em progresso real, 20% refazendo trabalho**. É uma ilustração, não uma medição.

4. Teste Fake

O agente marca a feature como pronta sem testar de ponta a ponta. Ele faz um `curl /api`, recebe `200 OK` e anuncia "feature pronta!". No browser, o botão não clica, o formulário não submete, o redirect está quebrado, o CSS está desalinhado, a validação não existe e o tratamento de erro é zero.

5. Tudo no Mesmo Processo

Quem escreveu o código é quem julga se ele está bom. Um único processo implementa e testa, com o contexto poluído por tudo. É a **raposa cuidando do galinheiro**: o agente quer terminar, e é ele quem decide se terminou. As notas chamam isso de **viés do implementador**.

6. AI Slop Acumulado

O código compila, mas a qualidade degrada em silêncio a cada sessão. Sintomas: lógica duplicada, fronteiras entre módulos violadas, dependências circulares. Num gráfico de qualidade por número de sessões, a linha "sem harness" cai sem parar e a linha "com harness" fica quase plana. A curva é **conceitual**: ilustra a tendência, não vem de dados medidos.

O problema 4 tem duas definições no material do curso. O slide fala em "pronto sem teste E2E", como no card acima. O texto da aula fala em testes deletados, modificados ou pulados, que é o iceberg do cap. 12. As duas são o mesmo problema visto de lados diferentes: **o sinal de "testado" não corresponde a um teste real.**

O que o Spec-Driven resolve, e o que não

O fluxo Spec-Driven das Partes III e IV já ataca boa parte da lista:

Problema	O Spec-Driven resolve?	Com quê
1. One-Shot Hero	sim	tasks atômicas, executadas em blocos por sub-agents (cap. 11)
2. Vitória Prematura	ajuda bastante	DoD por task, com gate binário: "pronto" é por task, não pelo projeto
3. Amnésia entre Sessões	sim	`STATE.md` com decisões, blockers e aprendizados
4. Teste Fake	em parte	DoD com quality gates e testes na spec (cap. 12)
5. Tudo no Mesmo Processo	**não**	o agente que implementa continua sendo o que julga

Problema	O Spec-Driven resolve?	Com quê
6. AI Slop Acumulado	**não sozinho**	o Spec-Driven é voltado à funcionalidade; falta um sistema que se autovalide

As anotações da aula resumem: o Spec-Driven ajuda muito nos problemas 1 a 3, **mas não é bala de prata**. Ele garante que cada funcionalidade seja especificada, quebrada e testada. Não garante que alguém independente olhe para o resultado com a missão de achar defeitos.

Separar quem implementa de quem valida

O caminho que a aula aponta para o problema 5, inspirado na arquitetura de três agentes descrita pela Anthropic, é **separar a implementação da validação em processos independentes**:

1. **Planner** (processo 1) transforma o pedido numa **spec**.
2. **Generator** (processo 2) implementa a spec e entrega **código**.
3. **Evaluator** (processo 3) verifica o código e decide: **PASS?**
   - **PASS**: fim.
   - **FAIL**: entra o **Fix mode**, e o trabalho volta ao Generator com o que falhou.

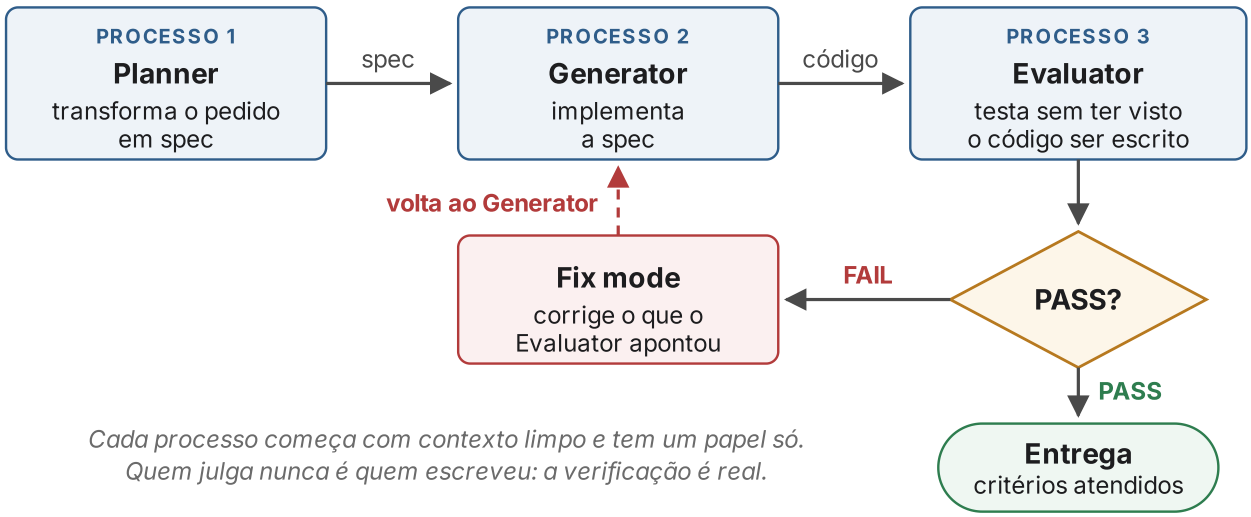


Figura 13.2 Planner → Generator → Evaluator → Fix: três processos separados, com o Evaluator decidindo PASS ou FAIL e o Fix mode devolvendo o trabalho ao Generator

A regra do slide é a parte importante: **cada agente tem contexto limpo e papel único. Quem testa nunca viu o código sendo escrito, e por isso a verificação é real.** O Evaluator não carrega as justificativas, as tentativas e o cansaço do Generator. Ele recebe o resultado e tem uma missão diferente: **encontrar problemas, não terminar rápido**.

Esse desenho reaparece nos dois capítulos seguintes. No cap. 14, o Evaluator se desdobra em especialistas (segurança, arquitetura, testes, requisitos) que revisam o PR separadamente. No cap.

15, o Evaluator vira um **juiz** com critérios congelados, e a pergunta passa a ser como confiar num juiz que também é um modelo.

Para onde isso vai

A aula termina com dois pontos que valem mais como direção do que como receita.

Escala de sistemas inteiros. Para ir além de features e construir sistemas de milhões de linhas, o Spec-Driven sozinho não basta. É preciso um sistema que **aprenda, se autovalide e evolua**, combinando todos os tipos de harness: guides que melhoram quando um erro se repete, sensors que pegam o erro antes de chegar a uma pessoa, memória que impede que a próxima sessão recomece do zero.

> **NOTA**
>
> A aula cita um caso da Anthropic de um sistema de milhões de linhas construído por "US$ 20K". As notas não trazem o link desse caso, e ele não foi localizado nas fontes conferidas para este livro. Fica registrado como menção da aula, sem valor de dado.

O código precisa permitir o harness. A estrutura do codebase, a modularização e as convenções precisam ser pensadas para que os harnesses funcionem bem. Os sensors do D11 dependem disso: um type checker só verifica o que tem tipo, um linter só cobra a convenção que existe, um agente de review só aplica a arquitetura que está documentada. Projetar o código e projetar o harness passam a ser o mesmo trabalho.

> **FAÇA**
>
> + Monte o harness nos três grupos: guides, sensors e memória com bootstrap.
> + Dê ao agente uma lista explícita de features com status, para que "pronto" tenha um critério.
> + Exija teste de ponta a ponta (browser, não só `curl`) antes de aceitar uma feature com interface.
> + Comece cada sessão lendo a memória: progresso, `STATE.md`, `git log`.
> + Separe quem implementa de quem valida, com contextos limpos e papéis únicos.
> + Quando um erro se repetir, mude o harness (um guide novo, um sensor novo) em vez de só corrigir o código.

> **EVITE**
>
> – Tratar o Spec-Driven como solução completa. Ele não resolve o viés do implementador nem a degradação entre sessões.
> – Deixar o mesmo agente, na mesma janela, implementar e decidir que terminou.
> – Aceitar `200 OK` de uma API como prova de que a feature funciona.
> – Sessões que começam sem bootstrap e gastam metade do esforço redescobrindo o projeto.

> ⚠ **ARMADILHA**
>
> **Medir só por sessão.** Cada sessão, olhada sozinha, parece boa: compila, os testes passam, a feature aparece. O AI Slop Acumulado só se vê na soma: a lógica duplicada da sessão 5, a fronteira violada na 12, a dependência circular na 20. Se nada no harness compara o código de hoje com as regras de arquitetura (um sensor que roda sempre, um review com docs do projeto), a degradação passa despercebida até ficar cara.

CAPÍTULO 14

O loop de code review

TL;DR

O review humano não escala, e o review por IA "de um prompt só" gera ruído até ninguém mais ler. A arquitetura da aula põe **6 agentes especialistas em paralelo**, cada um com os próprios docs e um *second pass* obrigatório, e **1 consolidador** que agrupa, deduplica e posta no PR. Em volta dela roda um loop com o humano no meio: `/review` → **o humano decide o que vale** → `/fix` → **o workflow faz o commit**. A IA nunca aprova, nunca rejeita e nunca comita. Sete pilares separam um review em que o time confia de um review que o time ignora. O principal é bater o PR contra os **requisitos**, não só contra o código.

Por que o review costuma falhar

A aula começa pelo que não funciona, dos dois lados.

Review humano	IA ingênua (um prompt só)
Gargalo de velocidade: PRs esperam dias na fila, e o desenvolvimento para junto.	**Alucinações:** inventa APIs e imports que não existem, cita regras que o projeto nunca definiu.
Fadiga: no 47º review do dia, a atenção já se foi e detalhes críticos passam.	**Falsos positivos (efeito "Cry Wolf"):** com 30% de ruído, as pessoas aprendem a ignorar tudo, e em poucos sprints a confiança chega a zero.
Inconsistência: o revisor A e o revisor B usam critérios completamente diferentes, e o padrão do projeto nunca é aplicado por igual.	**Sem contexto do projeto:** não conhece a arquitetura, os padrões nem as regras; o review sai genérico.
Cobertura parcial: cada um vê o que conhece; segurança, performance e requisitos ficam sem verificação consistente.	**Sem verificação de requisitos:** revisa o código, mas não a entrega. "O Jira dizia X, foi entregue Y, e ninguém viu."

Os 30% de ruído são um número ilustrativo do slide, não uma medição. A ideia é o que importa: um review que erra com frequência é pior do que nenhum, porque ensina o time a não ler.

A aula resume o problema numa imagem: o review por IA mais comum é um **"júnior que leu Clean Code"** e só comenta nome de variável. O objetivo é outro. É um review que bate requisitos, detecta alucinação da própria IA e sugere correções concretas.

A arquitetura: 6 especialistas e 1 consolidador

A solução aplica ao review a ideia do cap. 13, de separar papéis e dar a cada agente contexto limpo, levada um passo adiante: **um especialista por tema**.

Passo 1: Initialize

Antes de qualquer agente rodar, o sistema junta o material: o **diff do PR**, o título e a descrição, o **ID do ticket no Jira**, os **arquivos de spec** e os comentários já existentes.

Passo 2: seis agentes em paralelo

Agente	O que carrega ou procura	Second pass
Security	`docs/integration-patterns.md`	obrigatório
Requirements	API do Jira e arquivos de spec	obrigatório
E2E Tests	a skill `create-e2e-tests` (`SKILL.md`)	obrigatório
Architecture	7 docs do projeto, carregados antes de tudo	obrigatório
Regression & Hallucination	*phantom imports* (imports de coisas que não existem), código morto	obrigatório
Performance	queries N+1, `find()` sem limite, uso de `Promise.all`	obrigatório

Cada agente tem **escopo, docs e checklist próprios**, e um não distrai o outro. Na aula, os agentes de review rodam com **Sonnet**.

Passo 3: consolidação

O consolidador recebe os achados dos seis e:

- **agrupa por severidade**, em cinco níveis marcados por ícone (🔒, 🚨, ⚡, ⚠️, 💡);
- **deduplica**: o mesmo problema visto por dois agentes vira um comentário só;
- faz ***gap detection***: lista os arquivos que ninguém comentou, para deixar claro o que foi e o que não foi revisado;
- **posta** um resumo no PR e os comentários *inline* nas linhas certas.

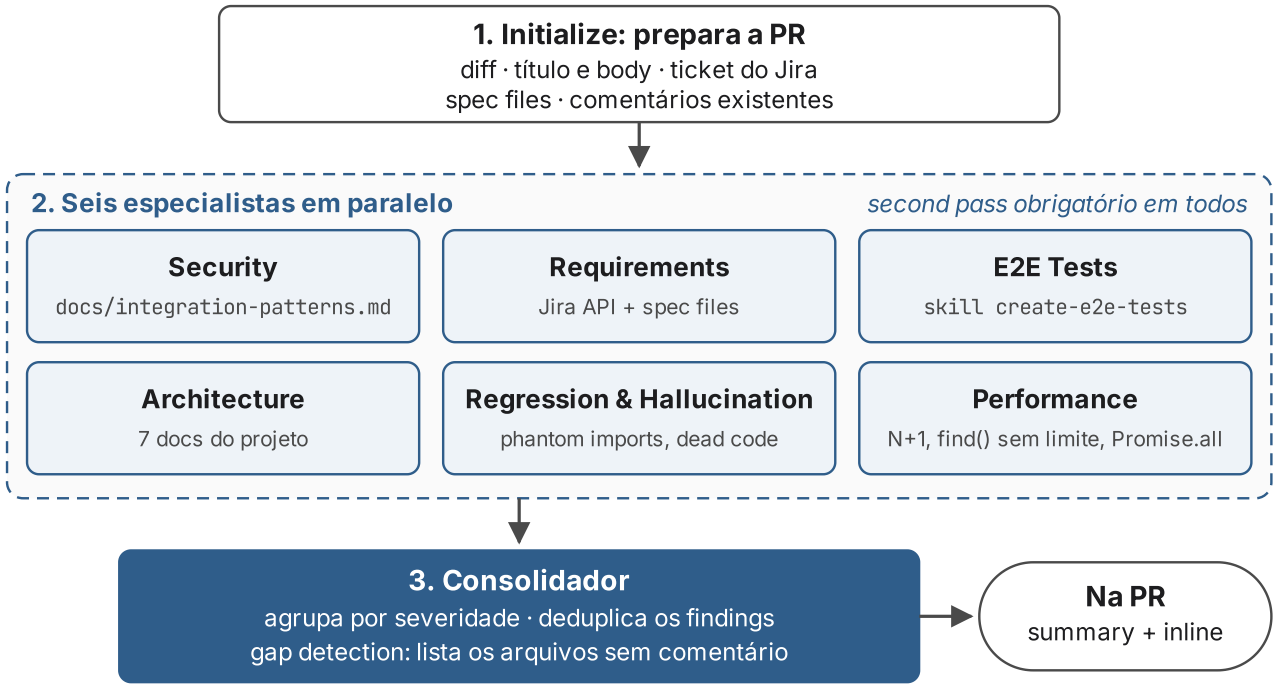


Figura 14.1 Review multi-agente: o PR é preparado, seis especialistas revisam em paralelo com second pass obrigatório e um consolidador agrupa, deduplica e posta no PR

O agente que revisa a IA

Entre os seis, a aula destaca o de **Regression & Hallucination** como "a cereja do bolo". A função dele é sinalizar **mudanças que não fazem sentido para a task**: o arquivo que ninguém pediu para mexer, o import de um módulo que não existe, a função que ficou sem uso. São os erros típicos de quem escreveu o código, uma IA, e que um revisor focado só em estilo ou só em segurança deixa passar.

Requisitos: o que quase nenhuma ferramenta faz

O agente de **Requirements** bate o PR contra o que foi pedido, a spec e o ticket do Jira, item a item. Segundo a aula, é o que **quase nenhuma ferramenta de mercado faz**. É também o que fecha a ponta solta da tabela inicial: "o Jira dizia X, foi entregue Y". Sem isso, o review avalia se o código está bem escrito, não se ele resolve o problema certo.

Checklists dinâmicas

Cada especialista é orquestrado por **skills com checklists**. Isso evita o comentário aleatório: o agente não revisa "o que lhe chamou a atenção", revisa uma lista definida pelo projeto, adaptada ao que o PR toca.

O loop: review, decisão humana e auto-fix

A arquitetura acima é um passo. O que a torna útil no dia a dia é o loop em volta dela, desenhado sobre o fluxo do GitHub:

1. Quem abriu o PR comenta `/review`.
2. O **GitHub Actions** dispara o workflow (reação 👀 no comentário, para indicar que recebeu).

3. Os **6 sub-agents** revisam em paralelo.
4. Os comentários inline e o resumo são **postados no PR**.
5. **O humano decide** comentário por comentário: 👍 (concordo, corrija) ou ignora (não é relevante).
6. Quem abriu o PR comenta `/fix`.
7. O agente de correção **filtra** o que vai tratar (os achados 🚨 e 🔒, mais os que receberam 👍), aplica **correções mínimas** e marca cada item como **[RESOLVED]**.
8. **O workflow faz `git commit` e `git push`. A IA, não.** Reação 🚀 quando termina.
9. O próximo ciclo começa com um novo `/review`.

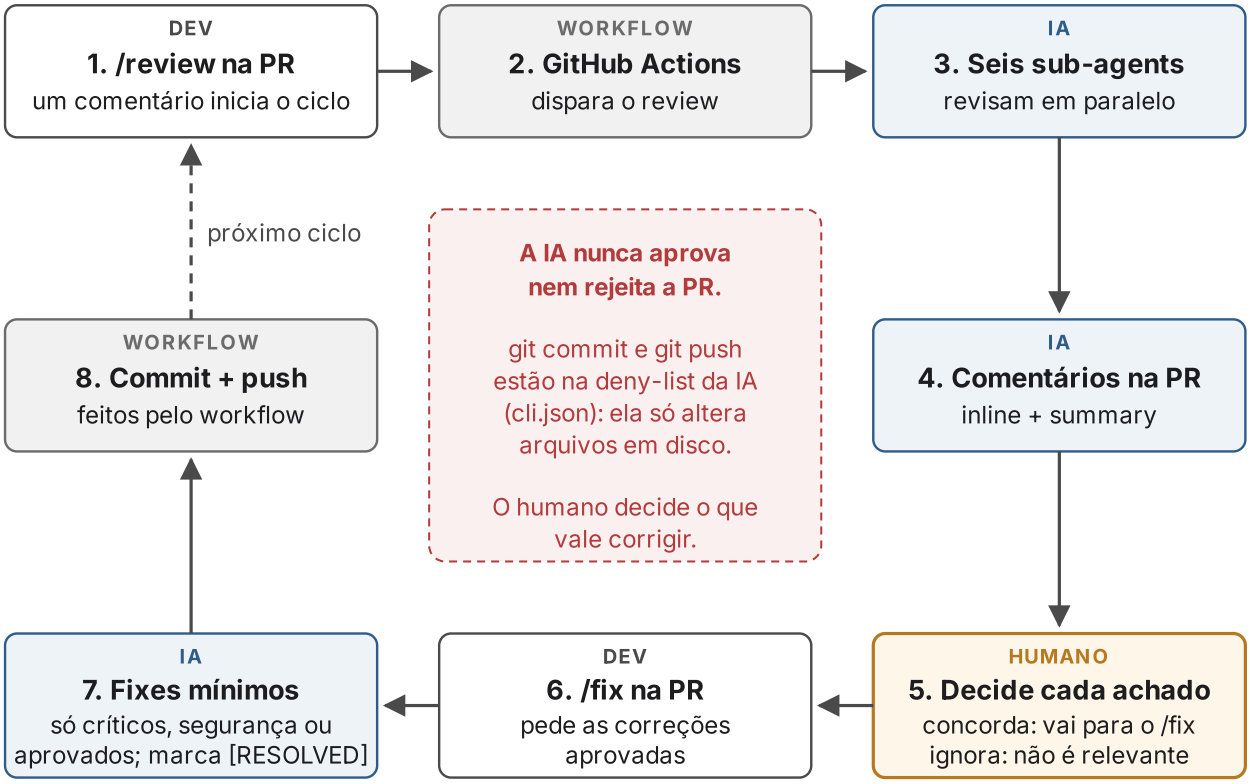


Figura 14.2 O loop de review: /review dispara os especialistas, o humano decide o que vale, /fix aplica correções mínimas e o workflow, não a IA, faz o commit

O detalhe de segurança do passo 8 é deliberado. **A IA não pode rodar `git commit` nem `git push`**: os dois comandos estão numa *deny-list* (lista de comandos bloqueados) na configuração do agente (`cli.json`). A IA só modifica arquivos em disco. Quem grava no repositório é o workflow, que é código determinístico e auditável.

O subtítulo do slide diz o mesmo: **a IA nunca aprova nem rejeita o PR. O humano sempre decide o que vale corrigir.**

Na demonstração da aula, esse loop revisou automaticamente um **PR de 74 arquivos**, com correções via `/fix` e integração completa com o GitHub.

Os 7 pilares do review confiável

O que separa um review em que os desenvolvedores confiam de um que eles ignoram é, no fim, **ter menos falsos positivos**. A aula organiza o caminho em sete pilares.

> **1. Especialização**
> Seis agentes focados, não um prompt genérico. Cada um com escopo, docs e checklist próprios. Um agente não distrai o outro.

> **2. Grounding**
> Antes de tocar no diff, carrega 7 ou mais docs do projeto. Nunca depende do "conhecimento geral" do modelo. As regras são as **suas**, não inventadas.

> **3. Second pass**
> Relê o diff inteiro. Lista explicitamente o que **não** comentou e justifica por que aquilo está limpo. Combate a "preguiça" do modelo.

> **4. Precision > Recall**
> Limiar de **80% de confiança**: se está incerto, não reporta. É melhor deixar de reportar do que reportar errado. Quem grita "lobo" à toa perde a confiança de todos.

> **5. Human-in-the-loop**
> Nunca aprova nem rejeita PRs. O humano decide via 👍 e `/fix`. A IA sugere, o humano autoriza. Sempre.

> **6. Rastreabilidade**
> Jira e arquivos de spec conferidos contra o diff, com checklist item a item (atendido, não atendido, não marcado) e second pass. Revisa a **entrega**, não só o código.

> **7. Meta-review**
> Um agente dedicado a detectar alucinações da própria IA: phantom imports, código morto, assinaturas erradas. A IA revisando a IA.

A fundação dos sete é uma só: **skills versionadas no repositório como fonte de verdade**. Os docs do seu projeto são a lei.

O texto da aula lista seis pilares. O slide acrescenta o sétimo, o meta-review, que corresponde ao agente de Regression & Hallucination. Este livro segue o slide.

Automatizar o merge?

A pergunta seguinte é natural: se o review e o fix são automáticos, por que não o merge também? A aula discute **quando vale e quando não vale** e mostra um caso da comunidade, um bot que

marca o PR com a tag **"precisa de humano"** de acordo com a severidade dos achados. PRs sem achados graves seguem; os demais esperam uma pessoa. As notas não detalham os critérios, então fica a ideia: **a automação do merge é uma decisão por severidade, não um tudo ou nada.**

Comparando ferramentas

A aula compara a abordagem "faça você mesmo" (o pipeline do projeto Fakeflix, chamado aqui de TLC DIY) com quatro ferramentas de mercado. Os valores são da época da aula.

Critério	TLC DIY	OpenReview (Vercel)	CodeRabbit	Copilot Code Review	Cursor BugBot
FLEXIBILIDADE					
Controle do prompt	Sim: 100%, você escreve	Parcial: ~80% (OSS)	Parcial (YAML)	Não: limite de 4 mil caracteres	Não: só acrescentar
Skills e regras próprias	Sim: ilimitado (`SKILL.md`)	Sim: `.agents/skills/`	Parcial: YAML + checks	Parcial: arquivo de instruções	Não: só `BUGBOT.md`
Bloquear o pipeline	Sim: flag `BLOCKING_REVIEW`	Não	Sim: custom checks	Não	Não
AGENTES E SKILLS					
Arquitetura	Sim: multi-agente (6+1)	Parcial: um agente + skills	Parcial: multi-agente fixo	Não: um agente	Não: um agente
Second pass	Sim: obrigatório	Não	Não	Não	Não
Detecção de alucinação	Sim: agente dedicado	Não	Não	Não	Não
RASTREABILIDADE DE REQUISITOS					
Integração com Jira	Sim: API direta	Não nativa	Sim: marketplace	Não no review	Não
Arquivos de spec	Sim: Jira + `.specs/`	Não	Não	Não	Não
EXECUÇÃO E CUSTO					
Auto-fix	Sim: `/fix` + aprovação humana	Sim: commit direto	Parcial: sugestões para aplicar	Não	Sim: autofix (VM)
Open source	Sim (os arquivos são seus)	Sim (MIT)	Não	Não	Não
Custo mensal	API, por uso	Vercel + Anthropic	US$ 24–30 por usuário	incluso no Copilot	US$ 40 por usuário

As ferramentas formam um eixo, do **mais flexível** (TLC DIY, OpenReview) ao **mais simples** (CodeRabbit, Copilot, BugBot). A pergunta do slide define a escolha: **você quer definir o que é um bom review, ou quer que alguém defina por você?**

A aula também comenta o **Claude Code Review**, que não está na tabela do slide. O ponto é o custo: na época, **cerca de US$ 25 por review**, o que muitas vezes não compensa diante de um agente customizado rodando no próprio CI, com os docs e as regras do projeto.

FAÇA

+ Divida o review por especialidade, cada agente com docs, checklist e escopo próprios.
+ Carregue os docs do projeto antes do diff. Grounding vem primeiro.
+ Exija second pass: o agente lista o que não comentou e por quê.
+ Prefira precisão a cobertura: com dúvida, o agente não reporta.
+ Confira o PR contra a spec e o ticket, item a item.
+ Mantenha o humano como quem decide e o workflow como quem comita.

EVITE

– Um prompt único de “revise este PR”.
– Deixar a IA aprovar, rejeitar ou comitar.
– Comentários sem base nas regras do projeto: são a origem do ruído.
– Medir o review pela quantidade de comentários.

ARMADILHA

O efeito “Cry Wolf”. Um review que comenta muito parece cuidadoso. Mas cada falso positivo ensina o time a pular os comentários, e em poucos sprints os bugs reais que o review encontra passam a ser ignorados junto com o ruído. O limiar de confiança e o second pass não são refinamentos: são o que mantém o review sendo lido.

CAPÍTULO 15

Harness determinístico e Loop Engineering

TL;DR

Para comparar com justiça vários runs de agentes (frameworks diferentes, modelos diferentes, a mesma tarefa), a TLC montou um **harness de avaliação com LLM as judge**. O juiz é um modelo, e portanto não é determinístico. A estratégia é tirar dele toda a liberdade que não precisa ter: **critérios de aceite viram checks binários**, num **baseline congelado** igual para todos os runs. O juiz roda num **sub-agent com modelo fixo e contexto isolado** e tem que **provar cada check com arquivo e linha**. Os **pesos são somados por um script**, não pelo modelo. Ninguém avalia o próprio trabalho, e **cada configuração roda mais de uma vez**. O capítulo termina com a porta para o próximo nível, **Loop Engineering**.

NOTA

Sobre este capítulo. O material da aula *Caso Real: Multi Agents, Loops e Harness Determinístico* traz a proposta e a lista de tópicos, sem desenvolvê-los. Cada tópico aparece aqui como um item curto, ligado ao que os capítulos anteriores já mostraram, sem detalhes de implementação que a aula não registra. O aprofundamento vem das fontes externas, sempre nos blocos **Complemento**, separados do conteúdo do curso.

O problema: julgar runs de agentes com justiça

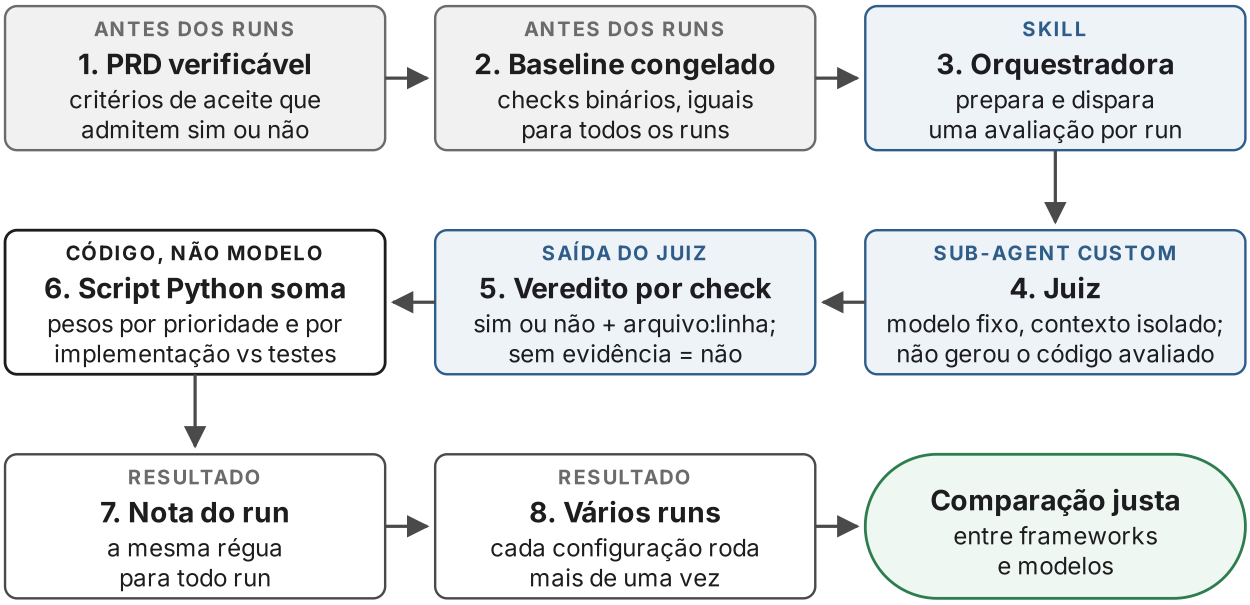
A técnica deste capítulo é a que está por trás do **benchmark de frameworks Spec-Driven da TLC**. Ele compara, na mesma tarefa, o resultado de fluxos diferentes (como os do cap. 9) e de modelos diferentes. Para isso, é preciso dar uma nota a cada run, e a nota precisa ser comparável entre runs.

Uma pessoa revisando dezenas de runs à mão não escala e não é consistente: é o problema do review humano do cap. 14. Um modelo julgando sozinho, "dê uma nota de 0 a 10 para esta implementação", é o problema da IA ingênua do mesmo capítulo. O harness de avaliação fica entre os dois. **O modelo julga, mas dentro de uma estrutura que torna o julgamento verificável e repetível.** É o Evaluator do cap. 13 levado ao extremo.

A proposta da aula cabe em três decisões:

- transformar **critérios de aceite em checks binários**;
- **congelar um baseline** justo para todos os runs;

- usar **sub-agents custom** para **fixar o modelo** do juiz e **isolar o contexto.**



Cada etapa tira uma liberdade do juiz: o que avaliar, como responder, como justificar, como somar e quantas vezes medir.

Figura 15.1 O harness de avaliação: PRD com critérios verificáveis, baseline congelado de checks binários, juiz em sub-agent com modelo fixo, evidência por arquivo e linha, soma por script e vários runs

Os itens a seguir seguem a ordem dos tópicos da aula.

1. Um modelo não determinístico como juiz

O ponto de partida é um paradoxo. O mesmo tipo de modelo que alucina, declara vitória cedo e enfraquece asserções (caps. 12 e 13) vai decidir se o trabalho de outro modelo está bom. A resposta da aula não é confiar no juiz. É **reduzir o espaço em que ele pode errar**. Cada tópico seguinte tira uma liberdade do juiz: o que avaliar (checks congelados), como responder (sim ou não), como justificar (arquivo e linha), como somar (script) e quantas vezes medir (vários runs).

COMPLEMENTO · fonte externa

Lianmin Zheng et al. — Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena (09/06/2023; NeurIPS 2023)

O artigo de referência sobre LLM as judge mediu o quanto um modelo forte concorda com pessoas ao julgar **respostas de assistentes de chat**, em 2023 (GPT-4, GPT-3.5, Claude-v1). Não avaliou código nem agentes. Resultado: "*strong LLM judges like GPT-4 can match both controlled and crowdsourced human preferences well, achieving over 80% agreement, the same level of agreement between humans*". Mais de 80% de concordância, o mesmo nível de concordância entre duas pessoas.

Os autores também catalogam os vieses do juiz: de **posição** (preferir a resposta que aparece primeiro, ou segundo), de **verbosidade** (preferir a resposta mais longa, mesmo pior) e de ***self-enhancement***

(autopreferência, preferir respostas geradas por si mesmo). E apontam um limite de raciocínio: o juiz pode errar até um cálculo que saberia fazer se perguntado separadamente.

https://arxiv.org/abs/2306.05685

COMPLEMENTO · fonte externa

Jeremy Hadfield et al. (Anthropic) — How we built our multi-agent research system (13/06/2025)

No sistema de pesquisa da Anthropic, o juiz avalia cada resultado contra uma **rubrica** com critérios nomeados: precisão factual, precisão das citações, completude, qualidade das fontes e eficiência no uso de tools. Testaram vários juízes, um por componente, mas *"a single LLM call with a single prompt outputting scores from 0.0-1.0 and a pass-fail grade was the most consistent and aligned with human judgements"*: uma única chamada, com nota de 0,0 a 1,0 e um veredito de passa ou falha, foi o mais consistente.

Duas lições operacionais do mesmo texto: começar a avaliar cedo, com amostras pequenas (eles começaram com cerca de 20 queries), e lembrar que *"human evaluation catches what automation misses"*.

https://www.anthropic.com/engineering/multi-agent-research-system

2. PRD com critérios de aceite verificáveis

O juiz só consegue ser objetivo se o pedido foi objetivo. Por isso a avaliação começa **antes da implementação**, no PRD. A anotação da aula é curta e diz o essencial: **PRD bem detalhado, com o eval.** O documento de requisitos já é escrito pensando em como cada critério será verificado.

O cap. 9 já mostrou o formato que ajuda nisso. A fase Specify do TLC Spec-Driven escreve os requisitos como user stories priorizadas (P1, P2, P3), em **WHEN / THEN / SHALL**, com **IDs de rastreabilidade**. Um critério verificável é aquele que admite resposta sim ou não olhando o código e os testes. "O checkout deve ser rápido" não é verificável. "WHEN o pagamento é recusado, THEN o sistema SHALL exibir a mensagem de erro e manter o carrinho" é.

COMPLEMENTO · fonte externa

Mikaela Grace, Jeremy Hadfield, Rodrigo Olivares e Jiri De Jonghe (Anthropic) — Demystifying evals for AI agents (09/01/2026)

O guia de avaliação de agentes da Anthropic dá um teste simples para saber se um critério está bem escrito: *"A good task is one where two domain experts would independently reach the same pass/fail verdict."* Uma boa tarefa é aquela em que dois especialistas, cada um por conta própria, chegariam ao mesmo veredito de passa ou falha. Se duas pessoas divergiriam, o juiz também vai oscilar.

O texto separa também o que o agente **diz** do que ele **fez**: o *outcome* é o estado final do ambiente, não a mensagem do agente. No exemplo deles, um agente de reservas pode dizer "seu voo foi reservado", mas o resultado é se a reserva existe no banco de dados. É a Vitória Prematura do cap. 13, vista do lado de quem avalia.

https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents

3. Baseline congelado: de frases para checks binários

Um PRD, por mais detalhado, ainda é texto. O passo seguinte é traduzir cada critério em **checks binários**: perguntas com resposta sim ou não, que o juiz responde uma de cada vez. Esse conjunto de checks é o **baseline**.

E o baseline é **congelado**. Fica igual para todos os runs, sem ajuste no meio do benchmark para acomodar o resultado de um framework ou de outro. Só assim a comparação é justa: todo run responde exatamente às mesmas perguntas. É a mesma ideia dos testes congelados na fase red do cap. 12 (princípio 6), aplicada à avaliação: **a prova não muda depois que o trabalho começou.**

COMPLEMENTO · fonte externa
Hamel Husain — Using LLM-as-a-Judge For Evaluation: A Complete Guide (29/10/2024, atualizado em 01/09/2026)

Husain defende o binário sem meio-termo, pelo menos no começo: *"If your evaluations consist of a bunch of metrics that LLMs score on a 1-5 scale (or any other scale), you're doing it wrong."* O problema das escalas é que ninguém sabe dizer o que separa um 3 de um 4. A decisão binária obriga quem define o critério a explicitar o que espera.

Ele acrescenta uma peça que o binário sozinho não tem: junto com cada veredito, uma **crítica escrita**, detalhada o bastante para *"a new employee could understand it"*. No prompt do juiz, a ordem é primeiro a crítica, depois o veredito.

https://hamel.dev/blog/posts/llm-judge/

COMPLEMENTO · fonte externa
Mikaela Grace et al. (Anthropic) — Demystifying evals for AI agents (09/01/2026)

"Congelado" também vale para o **ambiente**. O guia recomenda que cada tentativa comece de um estado limpo: *"Each trial should be "isolated" by starting from a clean environment."* E dá um exemplo do que acontece quando não começa: em avaliações internas, o Claude ganhou vantagem indevida em algumas tarefas *"by examining the git history from previous trials"*, lendo o histórico do git de tentativas anteriores. Daí a regra: os graders devem ser resistentes a atalhos, e o agente não pode conseguir "colar" na avaliação.

https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents

4. Skill orquestradora vs skill avaliadora

O harness separa dois papéis em skills diferentes. A **orquestradora** conduz o processo: prepara o que cada avaliação precisa, dispara as avaliações e junta os resultados. A **avaliadora** faz uma coisa só: julgar um run contra o baseline.

É a separação do cap. 13 (Generator e Evaluator em processos distintos) e do cap. 14 (especialistas e consolidador) aplicada à avaliação. Quem conduz não julga; quem julga não conduz. E nenhum dos dois é o agente que produziu o código avaliado.

COMPLEMENTO · fonte externa

Prithvi Rajasekaran (Anthropic) — Harness design for long-running application development (24/03/2026)

O autor da arquitetura Planner → Generator → Evaluator explica por que separar quem faz de quem julga: *"When asked to evaluate work they've produced, agents tend to respond by confidently praising the work—even when, to a human observer, the quality is obviously mediocre."* Agentes que avaliam o próprio trabalho tendem a elogiá-lo com confiança, mesmo quando a qualidade é medíocre.

A separação, no entanto, não basta sozinha. *"Out of the box, Claude is a poor QA agent"*: nas primeiras versões, o evaluator achava problemas reais e depois se convencia de que não eram graves. A conclusão é que dá trabalho, mas é viável: *"tuning a standalone evaluator to be skeptical turns out to be far more tractable than making a generator critical of its own work"*. Afinar um avaliador separado para ser cético é muito mais tratável do que tornar o gerador crítico do próprio trabalho. O ajuste é um loop: ler os logs do avaliador, achar onde o julgamento dele divergiu do de uma pessoa e corrigir o prompt.

https://www.anthropic.com/engineering/harness-design-long-running-apps

COMPLEMENTO · fonte externa

Walden Yan (Cognition) — Multi-Agents: What's Actually Working (22/04/2026)

Dez meses depois de *Don't Build Multi-Agents* (cap. 11), o mesmo autor revisa a posição. Os enxames de agentes que escrevem em paralelo continuam não funcionando. Mas funciona uma classe mais estreita de arranjos: *"multi-agent systems work best today when writes stay single-threaded and the additional agents contribute intelligence rather than actions"*. Vários agentes contribuem com inteligência, e só um escreve.

O primeiro padrão que ele descreve é o loop de review com um revisor de **contexto limpo**. A técnica funciona melhor *"when the coding and review agents do not share any context beforehand"*: o revisor olha só o diff e redescobre o contexto lendo o código do zero. É o mesmo princípio de "quem testa nunca viu o código sendo escrito" do cap. 13.

https://cognition.com/blog/multi-agents-working

5. Quando usar sub-agents custom em vez de skills

A proposta da aula diz que **sub-agents custom** foram a escolha certa para o harness, por dois motivos: **fixar ("pinar") o modelo** e **isolar o contexto**.

Os dois motivos saem do que as Partes II e IV já mostraram. Uma skill (cap. 5) é carregada no contexto do agente que a aciona e roda com o modelo que esse agente estiver usando. Um sub-agent (cap. 11) tem janela própria e, quando é custom, pode ter um modelo definido. Num benchmark, as duas propriedades importam:

- **Modelo fixo:** todos os runs são julgados pelo mesmo modelo, qualquer que seja o modelo da sessão que disparou a avaliação. Sem isso, a nota mede o juiz, não o run.
- **Contexto isolado:** o juiz não vê o histórico do run que avalia, nem os resultados das avaliações anteriores. Cada julgamento começa limpo.

COMPLEMENTO · fonte externa
Addy Osmani — Loop Engineering (07/06/2026)

Osmani lista os sub-agents entre os componentes de um loop bem montado, com uma função precisa: *"Sub-agents so one of them has the idea and a different one checks it."* Um tem a ideia e outro a verifica. E põe o custo na conta: *"Subagents do burn more tokens since each one does its own model and tool work, so spend them where a second opinion is worth paying for."* Sub-agents gastam mais tokens; use-os onde uma segunda opinião vale o preço. A avaliação é um desses lugares.

https://addyosmani.com/blog/loop-engineering/

6. Evidência obrigatória: provar com arquivo e linha

Um "sim" do juiz não vale nada sozinho. Para cada check que marca como atendido, o juiz precisa **apontar o arquivo e a linha** onde a evidência está. Isso muda a natureza do julgamento: em vez de uma opinião, o juiz faz uma **afirmação verificável**. Uma pessoa consegue conferir qualquer check em segundos, e um check sem evidência é tratado como não atendido.

É o grounding e o second pass do review (cap. 14) e a verificação visível do cap. 12 (princípio 4) aplicados ao juiz. Evidência obrigatória também combate a "preguiça" do modelo: é mais difícil marcar "sim" por impressão quando é preciso mostrar onde.

COMPLEMENTO · fonte externa
Prithvi Rajasekaran (Anthropic) — Harness design for long-running application development (24/03/2026)

No harness do autor, cada critério do contrato de sprint recebe um achado concreto do evaluator, com localização. Um exemplo do próprio artigo, para o critério *"User can select and delete placed entity spawn points"*: *"Delete key handler at* `LevelEditor.tsx:892` *requires both* `selection` *and* `selectedEntityId` *to be set, but clicking an entity only sets* `selectedEntityId`." O achado diz o arquivo, a linha, a condição errada e por quê. Quem lê não precisa refazer a investigação.

https://www.anthropic.com/engineering/harness-design-long-running-apps

7. Pesos por prioridade e por implementação vs testes

Nem todo check vale o mesmo. O harness atribui **pesos** em dois eixos:

- **por prioridade:** critérios mais importantes pesam mais. A spec do cap. 9 já traz essa priorização nas user stories (P1, P2, P3);
- **por implementação vs testes:** checks de código e checks de testes recebem pesos próprios. Assim, a nota mostra o equilíbrio entre os dois, e não só a quantidade de código entregue.

As notas da aula não registram os valores usados.

Os pesos não reintroduzem a escala de 1 a 5. **Cada check continua sendo binário** (0 ou 1); o peso só diz quanto aquele 1 vale na soma.

COMPLEMENTO · fonte externa

Mikaela Grace et al. (Anthropic) — Demystifying evals for AI agents (09/01/2026)

O guia trata a combinação de vários graders como escolha de projeto: "*scoring can be weighted (combined grader scores must hit a threshold), binary (all graders must pass), or a hybrid.*" A nota pode ser ponderada (a soma precisa atingir um limiar), binária (tudo precisa passar) ou híbrida. Checks binários com pesos são uma forma de ponderação. Combinam com o conselho de Husain (Complemento do item 3), que desaconselha escalas, não pesos.

https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents

8. Script Python na soma, para evitar erro de aritmética

Depois que o juiz responde cada check, sobra uma conta: multiplicar cada resultado pelo peso e somar. **Essa conta não fica com o modelo.** Um script Python lê os vereditos e faz a soma.

É a regra do cap. 12 em outra forma: o que pode ser determinístico deve ser determinístico. O modelo é necessário para a parte que exige interpretação (este trecho de código atende este critério?). A aritmética não exige interpretação, e um modelo pode errar uma soma de vinte parcelas sem perceber.

COMPLEMENTO · fonte externa

Lianmin Zheng et al. — Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena (09/06/2023)

O estudo documenta o juiz errando em perguntas de matemática e raciocínio mesmo quando sabia a resposta: "*although GPT-4 can solve the problem (when asked separately), it was misled by the provided answers, ultimately resulting in incorrect judgment.*" O GPT-4 resolvia o problema quando perguntado separadamente, mas, ao julgar respostas, era influenciado por elas e errava o veredito. Tirar a conta do juiz elimina essa fonte de erro por completo.

https://arxiv.org/abs/2306.05685

9. Princípios do harness: sem autopreferência, múltiplos runs

Dois princípios fecham o desenho.

Sem autopreferência. O juiz não deve avaliar trabalho produzido por ele mesmo. É a raposa e o galinheiro do cap. 13 aplicados ao benchmark: um juiz que tende a gostar do próprio estilo distorce a comparação a favor de um dos runs.

Múltiplos runs. Um run de um agente é uma amostra, não um resultado. O mesmo framework, com o mesmo modelo e a mesma tarefa, pode ir bem numa execução e mal na seguinte. Uma comparação justa roda cada configuração **mais de uma vez** e olha para o conjunto.

COMPLEMENTO · fonte externa

Mikaela Grace et al. (Anthropic) — Demystifying evals for AI agents (09/01/2026)

O guia dá nome a cada tentativa, *trial*, e duas métricas para resumi-las. *pass@k* é a chance de acertar **pelo menos uma vez** em k tentativas. *pass^k* é a chance de acertar **todas as k**. Elas contam histórias opostas à medida que k cresce: *"If your agent has a 75% per-trial success rate and you run 3 trials, the probability of passing all three is $(0.75)^3 \approx 42\%$."* Um agente que acerta 75% das vezes passa nas três tentativas só em cerca de 42% dos casos. A escolha depende do uso: *"pass@k for tools where one success matters, pass^k for agents where consistency is essential."*

E um lembrete: *"we do not take eval scores at face value until someone digs into the details of the eval and reads some transcripts."* Ninguém deve confiar numa nota antes de ler alguns dos registros completos das execuções.

https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents

COMPLEMENTO · fonte externa

Lianmin Zheng et al. — Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena (09/06/2023)

A autopreferência tem nome no artigo, mas **não foi confirmada**. Os autores viram indícios (*"GPT-4 favors itself with a 10% higher win rate; Claude-v1 favors itself with a 25% higher win rate"*), mas também que os modelos favoreciam outros modelos e que o GPT-3.5 não se favorecia. E concluem: *"Due to limited data and small differences, our study cannot determine whether the models exhibit a self-enhancement bias."* A regra "sem autopreferência" é, portanto, uma **precaução barata**, não a correção de um efeito comprovado.

Para o viés de posição, o artigo propõe uma mitigação simples: chamar o juiz duas vezes, trocando a ordem das respostas, e só declarar vencedor quem ganhar nas duas ordens.

https://arxiv.org/abs/2306.05685

COMPLEMENTO · fonte externa

Walden Yan (Cognition) — Multi-Agents: What's Actually Working (22/04/2026)

O contraponto vem de quem usa revisores em produção. Para a Cognition, o mesmo modelo como autor e revisor não se enviesa como uma pessoa se enviesaria, **desde que o contexto seja limpo**: *"putting the same model in two agents [...] does not quite make them self-biased/correlated in the same way you might imagine one human doing both tasks would be. These agents are ultimately systems that perform based on their context."*

As duas posições são compatíveis com o harness do curso. Zheng et al. falam de um juiz comparando respostas; a Cognition, de um revisor de código com contexto isolado. Separar o juiz e isolar o contexto atende às duas.

https://cognition.com/blog/multi-agents-working

De volta aos harnesses longos

As peças deste capítulo (critérios verificáveis, checks binários, avaliador separado, estado em arquivo) são as mesmas do harness do cap. 13, com outro objetivo. Lá, o objetivo é construir.

Aqui, é medir. As duas fontes da Anthropic que inspiraram aquela aula mostram como esses componentes se encaixam numa implementação longa.

COMPLEMENTO · fonte externa

Justin Young (Anthropic) — Effective harnesses for long-running agents (26/11/2025)

Young descreve o problema central dos agentes longos com uma imagem: um projeto tocado por engenheiros em turnos, em que cada um chega sem lembrar nada do turno anterior. É a Amnésia entre Sessões do cap. 13. A solução tem dois agentes, que diferem só no prompt inicial:

- um **initializer agent**, que na primeira sessão prepara o ambiente: um `init.sh`, um `claude-progress.txt` com o log do que foi feito e um commit inicial;
- um **coding agent**, que em toda sessão faz progresso incremental, **uma feature por vez**, e deixa artefatos claros para a próxima.

O coração do arranjo é uma **feature list em JSON** (mais de 200 features no exemplo de um clone do claude.ai), com tudo começando como *failing*. O agente só pode mudar o campo de status. O JSON foi escolhido porque o modelo altera ou sobrescreve um JSON indevidamente com menos frequência do que um Markdown. Isso ataca de frente o One-Shot Hero e a Vitória Prematura.

O Teste Fake também aparece. O agente fazia testes de unidade ou `curl` contra o servidor e não percebia que a feature não funcionava de ponta a ponta. Melhorou muito quando foi instruído a testar *"as a human user would"*, com automação de browser (Puppeteer MCP no experimento).

https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents

COMPLEMENTO · fonte externa

Prithvi Rajasekaran (Anthropic) — Harness design for long-running application development (24/03/2026)

É o texto por trás do Planner → Generator → Evaluator do cap. 13. O planner expande um pedido de 1 a 4 frases numa spec de produto. O generator trabalha em sprints, uma feature por vez. O evaluator usa o Playwright MCP para clicar pela aplicação *"the way a user would"*. Antes de cada sprint, generator e evaluator negociam um ***sprint contract***, um acordo sobre o que significa "pronto" antes de qualquer código, trocado por arquivos.

O artigo compara as duas abordagens num único experimento: criar um *game maker* 2D retrô com Claude Opus 4.5. Agente solo: 20 minutos, US$ 9, e o jogo não respondia aos comandos. Harness completo: 6 horas, US$ 200, e o jogo funcionava. Mais de 20 vezes o custo, por uma diferença de qualidade *"immediately apparent"*. Os números valem para esse experimento, não como regra geral.

A lição mais duradoura é sobre manutenção: *"every component in a harness encodes an assumption about what the model can't do on its own"*. Cada componente do harness embute uma suposição sobre o que o modelo não faz sozinho, e essas suposições envelhecem à medida que os modelos melhoram. O autor passou a remover um componente de cada vez e medir o impacto.

https://www.anthropic.com/engineering/harness-design-long-running-apps

10. Introdução a Loop Engineering

A aula termina apresentando **Loop Engineering**, sem desenvolvê-lo. Vale olhar para o caminho do livro até aqui: o prompt (cap. 2), o contexto (caps. 2 e 3), o harness (cap. 13). Em todos esses níveis, quem dispara o agente ainda é uma pessoa. O próximo degrau, que os Complementos

abaixo descrevem, é tirar a pessoa dessa função e colocar no lugar dela um sistema, com tudo o que a Parte IV construiu: condições de parada objetivas, verificadores separados, estado em arquivo.

COMPLEMENTO · fonte externa

Addy Osmani — Loop Engineering (07/06/2026)

Osmani reúne a prática sob o nome Loop Engineering, a partir de frases de Peter Steinberger e Boris Cherny. A definição dele: *"Loop engineering is replacing yourself as the person who prompts the agent. You design the system that does it instead."* Você deixa de ser quem dá o prompt e passa a projetar o sistema que dá. Esse sistema encontra o trabalho, distribui, verifica, registra o que foi feito e decide o próximo passo. Ele fica *"one floor above the harness"*, um andar acima do harness.

Os componentes, segundo ele: automações agendadas, que fazem descoberta e triagem sozinhas; *worktrees*, para agentes em paralelo não pisarem uns nos outros; skills; plugins e conectores; sub-agents que separam quem faz de quem checa. E uma memória fora da conversa: *"The agent forgets, the repo doesnt."*

O ponto estrutural mais importante, para ele, é o mesmo deste capítulo: *"splitting the one who writes from the one who checks. The model that wrote the code is way too nice grading its own homework."* Com dois avisos. O custo em tokens exige cuidado. E *"A loop running unattended is also a loop making mistakes unattended"*: um loop rodando sem supervisão também erra sem supervisão.

https://addyosmani.com/blog/loop-engineering/

COMPLEMENTO · fonte externa

Geoffrey Huntley — Ralph Wiggum as a "software engineer" (14/07/2025)

O contraponto mais simples a tudo isto é o "Ralph": um `while` em bash que roda o agente sempre com o mesmo prompt.

```
while :; do cat PROMPT.md | claude-code ; done
```

Cada volta começa com a **janela zerada**. O estado vive no filesystem: um plano de correções em ordem de prioridade (`fix_plan.md`), as specs e notas deixadas pelas voltas anteriores. A regra de ouro do autor: *"One item per loop."* Uma coisa por volta, e se o processo sair dos trilhos, estreite ainda mais. O que impede o código ruim de entrar é a *back pressure*: testes, tipos, scanners de segurança, *"anything can be wired in as back pressure to reject invalid code generation"*.

Huntley não esconde a fragilidade: a técnica é *"deterministically bad in an undeterministic world"*, e cada vez que o Ralph erra, ele é afinado *"like a guitar"*. Vale como lembrete de que um loop útil não precisa começar sofisticado, mas precisa de verificação de verdade em cada volta.

https://ghuntley.com/ralph/

COMPLEMENTO · fonte externa

Jai Lal Lulla, Vahram Nersesyan, Seyedmoein Mohsenimofidi, Christoph Treude e Sebastian Baltes — Loop Engineering: Building Blocks, Adoption, and Impact (ago/2026; preprint em revisão)

Um primeiro estudo acadêmico sobre o tema, **exploratório**, descreve a escada que este livro percorreu: dos prompts ao contexto, do contexto ao harness, do harness ao loop. Nesse nível, os sistemas disparam

agentes por agenda ou por eventos do repositório e os param "*when a machine-checkable condition holds*", quando uma condição verificável por máquina é satisfeita.

A revisão da literatura cinzenta (posts, documentação, relatos) encontra os mesmos componentes que a Parte IV discutiu: condições de parada verificáveis, *state files* persistentes, sub-agents verificadores, orçamentos de tokens e pontos definidos de escalonamento para humanos. Na mineração de repositórios, os autores confirmaram loops autônomos em 217 dos 256 repositórios selecionados pelas heurísticas (de 36.710 analisados). Os repositórios commitam a configuração dos loops, mas quase nenhum commita os state files.

Os modos de falha catalogados servem de checklist: custos que disparam, loops de correção infinitos, verificadores que aprovam sem checar de verdade ("*verifier theater*"), fadiga de notificações e divergência entre as instruções commitadas e o estado do loop.

https://arxiv.org/abs/2608.21884

FAÇA

+ Escreva o PRD já pensando na avaliação: cada critério deve admitir resposta sim ou não.
+ Traduza os critérios em checks binários e congele o baseline antes do primeiro run.
+ Rode o juiz num sub-agent com modelo fixo e contexto isolado.
+ Exija arquivo e linha para cada check marcado como atendido.
+ Deixe a soma dos pesos para um script.
+ Rode cada configuração mais de uma vez antes de concluir qualquer coisa.
+ Leia alguns julgamentos por inteiro, não só a nota final.

EVITE

– Pedir ao modelo uma nota de 0 a 10 "para a implementação".
– Ajustar o baseline no meio da comparação.
– Deixar o juiz avaliar trabalho produzido por ele mesmo, ou ver o histórico do run.
– Aceitar "sim" sem evidência.
– Tirar conclusões de um run só.

ARMADILHA

O juiz que concorda com tudo. Um harness de avaliação mal montado não falha de forma barulhenta. Ele devolve notas altas, consistentes e plausíveis, e por isso ninguém desconfia. Checks vagos, juiz sem obrigação de evidência, um único run: cada uma dessas brechas empurra a nota para cima sem que nada pareça errado. É o mesmo "verde" enganoso do cap. 12, um nível acima. A defesa é a mesma: tornar cada "sim" verificável e conferir alguns à mão.

PARTE V

Na prática

16 Fluxo de ponta a ponta

17 Playbook: montando seu primeiro loop de agentes

18 Kit de Skills

V

CAPÍTULO 16

Fluxo de ponta a ponta

TL;DR

O fluxo começa com o problema do stakeholder e termina com uma mudança conferida contra esse problema. Entre os dois, o contexto muda de formato: conversa → Design Doc → tasks no Linear → uma spec por task → implementação por fases → verificação → validação de telas → review local. Cada passagem precisa preservar intenção, limites e critérios de aceite. O caso de Frances Thai, registrado na aula sobre o Cursor, mostra outra aplicação dessas peças: tasks pequenas, contexto distribuído entre issue, documento do projeto e skill de domínio, cloud agents e review humano.

O contexto atravessa o processo

Nas Partes II a IV, cada peça apareceu separadamente. O `AGENTS.md` guarda regras do projeto; skills carregam procedimentos; a spec transforma uma intenção em contrato; tasks delimitam execução; o harness exige evidência. No fluxo completo, essas peças precisam conversar. Um agente pode executar uma task perfeitamente e ainda entregar a coisa errada se a necessidade do stakeholder se perdeu antes de chegar à spec.

Um *stakeholder* é a pessoa interessada no resultado, que traz o problema e ajuda a esclarecer o que conta como sucesso. O research começa na conversa com ela. O objetivo é sair com problema, comportamento esperado e dúvidas registradas, em vez de passar uma transcrição bruta adiante esperando que o agente descubra sozinho o que importa. É a Discovery do cap. 7 fornecendo contexto para o desenvolvimento.

A nota **Conectando Tudo** enumera as etapas de uma demonstração de feature real, mas não registra seus requisitos, arquivos ou resultados. Este capítulo costura essa sequência com os caps. 7–15. Não reconstrói a feature nem atribui à aula exemplos que não foram registrados.

Do Design Doc à task do time

O **Design Doc** organiza a solução técnica para as pessoas. Explica o como, as decisões e os motivos, conforme o cap. 7. Um RFC pode guardar as alternativas ainda em discussão; um ADR, a decisão já tomada. Os três viram fontes de contexto para o agente, mas a documentação de uma decisão não substitui a especificação do comportamento a implementar.

Com essa base, o trabalho vira projeto e tasks no Linear via MCP, na sequência descrita pela nota. O MCP faz a ponte com a ferramenta; o planejamento precisa fornecer o conteúdo. Criar

tickets automaticamente só ajuda quando eles representam blocos de trabalho compreensíveis e delimitados.

Há duas necessidades aqui. A **task para humanos** permite ao time organizar prioridade, responsabilidade e andamento. A **spec para o modelo** torna o trabalho executável e verificável: comportamento, restrições, casos de borda e critérios de sucesso. Uma pode apontar para a outra sem duplicar toda a documentação.

Uma spec por task, na hora de implementar

A sequência da aula propõe criar **uma spec por task, só quando ela vai ser implementada**. A task já existe no planejamento do time; o detalhamento executável entra quando há contexto para fazê-lo. Isso evita tratar uma lista de tickets como se já fosse um plano de implementação completo.

Nesse recorte, a task do Linear pode abrir um pequeno ciclo Spec-Driven. Dentro dele, o `tasks.md` do cap. 9 decompõe a execução em unidades com o quê, onde e DoD. O ticket de gestão e a task atômica de implementação podem, portanto, ter tamanhos diferentes. O importante é manter a referência ao requisito entre essas camadas.

A spec nasce dos documentos existentes e do código atual. Ao implementar, o agente recebe a parte relevante desse material, junto das regras permanentes do projeto. O cap. 9 explica os arquivos do fluxo; o cap. 12 explica por que a DoD precisa incluir gates que realmente rodem.

Implementação por fases e verificação

Sub-agents por fase preservam espaço na janela principal quando a implementação é longa. O critério continua sendo o cap. 11: agrupar blocos coesos e respeitar dependências. Se uma fase depende da anterior, a delegação é sequencial. O ganho é contexto disponível para acompanhar e corrigir o trabalho.

Cada bloco percorre implementar → testar → corrigir até passar o gate da task. O resultado volta acompanhado de arquivos alterados, verificações realizadas e impedimentos. Decisões e estado de retomada ficam fora da conversa, no `STATE.md`, como nos caps. 9 e 13.

Depois vem o *verifier*, o papel que confere o trabalho contra a spec. Ele precisa olhar os critérios e a evidência da entrega. A explicação do implementador ajuda a localizar o que mudou, mas não é prova de que o requisito foi atendido. O cap. 15 desenvolve essa separação entre produzir e julgar, com checks binários e evidência por arquivo e linha.

Telas e review local

A validação de frontend com o MCP do Playwright aparece como etapa própria na nota. Faz sentido dentro do princípio do cap. 12: um teste precisa observar o comportamento real que pretende provar. Para um requisito de interface, ler o componente não substitui percorrer a interação no navegador e comparar o resultado com a spec.

A nota não fornece um roteiro de telas, então o roteiro deve sair dos critérios da própria feature. O registro precisa dizer o que foi exercitado e o que ficou sem validação. Uma tela que abriu não prova todas as suas interações.

O fluxo fecha com **PR review em local mode antes de subir**. É uma revisão do diff local à luz de requisitos, regras e arquitetura, com correção dos achados pertinentes. O cap. 14 fornece os critérios do review confiável e o papel da decisão humana. O capítulo não presume que a revisão local publique comentários ou execute o workflow de GitHub Actions descrito ali: são momentos diferentes de aplicar os critérios de revisão.

Caso do Cursor: o relato de Frances Thai

A nota da conversa com Frances Thai descreve features que chegam pelo planejamento do produto, por pedido de cliente ou por relato de erro. O nível de planejamento muda conforme a origem e o tamanho do trabalho. No exemplo de um projeto de três meses, o trabalho é organizado em **P0, P1 e P2**, depois em tasks no Linear pequenas o suficiente para um agente concluir sozinho sem interferir no trabalho de outro.

O contexto fica em três lugares: **a issue**, **um documento guarda-chuva do projeto** e **uma skill de domínio mantida para o projeto inteiro**. A issue delimita a entrega; o documento situa o projeto; a skill reúne conhecimento recorrente. É uma aplicação da separação entre contexto da tarefa e contexto reutilizável dos caps. 4 e 5.

No fluxo relatado, mover a task para **in progress** no Linear dispara um *cloud agent*, um agente que executa no ambiente remoto. A automação do disparo não elimina o gargalo humano: a própria Frances ainda se identifica como gargalo de seu fluxo. A nota também registra que o code review humano continua importante, mesmo com Bugbot e Security bot atuando sobre PRs grandes.

Ela atribui o funcionamento dos cloud agents a um harness bom, ao *monorepo* (um repositório que reúne os projetos) e ao MCP no ambiente remoto apontando para o banco de produção para pesquisa antes de codar. **Isso é uma descrição da prática relatada; a nota não registra as permissões desse acesso.** Como orientação editorial para adaptar o exemplo, acesso de pesquisa deve ser somente leitura e limitado ao necessário. Essa orientação não é uma afirmação sobre a configuração usada por Frances.

A conversa ainda menciona a passagem para Cursor 3 (Glass) em tempo integral e o uso de Canvas para diagramação, investigação de incidentes e comunicação técnica. A sinopse não detalha essas ferramentas nem traz medições de produtividade. A lição sustentada pelo registro é a combinação de tasks delimitadas, contexto acessível e revisão humana. O Apêndice C aponta para o perfil de Frances; a nota não fornece URL do vídeo.

Lendo o percurso completo

Passagem	O que precisa chegar à próxima etapa	Onde retomar
Conversa → Design Doc	problema, limites, decisões e dúvidas	cap. 7

Passagem	O que precisa chegar à próxima etapa	Onde retomar
Design Doc → tasks no Linear	blocos de entrega compreensíveis para o time	caps. 7–8
Task → spec de implementação	critérios verificáveis e fontes de contexto	cap. 9
Spec → execução por fases	escopo, dependências e DoD	caps. 11–12
Implementação → verifier	mudança e evidência contra a spec	caps. 13 e 15
Verificação → review local	diff, requisitos e resultados de validação	cap. 14

FAÇA

+ Preserve a ligação entre problema, task, spec e evidência de validação.
+ Use Design Doc, RFC e ADR como fontes para a spec da task que entra em implementação.
+ Agrupe a delegação por fases coesas e respeite as dependências.
+ Confira comportamento de interface no navegador e requisitos no review local.

EVITE

– Mandar um ticket de gestão pouco detalhado direto para implementação.
– Confundir verificação da spec com aprovação da explicação do implementador.
– Copiar uma automação de cloud agents sem entender onde cada agente encontra contexto.
– Atribuir ao caso do Cursor permissões, resultados ou detalhes ausentes da nota.

ARMADILHA

Automatizar as passagens e perder a intenção. Tickets podem ser criados, agentes disparados e testes executados sem que ninguém confira se a entrega resolve o problema inicial. A rastreabilidade precisa atravessar o processo: o review compara a mudança com a spec, e a spec precisa continuar representando a necessidade que abriu o trabalho.

CAPÍTULO 17

Playbook: montando seu primeiro loop de agentes

TL;DR

Escolha uma mudança pequena, prepare as regras do projeto, carregue só os procedimentos necessários e escreva um contrato verificável. Execute implementar → testar → corrigir, guarde o estado fora da conversa e confira a entrega contra a spec. Comece com um agente; delegue quando o contexto ou as fronteiras do trabalho justificarem. Este capítulo reúne os caps. 4, 5, 9, 11, 12, 13 e 15 num roteiro de uso, sem acrescentar um framework.

Antes de abrir o loop

O primeiro loop precisa caber na sua capacidade de inspecionar o resultado. Escolha uma mudança com comportamento observável e fronteira clara. O cap. 8 trata das primeiras tasks; aqui o foco é montar o percurso que permite executar uma delas sem depender do "terminei" do agente.

O roteiro abaixo é uma costura editorial dos capítulos anteriores. Os nomes dos arquivos vêm do fluxo Spec-Driven apresentado no cap. 9; adapte a profundidade ao tamanho do trabalho, como na seção de auto-sizing daquele capítulo.

1. Prepare o contexto permanente — cap. 4

Revise o `AGENTS.md`. Ele deve dizer o necessário para trabalhar no projeto: stack, comandos reais, convenções que o agente não deduz do código e limites de atuação. Documentação extensa fica em arquivos apontados por ele.

Antes de delegar, confira se os comandos de desenvolvimento e validação descritos existem. Um comando inventado transforma a primeira execução numa investigação do ambiente. O checklist do Apêndice A serve para esta passagem.

Saída: regras curtas, comandos utilizáveis e caminhos explícitos para o contexto complementar.

2. Selecione o procedimento — cap. 5

Veja se existe uma skill adequada à tarefa. Leia sua description para entender o acionamento e seu corpo para conferir o procedimento. Se o trabalho exige uma referência, o corpo deve dizer quando carregá-la. Não instale um catálogo inteiro para executar uma mudança pequena.

Uma skill descreve como fazer uma classe de trabalho. Os arquivos da tarefa dizem qual mudança fazer agora. Manter esses papéis claros evita transformar o `SKILL.md` em memória de uma feature específica.

Saída: procedimento escolhido e referências necessárias identificadas, ou decisão de resolver diretamente sem skill.

3. Escreva o contrato — cap. 9

Transforme o pedido numa spec com comportamento esperado, restrições e critérios de sucesso. No fluxo completo, o `design.md` organiza a solução e o `tasks.md` divide a execução. A unidade executável precisa trazer **o quê, onde e DoD**.

Inclua os cenários de teste e o comando que comprova a conclusão. Se faltar uma decisão de produto ou arquitetura, registre a dúvida antes de deixar o implementador preenchê-la por conta própria. Research e planejamento devem produzir arquivos que possam atravessar uma troca de janela.

Saída: task delimitada, critérios verificáveis, arquivos ou módulos envolvidos e dependências conhecidas. O template do Apêndice A ajuda a conferir o contrato.

4. Escolha a forma de execução — cap. 11

Aplique a regra do cap. 11. Se a task cabe numa janela com folga para uma correção, comece com um agente. Se não cabe, procure blocos coesos com contrato claro. Blocos dependentes vão para sub-agents em sequência; os independentes podem rodar em paralelo.

Não use a quantidade de tasks como quantidade de agentes. A fronteira de acoplamento orienta a divisão. Quem recebe um bloco precisa do contrato e do contexto desse bloco, sem herdar toda a conversa de planejamento.

Saída: um agente ou um mapa de delegação que respeita as dependências. A árvore do Apêndice A reaproveita o D16.

5. Execute implementar → testar → corrigir — cap. 12

O agente implementa a task, executa os testes previstos, lê as falhas e corrige. Quando o roteiro adotar TDD (testes), preserve os critérios estabelecidos no red e faça a verificação exigida antes de concluir. O gate precisa executar e passar; teste apenas escrito não encerra o loop.

Se o comando não rodou por falta de ambiente, o resultado continua sem validação. Registre o impedimento. Se passou, guarde o comando e o resultado junto da entrega. Isso permite distinguir sucesso observado de confiança do implementador.

Saída: mudança dentro do escopo, testes executados e resultado dos gates.

6. Preserve o estado e limite a sessão — cap. 13

Ao longo da execução, atualize a memória persistente com decisões, progresso e blockers. O `STATE.md` permite que uma sessão seguinte comece do estado conhecido, em vez de reconstruir o trabalho pela conversa.

Use os problemas do cap. 13 como perguntas de controle: o agente tentou tudo de uma vez? Declarou vitória sem prova? Perdeu as decisões anteriores? Testou só uma aparência de funcionamento? Julgou o próprio trabalho? Deixou degradação acumulada? Cada resposta aponta para uma peça do harness que precisa atuar.

Saída: estado retomável, pendências explícitas e condição de parada respeitada. Um impedimento registrado é um resultado legítimo; não deve virar conclusão fictícia.

7. Confira contra os critérios — cap. 15

Separe a avaliação da implementação. O avaliador recebe os critérios e a entrega e procura evidência. Para cada check, a resposta precisa ser verificável: arquivo e linha quando a prova está no código, resultado de execução quando depende de comportamento.

O primeiro loop não exige um benchmark completo. Use a parte aplicável do cap. 15: checks binários, evidência e separação de papéis. Pesos, soma por script e múltiplos runs pertencem ao cenário de avaliação comparativa desenvolvido ali. Se esse for o objetivo, mantenha baseline e condições iguais entre execuções.

Saída: critérios atendidos ou não atendidos, sustentados por evidência; itens reprovados voltam como correções delimitadas. O resultado segue para o review do cap. 14, conforme o fluxo do cap. 16.

O que guardar ao encerrar

Artefato	Pergunta que ele responde
`AGENTS.md` e referências	Quais regras orientaram o trabalho?
Spec e task com DoD	O que deveria ser entregue?
Diff e testes	O que mudou e como foi exercitado?
Resultados dos gates	O que foi executado e passou?
Avaliação com evidência	Quais critérios a entrega atende?
`STATE.md`	O que falta e como retomar?

O conjunto permite inspecionar o loop depois de fechado. Se faltou um resultado, ele deve aparecer como pendência, sem ser substituído por uma frase otimista.

FAÇA

+ Percorra o roteiro com uma mudança pequena e confira os artefatos de cada passagem.
+ Defina DoD e comandos antes de implementar.
+ Comece com um agente e use a regra de acoplamento para decidir sobre delegação.
+ Devolva checks reprovados como correções delimitadas e verifique novamente os critérios afetados.

EVITE

– Trocar de ferramenta a cada falha sem investigar qual parte do loop faltou.
– Alterar o critério de aceite para acomodar uma implementação incompleta.
– Dar ao avaliador a justificativa do implementador como prova suficiente.
– Encerrar sem registrar o que não pôde ser validado.

ARMADILHA

Montar toda a infraestrutura antes de completar uma task. O livro apresenta peças para problemas de tamanhos diferentes. No primeiro loop, cada peça escolhida precisa resolver uma necessidade concreta: orientar, executar, observar, corrigir ou retomar. Complete um percurso verificável e use as falhas observadas para ajustar o harness.

CAPÍTULO 18

Kit de Skills

TL;DR

As cinco skills do pacotão cobrem exploração de código, domínios, acoplamento, comentários de PR e planejamento de migração. Escolha pela pergunta que precisa responder. Para decisões de arquitetura, a nota sobre Design It Twice acrescenta duas skills experimentais: `/domain-modeling` para afiar a linguagem do domínio e `/code-design` para explorar alternativas. O resultado desse processo é um RFC com opções e trade-offs, antes de construir.

Um kit organizado por necessidade

O cap. 5 explica a anatomia e o acionamento de skills. Aqui, o foco é o uso das habilidades listadas nas notas **Pacotão das Skills da TLC** e **Construindo Skills para Melhorar o Design da Arquitetura**. As descrições são as registradas no material do curso; o capítulo não pressupõe instalação nem reproduz os procedimentos internos dessas skills.

Skill	Pergunta que ajuda a responder	Entrega descrita na nota
CodeNavi	Como este codebase desconhecido está organizado?	exploração, mapeamento e documentação da estrutura
Domain Analysis	Quais domínios aparecem no código e como se relacionam?	mapas de domínio e sugestões de bounded contexts
Coupling Analysis	Quais módulos estão mais acoplados?	gráficos de dependência, mapas de módulos e força de integração
GitHub PR Review Comments (`gh-address-comments`)	Quais comentários do PR fazem sentido e como tratá-los?	análise, plano, correções e respostas explicando os pontos
Legacy Migration Planner	Como organizar uma migração de arquitetura?	análise de dependências e plano de migração

Explorar, entender domínios e medir acoplamento

CodeNavi atende à entrada num código desconhecido ou legado: explorar o repositório e documentar sua estrutura. Esse mapa ajuda a localizar onde pesquisar uma mudança. Ele não substitui a spec nem os critérios de validação da implementação.

Domain Analysis muda a pergunta da estrutura de pastas para o domínio. A nota descreve análise do código legado, identificação de domínios e do acoplamento entre eles, sugestões de bounded contexts e mapas de domínio. É uma visão macro antes de refatorar, citada como útil em

consultoria. Uma fronteira sugerida precisa ser confrontada com o significado do negócio; o mapa do código é uma entrada para essa conversa.

Coupling Analysis olha as dependências e a força de integração entre módulos. A nota a associa a *Balancing Coupling in Software Design*, de Vlad Khononov, e a gráficos que ajudam a identificar os módulos mais acoplados e priorizar refatorações. A conexão com o cap. 11 é direta: entender o acoplamento ajuda a enxergar onde mudanças compartilham decisões. Um mapa de dependência não é, sozinho, um mapa de execução paralela.

Essas três habilidades podem informar o mesmo trabalho sem que seja obrigatório executá-las em sequência. Se a dúvida é localizar código, comece pela exploração. Se a fronteira do negócio está confusa, a análise de domínio é relevante. Se o problema são dependências entre módulos, a análise de acoplamento traz o recorte adequado.

Tratar comentários e planejar migrações

A **GitHub PR Review Comments** busca comentários do PR, avalia se fazem sentido, propõe um plano de correção, implementa e responde explicando cada ponto. A avaliação vem antes da correção. Isso se conecta ao cap. 14: achados precisam ser conferidos contra o projeto, e as correções devem permanecer delimitadas. Usar essa skill não implica adotar a arquitetura inteira de review multiagente daquele capítulo.

A **Legacy Migration Planner** trata de decomposição de monólitos, migração para microserviços, dependências e planos de migração. O plano organiza a mudança; a implementação ainda precisa de recortes verificáveis como os caps. 9 e 12 ensinam. A existência da skill não é uma justificativa para migrar: o objetivo e as restrições precisam estar claros antes de escolher a solução.

Os links das cinco skills, do marketplace e do livro de Khononov estão no Apêndice C. Este capítulo descreve o material inventariado, sem acrescentar promessas sobre versões ou integrações.

Design It Twice: comparar antes de construir

Design It Twice é a prática de explorar alternativas de desenho antes de se comprometer com uma implementação, associada pela nota a John Ousterhout e a *A Philosophy of Software Design*. A aula aplica a ideia com IA e menciona um RFC real da TLC, mas a nota não registra seu conteúdo.

O material lista duas skills experimentais, inspiradas em Matt Pocock: `/domain-modeling` e `/code-design`. A primeira aparece ligada a afiar a linguagem do domínio. A segunda, à geração de alternativas de arquitetura, inclusive em paralelo. Não há instruções internas ou URLs próprias dessas duas skills no inventário.

A sequência registrada é curta:

1. **Afiar a linguagem do domínio com `/domain-modeling`.** Esclarecer os conceitos usados na discussão antes de comparar desenhos.
2. **Explorar três desenhos com três restrições diferentes.** Usar `/code-design` para gerar alternativas, que podem ser produzidas em paralelo.

3. **Comparar módulos profundos por depth, locality e seam.** Os termos são os eixos citados pela nota.
4. **Sintetizar alternativas e trade-offs num RFC.** Tornar explícito o que cada opção oferece e quais custos traz.

Como leitura editorial dos três eixos, *depth* é a profundidade do módulo, a relação entre o que ele encapsula e a simplicidade da interface; *locality* trata de quanto uma mudança fica localizada; *seam* é um ponto de separação onde partes podem ser isoladas ou substituídas. A nota só nomeia esses critérios: não fornece rubrica, pontuações nem um vencedor para o RFC mencionado.

Explorar alternativas em paralelo é diferente de implementar opções concorrentes no mesmo código. É trabalho de análise que pode ser consolidado num documento. A decisão de execução continua seguindo as fronteiras e dependências do cap. 11.

Do RFC à execução

O RFC guarda a discussão e os trade-offs, como no cap. 7. Quando uma decisão é tomada, seu resultado informa o trabalho seguinte. A spec delimita comportamento, o design detalha a solução necessária e as tasks tornam a implementação executável. A comparação de arquiteturas não pula essas passagens.

Este kit cobre momentos distintos desse percurso: entender o código, analisar limites, explorar opções, planejar uma transição e tratar feedback. Carregue a habilidade correspondente ao trabalho atual e preserve seus resultados em artefatos que a próxima etapa consiga usar.

FAÇA

+ Escolha a skill pela pergunta a responder e confira seu procedimento antes de usá-la.
+ Use mapas de domínio e dependência como entradas para decisões explícitas.
+ Compare alternativas de arquitetura e registre trade-offs no RFC.
+ Converta o desenho escolhido em spec, tasks e critérios verificáveis.

EVITE

– Confundir uma sugestão de bounded context com uma decisão de migração já aprovada.
– Corrigir todo comentário de PR sem avaliar sua pertinência.
– Supor comandos, arquivos de saída ou métricas não registrados para as skills experimentais.
– Tratar três alternativas geradas por IA como uma arquitetura automaticamente validada.

ARMADILHA

Escolher a ferramenta antes de formular a pergunta. Instalar uma skill de migração pode fazer qualquer problema parecer uma migração. Comece pelo que precisa descobrir ou decidir. A habilidade escolhida deve produzir informação útil para essa decisão, e o resultado precisa continuar sujeito à análise do time.

Apêndices

A Cheat sheets

B Glossário

C Referências

APÊNDICE A

Cheat sheets

Este apêndice reúne os pontos de consulta dos caps. 4, 5, 9, 11 e 12. Os templates são modelos editoriais para adaptar ao repositório. Substitua os campos entre `<...>` antes de usar; comandos e caminhos precisam corresponder ao projeto real.

Checklist do AGENTS.md — cap. 4

- [ ] Stack e versões relevantes identificadas.
- [ ] Comandos reais de desenvolvimento, build, lint e testes, com diretório de execução quando necessário.
- [ ] Regras específicas do projeto que não são óbvias pelo código.
- [ ] Convenções escritas de forma que possam ser verificadas.
- [ ] Limites separados em **sempre faça**, **pergunte antes** e **nunca faça**.
- [ ] Referências explícitas para arquitetura, domínio e procedimentos extensos, carregadas sob demanda.
- [ ] Regras antigas, redundantes ou contraditórias removidas.
- [ ] Texto revisado para permanecer curto, sem colar a documentação inteira no contexto permanente.

O cap. 4 explica as recomendações de tamanho divergentes das fontes. Use-as como orientação contextual; a finalidade é preservar instruções úteis com pouco ruído.

Template de SKILL.md — cap. 5

O exemplo descreve uma habilidade de análise, não uma skill instalada pelo livro. A description inclui **o quê + quando + sinônimos**; o corpo começa pelo fluxo principal.

```
---
name: mapear-modulo
description: >-
  Mapeia responsabilidades e dependências de um módulo existente.
  Use quando precisar entender um módulo antes de alterá-lo,
  explorar código legado ou localizar integrações e padrões de teste.
---
# Mapear módulo

## Regras críticas
- Sustente afirmações sobre o código com caminhos e trechos localizáveis.
- Registre dúvidas como dúvidas; não invente dependências.
```

```
- Mantenha a análise dentro do módulo solicitado e de suas integrações.

## Fluxo principal
1. Identifique o módulo solicitado e leia as regras do projeto.
2. Localize entradas, responsabilidades, dependências e testes existentes.
3. Para analisar integrações, leia references/integracoes.md.
4. Produza o mapa no formato abaixo e registre lacunas.

## Formato de saída
- Módulo e escopo examinado.
- Responsabilidades e pontos de entrada, com caminhos.
- Dependências e integrações, com evidência.
- Testes existentes e dúvidas abertas.

## Casos de borda
- Se houver mais de um módulo compatível com o pedido, explicite a
  ambiguidade antes de aprofundar a análise.
- Se não houver evidência suficiente, marque o item como não confirmado.
```

Crie `references/integracoes.md` com conteúdo pertinente ao projeto se mantiver esse passo. Referências não devem apontar para arquivos inexistentes. Acrescente `scripts/` para operações determinísticas e `assets/` quando houver templates ou arquivos de apoio; mantenha essas pastas apenas se forem necessárias.

Template de task com DoD — caps. 9 e 12

```
## <ID> — <resultado da task>

### O quê
<Comportamento a entregar e limites do escopo.>

### Contexto e requisito
- Spec: <caminho e IDs dos critérios de aceite>.
- Design: <caminho e decisões relevantes>.
- Regras e referências: <caminhos necessários para esta task>.

### Onde
- Criar: <arquivos previstos, ou nenhum>.
- Modificar: <arquivos/módulos previstos>.
- Testes: <arquivo de teste e comportamento observado>.

### Dependências e execução
- Depende de: <IDs anteriores, ou nenhuma>.
- Pode rodar em paralelo com: <IDs independentes, ou nenhuma>.
- Estado compartilhado que impede paralelismo: <qual, ou nenhum>.

### Cenários de teste
- <Cenário de sucesso, com entrada e resultado esperado>.
- <Erro ou caso de borda exigido pela spec>.

### Definition of Done (DoD)
- [ ] Critérios <IDs> implementados dentro do escopo.
```

```
- [ ] Arquivos, tipos exportados e configurações previstos conferidos.
- [ ] Testes dos cenários acima implementados e executados.
- [ ] Gate: em <diretório>, executar <comando real de testes>; saída 0.
- [ ] Demais gates aplicáveis: <comandos reais e resultado esperado>.
- [ ] Evidência registrada: comando, resultado e arquivos relevantes.
- [ ] STATE.md atualizado com decisões e impedimentos, se houver.

### Resultado da execução
- Arquivos alterados: <lista>.
- Comandos executados e resultados: <registro>.
- Critérios sem validação ou blockers: <lista, ou nenhum>.
```

O gate é uma condição de conclusão. Não executou ou falhou: a task não pode ser marcada como validada. Ajuste o template ao tipo de mudança; não invente comandos para preencher campos.

Árvore de decisão de sub-agent — cap. 11

1. **A tarefa cabe numa janela com folga para correção?**
   - **Sim:** um agente single-thread é o padrão. Paralelizar partes independentes é opcional.
   - **Não:** examine a fronteira de acoplamento.
2. **Há blocos coesos com contrato claro?**
   - **Não:** reduza o escopo ou reorganize o trabalho antes de dividir.
   - **Sim, dependem uns dos outros:** sub-agents em sequência, preservando contexto.
   - **Sim, são independentes:** sub-agents em paralelo, preservando contexto e ganhando velocidade.

O D16 abaixo é o mesmo do cap. 11. A árvore em texto permite consultar a regra junto dos templates.

Antes de chamar dois blocos de independentes, confira se compartilham arquivos, estado de teste ou decisões. Um sub-agent por task não é a regra: agrupe fases coesas.

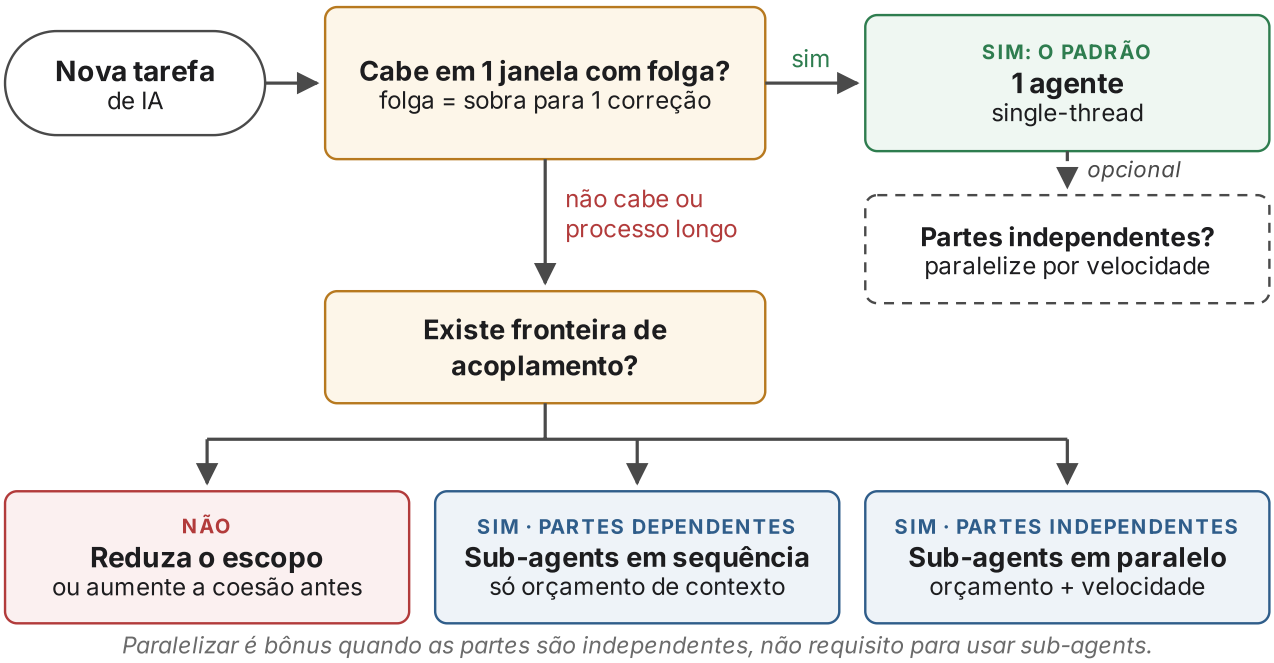


Figura A.1 Regra de decisão de sub-agents, reaproveitada do cap. 11: janela com folga e fronteiras de acoplamento orientam a divisão

APÊNDICE B

Glossário

Termos reunidos durante a escrita, em ordem alfabética. As indicações de capítulo apontam para o contexto de uso; "Complemento" identifica o texto externo dos caps. 10 e 15. Design Doc e TDD (testes) são entradas distintas.

- **acoplamento** (cap. 11) — o quanto as subtarefas dependem umas das outras; a variável que decide entre um agente, sub-agents em sequência ou em paralelo.
- **ADR (Architecture Decision Record)** (cap. 7) — registro imutável de uma decisão já tomada, com contexto e motivo.
- **agente** (cap. 1, 10) — programa que orquestra o loop (Claude Code, Cursor, Copilot): monta o contexto, chama o modelo e executa as tools que ele pede. Ver também *modelo*.
- **AGENTS.md** (cap. 4) — padrão aberto de arquivo de instruções para agentes, na raiz do repositório, enviado em toda requisição. Equivalentes: rules do Cursor, `CLAUDE.md`.
- **AI Slop Acumulado** (cap. 13) — problema 6: degradação silenciosa da qualidade a cada sessão.
- **Amnésia entre Sessões** (cap. 13) — problema 3: cada sessão começa do zero.
- **attention** (cap. 1) — mecanismo que mede quanto cada token se relaciona com os outros; o núcleo do cálculo do modelo.
- **attention budget** (cap. 10, Complemento) — "orçamento de atenção" do modelo: cada token a mais no contexto gasta um pouco dele.
- **auto-sizing** (cap. 9) — ajuste da profundidade do fluxo ao tamanho do trabalho.
- **back pressure** (cap. 15, Complemento) — tudo o que rejeita código inválido a cada volta do loop (testes, tipos, scanners).
- **banco de vetores** (cap. 6) — banco que guarda embeddings e devolve os trechos de significado mais próximo (busca semântica).
- **baseline** (cap. 15) — conjunto congelado de checks binários contra o qual todos os runs são avaliados.
- **blockers** (cap. 9, 11, 13) — impedimentos registrados no `STATE.md`.
- **boilerplate** (cap. 9) — estrutura repetitiva que o framework gera.
- **bootstrap** (cap. 13) — o que prepara cada sessão para começar sem redescobrir o projeto (`init.sh`, leitura do progresso e do `git log`).
- **boundaries** (cap. 4) — limites em três níveis: sempre faça, pergunte antes, nunca faça.
- **bounded context** (cap. 2) — fronteira de um subdomínio do negócio; base para organizar pastas por domínio.

- **breadth-first** (cap. 11) — exploração em largura: muitas frentes independentes ao mesmo tempo.
- **breakdown** (cap. 7) — quebra do trabalho em tasks.
- **brownfield** (cap. 9) — sistema que já existe (oposto de projeto novo).
- **cadeia de conhecimento (Knowledge Chain)** (cap. 9) — ordem de busca quando falta informação: codebase, docs, Context7, web; se ainda houver dúvida, marcar como incerto.
- **check binário** (cap. 15) — pergunta de resposta sim ou não (0 ou 1).
- **cloud agent** (cap. 16) — agente que executa o trabalho em um ambiente remoto.
- **código de saída / exit code** (cap. 12) — número que um comando devolve ao terminar: 0 = sucesso, diferente de 0 = falha.
- **commit atômico** (cap. 9) — uma task, um commit, uma verificação.
- **compaction** (cap. 10, Complemento) — resumir a conversa perto do limite e recomeçar numa janela nova com o resumo.
- **computational / inferential** (cap. 10, Complemento Böckeler) — controles determinísticos (testes, lint) vs semânticos e não determinísticos (review por IA, LLM as judge).
- **concerns.md** (cap. 9) — pontos de atenção do codebase levantados no mapeamento.
- **condição de parada / stopping condition** (cap. 10) — o que encerra o loop além do "terminei" do modelo: gate, limite de iterações, decisão humana.
- **constraints** (cap. 2) — restrições explícitas que guiam a saída (bibliotecas, performance, compatibilidade, estilo, acessibilidade).
- **context dilution** (cap. 1, 3, 10) — partes relevantes se perdem quando o contexto fica grande demais.
- **context engineering** (cap. 2, 11) — montar todo o pacote que o modelo recebe além do prompt: arquivos, erros, versões, convenções, codebase.
- **context rot** (cap. 2; cap. 10, Complemento) — queda de precisão da recuperação de informação à medida que o contexto cresce.
- **Context7** (cap. 4, 8) — MCP que busca documentação atualizada de bibliotecas de terceiros.
- **contexto** (cap. 1) — tudo o que o modelo recebe numa chamada.
- **Cry Wolf** (cap. 14) — efeito de falsos positivos em excesso: o time passa a ignorar todos os comentários.
- **Definition of Done (DoD)** (cap. 9, 12) — critérios objetivos para considerar uma task concluída, incluindo os gates.
- **delta (OpenSpec)** (cap. 9) — registro da mudança na spec: como era, por que mudou, como ficou.
- **deny-list** (cap. 14) — lista de comandos que o agente é proibido de rodar (ex.: `git commit`, `git push`).
- **depth** (cap. 18) — profundidade do módulo: relação entre o que ele encapsula e a simplicidade da interface; leitura editorial do eixo citado na nota.
- **description (da skill)** (cap. 5) — a linha que decide o acionamento: o quê, quando e sinônimos.

- **Design Doc (Technical Design Doc)** (cap. 7) — documento técnico do dev, depois da Discovery: o como. Não confundir com **TDD (testes)** nem com o `design.md` do Spec-Driven.
- **Design It Twice** (cap. 18) — explorar alternativas de desenho antes de escolher uma implementação; prática associada nas notas a John Ousterhout.
- **Discovery** (cap. 7) — fase guiada por humanos (produto e dev) em que se entende o problema; a IA ajuda pouco.
- **documentation first** (cap. 2, 4) — escrever uma vez, num arquivo de regras, o contexto que o agente carrega sozinho.
- **edge cases** (cap. 9) — casos de borda.
- **embedding** (cap. 1, 6) — representação de um token ou texto como vetor numérico; base da busca semântica.
- **end-to-end (E2E)** (cap. 4, 12, 13) — teste do fluxo completo, como um usuário faria (ex.: no browser).
- **enforcement** (cap. 12) — mecanismo que faz uma regra valer (um passo que bloqueia), em vez de só escrevê-la.
- **evaluator-optimizer** (cap. 10, Complemento) — padrão em que um modelo gera e outro avalia e devolve feedback, em loop.
- **feature list** (cap. 13, 15) — lista de features com status (passa / falha) que define o que é "pronto".
- **few-shot** (cap. 2) — dar poucos exemplos concretos para o modelo inferir o padrão.
- **Fix mode** (cap. 13) — volta do trabalho ao Generator com o que o Evaluator reprovou.
- **frontmatter** (cap. 5) — cabeçalho YAML do `SKILL.md` com `name` e `description`; sempre carregado.
- **gap detection** (cap. 14) — listar os arquivos que nenhum agente comentou.
- **grounding** (cap. 14) — ancorar o review nos docs do projeto antes de olhar o diff.
- **guardrails** (cap. 12) — limites que guiam o agente sem restringir o que ele consegue fazer.
- **guides / feedforward** (cap. 10, 13) — controles que orientam o agente antes de agir: AGENTS.md, specs, tasks, docs de arquitetura, convenções.
- **hardcoded** (cap. 12) — fixo no código, sem detecção do contexto real.
- **harness** (cap. 10, 13) — tudo num agente exceto o modelo: guides, sensors, memória e bootstrap. "Agent = Model + Harness".
- **harness engineering** (cap. 13) — projetar a instrumentação em volta do modelo para guiá-lo, constrangê-lo e validá-lo.
- **hooks** (cap. 6) — gatilhos que rodam automações em eventos da sessão do agente.
- **human-in-the-loop** (cap. 14) — o humano decide em pontos definidos do loop; a IA sugere.
- **initializer agent / coding agent** (cap. 15, Complemento) — agente da primeira sessão (prepara o ambiente) / agente das sessões seguintes (progresso incremental).
- **inline** (cap. 14) — comentário postado na linha do código, dentro do PR.
- **input / output** (cap. 1) — o texto que entra no modelo / o que sai dele.
- **janela de contexto** (cap. 1, 3) — o limite de tokens que o modelo considera numa chamada.

- **just-in-time retrieval** (cap. 10, Complemento) — guardar referências leves (caminhos, queries) e carregar o conteúdo só quando necessário.
- **Lean Inception** (cap. 7) — workshop curto de Discovery para alinhar escopo, jornadas e funcionalidades priorizadas por esforço e valor.
- **LLM (Large Language Model)** (cap. 1) — modelo de linguagem de grande escala: uma máquina de prever o próximo token, treinada com muito texto.
- **LLM as judge** (cap. 10, 15) — usar um modelo para avaliar a saída de outro modelo ou agente.
- **locality** (cap. 18) — quanto uma mudança fica localizada; leitura editorial do eixo citado na nota.
- **loop agêntico** (cap. 10) — ciclo contexto → decisão do modelo → tool → resultado → contexto, repetido até a condição de parada.
- **Loop Engineering** (cap. 10, 15) — projetar o sistema que dispara, verifica e dirige os agentes, em vez de dar os prompts você mesmo.
- **mapa de execução paralela** (cap. 9, 11) — parte do `tasks.md` que diz quais tasks rodam em paralelo e quais dependem de outras.
- **mapeamento de codebase** (cap. 9) — exploração inicial, por sub-agents, de dependências, arquitetura, convenções e padrões de teste.
- **Max Mode** (cap. 3, 6) — modo do Cursor com janelas grandes (1 milhão de tokens).
- **MCP (Model Context Protocol)** (cap. 6) — protocolo aberto para expor ao agente tools que falam com sistemas externos, com autenticação segura.
- **meta-review** (cap. 14) — agente dedicado a detectar alucinações da própria IA.
- **modelo** (cap. 1, 10) — a LLM que decide a próxima ação (Sonnet, GPT, Gemini, Composer). Não executa nada; só pede.
- **monólito modular** (cap. 2) — um só repositório e deploy, dividido em módulos por domínio; permite mudanças atômicas entre módulos.
- **monorepo** (cap. 16) — repositório que reúne os projetos; citado como parte do ambiente no relato de Frances Thai.
- **onboarding (da IA)** (cap. 4) — instrumentar o projeto para a IA: stack, arquitetura, regras e onde encontrar o resto.
- **one-shot** (cap. 11) — fazer o trabalho de uma vez só, sem iteração.
- **One-Shot Hero** (cap. 13) — problema 1: o agente tenta fazer tudo de uma vez e estoura a janela.
- **orchestrator-workers** (cap. 10, 11) — um agente central divide o trabalho, delega a workers e junta os resultados.
- **orquestrador** (cap. 11) — o agente principal que delega a sub-agents e mantém a própria janela baixa.
- **outcome** (cap. 15, Complemento) — o estado final do ambiente, não o que o agente diz que fez.
- **parallelization** (cap. 10, Complemento) — padrão de rodar subtarefas independentes ao mesmo tempo (por seções) ou a mesma tarefa várias vezes (votação).
- **pass@k / pass^k** (cap. 15, Complemento) — chance de acertar ao menos uma vez em k tentativas / de acertar todas as k.

- **persona** (cap. 2) — pedir que o modelo "aja como" alguém específico para afunilar tom e profundidade.
- **phantom imports** (cap. 14) — imports de módulos ou funções que não existem.
- **pinar (o modelo)** (cap. 15) — fixar o modelo usado por um agente, independentemente do modelo da sessão.
- **Plan Mode** (cap. 8) — modo em que o agente investiga e propõe um plano sem alterar arquivos.
- **Planner / Generator / Evaluator** (cap. 13) — três agentes em processos separados: planeja a spec, implementa, avalia.
- **PRD (Product Requirements Document)** (cap. 7) — documento de produto: o quê e por quê; problema, objetivos e requisitos.
- **Precision > Recall** (cap. 14) — preferir não reportar a reportar errado (limiar de confiança).
- **progressive disclosure** (cap. 2, 4, 5) — revelação progressiva: o texto sempre carregado aponta para documentos lidos só quando a tarefa pede.
- **prompt** (cap. 2) — o pedido que você escreve para o modelo.
- **prompt engineering** (cap. 2) — técnicas para escrever bons pedidos: exemplos, persona, restrições, iteração, validação.
- **quality gate** (cap. 9, 12) — verificação binária que bloqueia a conclusão de uma task até passar (ex.: comando de testes com código de saída 0).
- **Quick Mode** (cap. 9) — modo do TLC Spec-Driven para tarefas pequenas (até 3 arquivos): descrever, implementar, verificar, commitar.
- **RAG (Retrieval-Augmented Generation)** (cap. 1, 6) — buscar os trechos relevantes de uma base e colocar só eles no contexto.
- **rastreabilidade** (cap. 9) — IDs de requisito que atravessam spec, design, tasks e validação.
- **red / green / refactor** (cap. 12) — as três fases do TDD (testes): teste falhando, implementação mínima que passa, melhoria do código.
- **references / scripts / assets** (cap. 5) — pastas opcionais da skill: documentos sob demanda / código executável / templates e arquivos de apoio.
- **regra testável** (cap. 4) — regra que dá para conferir olhando o código (ex.: "valores monetários usam `Decimal`").
- **RFC (Request for Comments)** (cap. 7) — proposta aberta para discutir uma decisão técnica antes de tomá-la.
- **RPI (Research, Plan, Implement)** (cap. 9) — pesquisar com modelo forte e janela grande, salvar o plano em arquivo, implementar numa janela nova só com o plano.
- **rules** (cap. 2, 4) — arquivo de regras do projeto (no Cursor); o mesmo papel do `AGENTS.md`.
- **run** (cap. 15) — uma execução completa de uma configuração (framework, modelo, tarefa).
- **scope creep** (cap. 12) — escopo crescendo sem ninguém pedir.
- **seam** (cap. 18) — ponto de separação onde partes podem ser isoladas ou substituídas; leitura editorial do eixo citado na nota.

- **second pass** (cap. 14) — segunda leitura obrigatória do diff, listando o que não foi comentado e por quê.
- **self-enhancement bias (autopreferência)** (cap. 15) — tendência de um juiz a favorecer respostas geradas por ele mesmo.
- **sensors / feedback** (cap. 10, 13) — controles que observam depois da ação e ajudam a corrigir: linters, type checkers, testes, agente de review.
- **shift-left** (cap. 12) — trazer qualidade, segurança, padrões e arquitetura para o momento do desenvolvimento, em vez de deixá-los para o review.
- **single-thread** (cap. 11) — um agente só, numa linha de execução, sem dividir o trabalho.
- **skill** (cap. 5) — pasta com um `SKILL.md` (nome, description, instruções) e, opcionalmente, scripts, referências e assets; carregada sob demanda quando a tarefa casa com a description.
- **skill-architect** (cap. 5) — skill que ajuda a criar skills.
- **Smart Zone / zona de atenção / Dumb Zone** (cap. 3) — faixas da janela ocupada: até ~40% qualidade alta; 40–60% qualidade já degradada; acima de 60% alta chance de alucinação.
- **Spec** (cap. 7, 9) — contrato exato do que será entregue, escrito para a LLM.
- **Spec-Driven Development** (cap. 9) — fluxo em que o problema vira spec, design e tasks atômicas com DoD antes da implementação.
- **Specify / Design / Tasks / Execute** (cap. 9) — as quatro fases do TLC Spec-Driven (`spec.md`, `design.md`, `tasks.md`, commits).
- **sprint contract** (cap. 15, Complemento) — acordo entre generator e evaluator sobre o que é "pronto" antes de cada sprint.
- **stack trace** (cap. 2) — a pilha de chamadas registrada quando um erro acontece.
- **stakeholder** (cap. 16) — pessoa interessada no resultado, que traz o problema e ajuda a esclarecer os critérios de sucesso.
- **state files** (cap. 15, Complemento) — arquivos que guardam o estado de um loop entre execuções.
- **STATE.md** (cap. 9, 11, 13) — memória persistente entre sessões: decisões, blockers, aprendizados, ideias para depois.
- **stateless** (cap. 1, 10) — sem estado: o modelo não lembra de nada entre chamadas; o agente reenvia o contexto inteiro a cada volta.
- **structured note-taking** (cap. 10, Complemento) — notas persistidas fora da janela pelo próprio agente (papel do `STATE.md`).
- **sub-agent** (cap. 5, 11) — agente disparado por outro, com janela de contexto própria, que devolve só o resultado.
- **success criteria** (cap. 9) — critérios de sucesso da spec.
- **sweet spot** (cap. 11) — ponto de melhor equilíbrio; no benchmark, 3 sub-agents por blocos de fases.
- **TDD (testes)** (cap. 7, 12) — *Test-Driven Development*: escrever o teste antes do código, no ciclo red → green → refactor. Não confundir com **Design Doc**.

- **testcontainers** (cap. 12) — bancos e serviços descartáveis em contêiner, um por execução de teste.
- **Teste Fake** (cap. 13) — problema 4: feature marcada como pronta sem teste real de ponta a ponta.
- **thinking** (cap. 6) — nível de esforço de raciocínio configurável do modelo.
- **token / tokenização** (cap. 1) — pedaço de texto que o modelo lê (palavra, parte de palavra, pontuação) / a etapa que quebra o texto nesses pedaços. Custo e janela são medidos em tokens.
- **tool / function** (cap. 6, 10) — ação que o modelo pode pedir e o agente executa (ler arquivo, editar, rodar comando, chamar MCP).
- **transformer** (cap. 1) — a arquitetura dos modelos de linguagem atuais.
- **trial** (cap. 15, Complemento) — cada tentativa de uma tarefa numa avaliação.
- **Tudo no Mesmo Processo / viés do implementador** (cap. 13) — problema 5: quem implementa é quem julga.
- **upfront** (cap. 9) — logo no início, antes de começar.
- **user story** (cap. 9) — história de usuário; na spec, priorizada (P1, P2, P3) e escrita em WHEN / THEN / SHALL. Vira teste e critério de review.
- **usuário / builder** (cap. 1) — quem usa bem modelos, agentes e editores para entregar o próprio trabalho / quem constrói agentes.
- **verifier** (cap. 16) — papel que confere a entrega contra a spec e procura evidência para os critérios de aceite.
- **verifier theater** (cap. 15, Complemento) — verificador que aprova sem checar de verdade.
- **verify** (cap. 12) — passo de verificação que substitui o refactor no ciclo red → green → verify da mesma task.
- **vibe coding** (cap. 1) — pedir, aceitar e seguir em frente sem olhar muito o que saiu; o oposto do desenvolvimento assistido por IA.
- **viés de posição / de verbosidade** (cap. 15, Complemento) — juiz que favorece a resposta pela ordem em que aparece / por ser mais longa.
- **Vitória Prematura** (cap. 13) — problema 2: o agente declara “pronto” com progresso parcial.
- **WHEN / THEN / SHALL** (cap. 9) — formato verificável de requisito: quando X, então o sistema deve Y.
- **worker** (cap. 11) — sub-agent que executa um bloco de trabalho delegado pelo orquestrador.
- **workflow vs agent** (cap. 10, Complemento) — caminho definido em código vs caminho decidido pelo modelo.
- **worktree** (cap. 15, Complemento) — cópia de trabalho separada do mesmo repositório git, para agentes em paralelo não interferirem.

APÊNDICE C

Referências

Links das notas internas e fontes externas do plano, agrupados por capítulo. Uma fonte pode aparecer em mais de um capítulo quando atende a usos distintos. Os caps. 3, 17 e os apêndices retomam o material dos demais capítulos, sem links próprios no inventário.

As quinze fontes dos Complementos foram consultadas na E2, em 19/10/2026; suas entradas registram essa data. As demais referências foram inventariadas nas notas ou no plano. A inclusão aqui não indica uma nova consulta ao conteúdo ou uma verificação de disponibilidade.

Acesso: páginas da comunidade TLC podem exigir login; os três links do repositório fakeflix e o formulário de acesso dependem do acesso ao curso. Os quatro boards do Excalidraw são compartilhados por link. O perfil de Frances Thai não substitui o vídeo da conversa, cuja URL não consta na nota.

URLs normalizadas: os dois links genéricos de Harness Engineering foram substituídos pelas páginas de Building effective agents e Effective context engineering for AI agents; o Agent SDK usa o destino de redirecionamento registrado na E2; TLC Spec-Driven usa `/skills/tlc-spec-driven/`; Superpowers aponta para o repositório próprio. A checagem HTTP completa pertence à E9.

Cap. 1 — LLM e agentes

- Pesquisa Pragmatic Engineer. https://newsletter.pragmaticengineer.com/p/ai-tooling-2026
- Meetup IA: Prompt Engineering — comunidade; pode exigir login. https://www.techleads.club/c/eventos-da-comunidade/meetup-ia-prompt-engineering
- [Aula] MCP – the good parts — comunidade; pode exigir login. https://www.techleads.club/c/aulas-da-comunidade/aula-model-context-protocol-mcp-the-good-parts

Cap. 2 — Prompt e Context Engineering

- AGENTS.md. https://agents.md/

Cap. 4 — AGENTS.md / Rules

- Diagramas das aulas — board compartilhado por link. https://link.excalidraw.com/l/7V6DWtFSy3p/4F49Nw2eyZq

- AGENTS.md e CLAUDE.md são Inúteis? — comunidade; pode exigir login. https://www.techleads.club/c/blog/agents-md-e-claude-md-sao-inuteis
- AGENTS.md e Agent Skills: Por Debaixo dos Panos — comunidade; pode exigir login. https://www.techleads.club/c/blog/agents-md-e-agent-skills-por-debaixo-dos-panos-dos-ai-coding-agents
- AGENTS.md vs Skills — comunidade; pode exigir login. https://www.techleads.club/c/eventos-da-comunidade/agents-md-vs-skills-como-estruturar-o-conhecimento-dos-seus-agentes-de-ia
- padrão aberto. https://agents.md/
- Context7. https://context7.com/
- Context7 MCP no GitHub. https://github.com/upstash/context7

Cap. 5 — Skills

- Agent Skills, spec aberta. https://agentskills.io/
- documentação Anthropic. https://platform.claude.com/docs/en/agents-and-tools/agent-skills/overview
- post da Anthropic. https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills
- skill-architect. https://agent-skills.techleads.club/skills/skill-architect/
- marketplace TLC. https://agent-skills.techleads.club/
- TLC Agent Skills no GitHub. https://github.com/tech-leads-club/agent-skills

Cap. 6 — Tools, MCP e RAG

- spec oficial do MCP. https://modelcontextprotocol.io/
- Playwright MCP. https://github.com/microsoft/playwright-mcp
- Anthropic — Writing effective tools for agents. https://www.anthropic.com/engineering/writing-tools-for-agents

Cap. 7 — Fluxo de desenvolvimento

- Lean Inception, Paulo Caroli. https://caroli.org/en/lean-inception-4/
- Lean Inception, martinfowler.com. https://martinfowler.com/articles/lean-inception/
- skill Technical Design Doc Creator. https://agent-skills.techleads.club/skills/technical-design-doc-creator/

Cap. 8 — Primeiras tasks

- Atlassian MCP (Jira e Confluence). https://github.com/atlassian/atlassian-mcp-server
- formulário de acesso ao fakeflix — acesso do curso. https://www.techleads.club/c/curso-aplicacoes-node-js-enterprise/formulario-de-acesso-ao-repositorio-do-curso

Cap. 9 — RPI e Spec-Driven

- vídeo completo sobre RPI. https://www.youtube.com/watch?v=8fZUfVFWT4M
- TLC Spec-Driven. https://agent-skills.techleads.club/skills/tlc-spec-driven/
- [Aula] Spec Driven e eficiência de contexto — comunidade; pode exigir login. https://www.techleads.club/c/aulas-semanais/aula-spec-driven-e-eficiencia-de-contexto-em-escala-em-migracao-de-codigo-com-ia
- [Aula] Testes com IA — comunidade; pode exigir login. https://www.techleads.club/c/aulas-semanais/aula-testes-eficientes-com-ia
- Superpowers. https://github.com/obra/superpowers
- OpenSpec. https://github.com/Fission-AI/OpenSpec
- OpenSpec, site. https://openspec.dev/
- Spec Kit. https://github.com/github/spec-kit
- comparação de ferramentas SDD. https://www.augmentcode.com/tools/best-spec-driven-development-tools

Cap. 10 — Anatomia do loop agêntico

- Birgitta Böckeler — Harness engineering for coding agent users (02/04/2026; acesso em 19/10/2026). https://martinfowler.com/articles/harness-engineering.html
- Jeremy Hadfield et al. (Anthropic) — How we built our multi-agent research system (13/06/2025; acesso em 19/10/2026). https://www.anthropic.com/engineering/multi-agent-research-system
- Applied AI team (Anthropic) — Effective context engineering for AI agents (29/09/2025; acesso em 19/10/2026). https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents
- Erik Schluntz e Barry Zhang (Anthropic) — Building effective agents (19/12/2024; acesso em 19/10/2026). https://www.anthropic.com/engineering/building-effective-agents
- Thariq Shihipar — Building agents with the Claude Agent SDK (29/09/2025; acesso em 19/10/2026). https://claude.com/resources/articles/building-agents-with-the-claude-agent-sdk
- Simon Willison — I think “agent” may finally have a widely enough agreed upon definition to be useful jargon now (18/09/2025; acesso em 19/10/2026). https://simonwillison.net/2025/Sep/18/agents/

Cap. 11 — Sub-agents

- aula completa — comunidade; pode exigir login. https://www.techleads.club/c/aulas-semanais/aula-quando-usar-sub-agents-custo-e-performance
- Excalidraw da aula — board compartilhado por link. https://link.excalidraw.com/l/7V6DWtFSy3p/6vWl9PlXgD4
- Cognition, Walden Yan, 12/06/2025. https://cognition.com/blog/dont-build-multi-agents

- Jeremy Hadfield et al. (Anthropic) — How we built our multi-agent research system (13/06/2025; acesso em 19/10/2026). https://www.anthropic.com/engineering/multi-agent-research-system
- LangChain — How and when to build multi-agent systems. https://www.langchain.com/blog/how-and-when-to-build-multi-agent-systems

Cap. 12 — Loop de implementação e testes

- Excalidraw da aula — board compartilhado por link. https://link.excalidraw.com/l/7V6DWtFSy3p/1Ns0Pwvt2qL
- Excalidraw Spec-Driven Development — board compartilhado por link. https://link.excalidraw.com/l/7V6DWtFSy3p/Alq59lJUeEi
- código fakeflix — acesso do curso. https://github.com/tech-leads-club/fakeflix/tree/elevate-1-6
- Superpowers Marketplace. https://github.com/obra/superpowers-marketplace
- GSD. https://github.com/gsd-build/get-shit-done
- GSD 2. https://github.com/gsd-build/gsd-2
- Martin Fowler sobre computação não determinística. https://newsletter.pragmaticengineer.com/p/martin-fowler

Cap. 13 — Harness Engineering

- Spec Driven chegou no limite. https://www.youtube.com/watch?v=dLs-Pbn8stU
- Scaling Managed Agents. https://www.anthropic.com/engineering/managed-agents
- Prithvi Rajasekaran (Anthropic) — Harness design for long-running application development (24/03/2026; acesso em 19/10/2026). https://www.anthropic.com/engineering/harness-design-long-running-apps
- Applied AI team (Anthropic) — Effective context engineering for AI agents (29/09/2025; acesso em 19/10/2026). https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents
- Erik Schluntz e Barry Zhang (Anthropic) — Building effective agents (19/12/2024; acesso em 19/10/2026). https://www.anthropic.com/engineering/building-effective-agents
- Ryan Lopopolo (OpenAI) — Harness engineering: leveraging Codex in an agent-first world. https://openai.com/index/harness-engineering/

Cap. 14 — Code review

- PR mostrado na aula — acesso do curso. https://github.com/tech-leads-club/fakeflix/pull/90
- Code Review for Claude Code. https://claude.com/blog/code-review
- documentação. https://code.claude.com/docs/en/code-review
- plugin. https://claude.com/plugins/code-review
- uReview, Uber. https://www.uber.com/en-US/blog/ureview/

- How Uber uses AI. https://newsletter.pragmaticengineer.com/p/how-uber-uses-ai-for-development
- How Uber Engineers Use AI Agents. https://shiftmag.dev/how-uber-engineers-use-ai-agents-8617/
- OpenReview. https://github.com/vercel-labs/openreview
- Walden Yan (Cognition) — Multi-Agents: What's Actually Working (22/04/2026; acesso em 19/10/2026). https://cognition.com/blog/multi-agents-working

Cap. 15 — Harness determinístico

- PRD e baseline do benchmark — acesso do curso. https://github.com/tech-leads-club/fakeflix/tree/bench-base/.bench/stripe-trial
- vídeo do benchmark de frameworks spec-driven. https://youtu.be/gK1atE0ssfg
- [Aula] Comparação de modelos — comunidade; pode exigir login. https://www.techleads.club/c/aulas-semanais/aula-comparacao-de-modelos-fable-opus-sonnet-gemini-gpt-e-composer
- Chip Huyen, *AI Engineering*. https://www.goodreads.com/book/show/216848047-ai-engineering
- Prithvi Rajasekaran (Anthropic) — Harness design for long-running application development (24/03/2026; acesso em 19/10/2026). https://www.anthropic.com/engineering/harness-design-long-running-apps
- Justin Young (Anthropic) — Effective harnesses for long-running agents (26/11/2025; acesso em 19/10/2026). https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents
- Jeremy Hadfield et al. (Anthropic) — How we built our multi-agent research system (13/06/2025; acesso em 19/10/2026). https://www.anthropic.com/engineering/multi-agent-research-system
- Mikaela Grace et al. (Anthropic) — Demystifying evals for AI agents (09/01/2026; acesso em 19/10/2026). https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents
- Walden Yan (Cognition) — Multi-Agents: What's Actually Working (22/04/2026; acesso em 19/10/2026). https://cognition.com/blog/multi-agents-working
- Addy Osmani — Loop Engineering (07/06/2026; acesso em 19/10/2026). https://addyosmani.com/blog/loop-engineering/
- Geoffrey Huntley — Ralph Wiggum as a "software engineer" (14/07/2025; acesso em 19/10/2026). https://ghuntley.com/ralph/
- Jai Lal Lulla et al. — Loop Engineering: Building Blocks, Adoption, and Impact (22/08/2026; estudo exploratório, preprint em avaliação; acesso em 19/10/2026). https://arxiv.org/abs/2608.21884
- Lianmin Zheng et al. — Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena (09/06/2023; acesso em 19/10/2026). https://arxiv.org/abs/2306.05685
- Hamel Husain — Using LLM-as-a-Judge For Evaluation: A Complete Guide (29/10/2024; atualizado em 01/09/2026; acesso em 19/10/2026). https://hamel.dev/blog/posts/llm-judge/

- Eugene Yan — Evaluating the Effectiveness of LLM-Evaluators. https://eugeneyan.com/writing/llm-evaluators/

Cap. 16 — Fluxo de ponta a ponta

- Frances Thai. https://www.linkedin.com/in/frances-thai/
- aula Code Review Escalável — comunidade; pode exigir login. https://www.techleads.club/c/ia-first-curso-desenvolvimento-com-ia/sections/982679/lessons/3785892
- TLC Spec-Driven — caminho normalizado conforme o inventário. https://agent-skills.techleads.club/skills/tlc-spec-driven/

Cap. 18 — Kit de Skills

- Marketplace TLC Agent Skills. https://agent-skills.techleads.club/
- Coupling Analysis. https://agent-skills.techleads.club/skills/coupling-analysis/
- Domain Analysis. https://agent-skills.techleads.club/skills/domain-analysis/
- CodeNavi. https://agent-skills.techleads.club/skills/codenavi/
- GitHub PR Review Comments. https://agent-skills.techleads.club/skills/gh-address-comments/
- Legacy Migration Planner. https://agent-skills.techleads.club/skills/legacy-migration-planner/
- Vlad Khononov — Balancing Coupling in Software Design. https://www.amazon.com/Balancing-Coupling-Software-Design-Addison-Wesley/dp/0137353480
- [Aula] Design It Twice — comunidade; pode exigir login. https://www.techleads.club/c/aulas-semanais/aula-design-it-twice-explorando-arquiteturas-com-ai-antes-de-construir